



# Flash QSPI Controller IP

Part Number: IP6514E

NDA # \_\_\_\_\_

**User Guide**

**Revision 2.2**

CADENCE CONFIDENTIAL

## Confidentiality Notice

---

### Cadence Design Systems, Inc. San Jose, CA 95134

© 1996-2015 Cadence Design Systems, Inc. All rights reserved.

Cadence Design Systems, Inc. (Cadence), 2655 Seely Ave., San Jose, CA 95134, USA.

**Trademarks:** Trademarks and service marks of Cadence Design Systems, Inc. contained in this document are attributed to Cadence with the appropriate symbol. For queries regarding Cadence's trademarks, contact the corporate legal department at the address shown above or call 800.862.4522. All other trademarks are the property of their respective holders.

**Restricted Permission:** This publication is protected by copyright law and international treaties and contains trade secrets and proprietary information owned by Cadence. Unauthorized reproduction or distribution of this publication, or any portion of it, may result in civil and criminal penalties. Except as specified in this permission statement, this publication may not be copied, reproduced, modified, published, uploaded, posted, transmitted, or distributed in any way, without prior written permission from Cadence. Unless otherwise agreed to by Cadence in writing, this statement grants Cadence customers permission to print one (1) hard copy of this publication subject to the following conditions:

1. The publication may be used only in accordance with a written agreement between Cadence and its customer.
2. The publication may not be modified in any way.
3. Any authorized copy of the publication or portion thereof must include all original copyright, trademark, and other proprietary notices and this permission statement.
4. The information contained in this document cannot be used in the development of like products or software, whether for internal or external use, and shall not be used for the benefit of any other party, whether or not for consideration.

**Disclaimer:** Information in this publication is subject to change without notice and does not represent a commitment on the part of Cadence. Except as may be explicitly set forth in such agreement, Cadence does not make, and expressly disclaims, any representations or warranties as to the completeness, accuracy or usefulness of the information contained in this document. Cadence does not warrant that use of such information will not infringe any third party rights, nor does Cadence assume any liability for damages or costs of any kind that may result from use of such information.

**Restricted Rights:** Use, duplication, or disclosure by the Government is subject to restrictions as set forth in FAR52.227- 14 and DFAR252.227-7013 et seq. or its successor.

## Document Revision History

| Revision | Date        | Person | Changes Made from Previous Revision                 |
|----------|-------------|--------|-----------------------------------------------------|
| Rev 2.2  | 03-Mar-2015 | J.K.   | New document version (preliminary editorial review) |
|          |             |        |                                                     |

# Table of contents

|                                                                       |    |
|-----------------------------------------------------------------------|----|
| Table of contents.....                                                | 4  |
| Section 1 – Module Data.....                                          | 7  |
| Scope.....                                                            | 7  |
| Description .....                                                     | 9  |
| Block Diagram .....                                                   | 9  |
| Signal Interfaces .....                                               | 10 |
| AMBA (APB) Interface .....                                            | 10 |
| AMBA (AHB) Interface.....                                             | 11 |
| Optional Host Reference Clock Interface .....                         | 12 |
| SRAM Interface .....                                                  | 12 |
| External SPI Flash interface .....                                    | 12 |
| Optional DMA Peripheral Interface .....                               | 13 |
| Adapted Loopback Device Clock.....                                    | 14 |
| FSMs description .....                                                | 15 |
| Flash command generator FSM .....                                     | 15 |
| Low-Level SPI Control FSM .....                                       | 15 |
| Clock Diagram.....                                                    | 16 |
| Reset Diagram .....                                                   | 16 |
| Timing Requirements .....                                             | 17 |
| APB Timings .....                                                     | 17 |
| AHB Timing.....                                                       | 19 |
| QSPI Timing.....                                                      | 21 |
| Clock Domains .....                                                   | 22 |
| Programming Interface.....                                            | 23 |
| Controller Register Map.....                                          | 23 |
| Physical Estimates .....                                              | 24 |
| Section 2 - Programming the QSPI_FLASH_CTRL .....                     | 25 |
| Introduction .....                                                    | 25 |
| Functional Description and Operation .....                            | 25 |
| AHB Control Interface.....                                            | 25 |
| AHB Interface .....                                                   | 25 |
| Remapping the AHB address.....                                        | 25 |
| Write Protection .....                                                | 26 |
| Access Forwarding .....                                               | 26 |
| Sequential Access Detection and Burst Length .....                    | 26 |
| AHB Address Decoder .....                                             | 26 |
| Direct Access Controller (DAC) .....                                  | 27 |
| Indirect Controller (INDAC).....                                      | 28 |
| Indirect Read Controller .....                                        | 28 |
| Indirect read transfer process.....                                   | 29 |
| Indirect Write Controller.....                                        | 30 |
| Indirect write transfer process .....                                 | 32 |
| Indirect Access Queuing .....                                         | 33 |
| Consecutive Writes and Reads using Indirect Transfers .....           | 33 |
| Accessing the SRAM.....                                               | 33 |
| SRAM Access Priority.....                                             | 34 |
| DMA Peripheral Controller .....                                       | 35 |
| Interface.....                                                        | 35 |
| drtype/datatype Encoding .....                                        | 35 |
| Order of operation .....                                              | 36 |
| Software Triggered Instruction Generator (STIG) .....                 | 38 |
| Servicing a STIG request .....                                        | 38 |
| Arbitration between Direct / Indirect Access Controller and STIG..... | 39 |
| SPI Command Translation.....                                          | 39 |

|                                                                      |           |
|----------------------------------------------------------------------|-----------|
| Selecting the Flash Instruction Type .....                           | 40        |
| READS .....                                                          | 40        |
| WRITES.....                                                          | 41        |
| APB Interface and Register Module .....                              | 42        |
| SRAM Macro.....                                                      | 42        |
| Read Data Capturing .....                                            | 43        |
| Mechanism using taps.....                                            | 43        |
| Mechanism using external DLL .....                                   | 44        |
| BOOT feature description .....                                       | 45        |
| BOOT configuration:.....                                             | 45        |
| Example of an 8 byte Read Transfer .....                             | 46        |
| <b>Programmer's Guide .....</b>                                      | <b>47</b> |
| Configuring the QSPI Controller for use after reset.....             | 47        |
| Configuring the QSPI Controller for optimal use .....                | 47        |
| Using the Flash Command Control Register (STIG Operation) .....      | 48        |
| Using SPI Legacy Mode .....                                          | 48        |
| Entering and Exiting XIP mode.....                                   | 49        |
| Entering XIP mode from POR .....                                     | 49        |
| Entering XIP mode otherwise.....                                     | 49        |
| <i>Micron N25Q and MT25 Devices (No support for Basic-XIP)</i> ..... | 49        |
| <i>Other Micron Devices (Supporting Basic-XIP)</i> .....             | 49        |
| <i>Winbond Devices</i> .....                                         | 50        |
| <i>Spansion Devices</i> .....                                        | 50        |
| Exiting XIP mode .....                                               | 50        |
| Using Indirect Data Transfer Mode.....                               | 50        |
| Remapping AHB addresses .....                                        | 50        |
| Servicing Interrupts.....                                            | 50        |
| Using the AHB Protection Registers.....                              | 50        |
| <b>Programming Interface.....</b>                                    | <b>51</b> |
| Registers .....                                                      | 51        |
| QSPI Configuration Register .....                                    | 51        |
| Device Read Instruction Register .....                               | 56        |
| Device Write Instruction Configuration Register .....                | 58        |
| QSPI Device Delay Register.....                                      | 59        |
| Read Data Capture Register .....                                     | 61        |
| Device Size Configuration Register .....                             | 63        |
| SRAM Partition Configuration Register .....                          | 64        |
| Indirect AHB Address Trigger register .....                          | 64        |
| DMA Peripheral Configuration Register .....                          | 64        |
| Remap Address Register.....                                          | 65        |
| Mode Bit Configuration Register.....                                 | 65        |
| SRAM Fill Level Register .....                                       | 65        |
| TX Threshold Register .....                                          | 65        |
| RX Threshold Register .....                                          | 65        |
| Write Completion Control Register .....                              | 66        |
| Polling Expiration Register.....                                     | 67        |
| Interrupt Status Register.....                                       | 67        |
| Interrupt Mask Register .....                                        | 68        |
| Lower Write Protection Register .....                                | 69        |
| Upper Write Protection Register .....                                | 69        |
| Write Protection Register .....                                      | 70        |
| Indirect Read Transfer Control Register.....                         | 71        |
| Indirect Read Transfer Watermark Register.....                       | 71        |
| Indirect Read Transfer Start Address Register.....                   | 72        |
| Indirect Read Transfer Number Bytes Register.....                    | 72        |
| Indirect Write Transfer Control Register .....                       | 72        |
| Indirect Write Transfer Watermark Register .....                     | 73        |
| Indirect Write Transfer Start Address Register .....                 | 73        |

|                                                          |           |
|----------------------------------------------------------|-----------|
| Indirect Write Transfer Number Bytes Register .....      | 73        |
| Indirect Trigger Address Range Register .....            | 73        |
| Flash Command Control Register (Using STIG) .....        | 74        |
| Flash Command Address Register .....                     | 75        |
| Flash Command Read Data Register (Lower) .....           | 76        |
| Flash Command Read Data Register (Upper) .....           | 76        |
| Flash Command Write Data Register (Lower) .....          | 76        |
| Flash Command Write Data Register (Upper) .....          | 76        |
| Polling Flash Status Register .....                      | 77        |
| Module ID Register .....                                 | 77        |
| <b>Performance Characteristics .....</b>                 | <b>78</b> |
| Performance (Direct Access) .....                        | 78        |
| Performance (XIP Mode) .....                             | 79        |
| Performance (Indirect Access) .....                      | 80        |
| <b>Section 3 - Integrating the QSPI_FLASH_CTRL .....</b> | <b>81</b> |
| <b>Introduction .....</b>                                | <b>81</b> |
| <b>IP Content .....</b>                                  | <b>81</b> |
| Reference Environment .....                              | 81        |
| Tools .....                                              | 81        |
| Implementation Application Notes .....                   | 81        |
| Micron N25Q Family of Devices .....                      | 81        |
| Micron MT25Q Family of Devices .....                     | 82        |
| Clocking .....                                           | 82        |
| AHB Clock (hclk) .....                                   | 82        |
| APB Clock (pclk) .....                                   | 82        |
| Reference Clock (ref_clk) .....                          | 82        |
| Loopback Clock (spi_dlyd_i) .....                        | 83        |
| Generated Clock (sclk_out) .....                         | 83        |
| Clock Relationships .....                                | 83        |
| Timing and Load Parameters .....                         | 83        |
| Designing Across Clock Boundaries .....                  | 84        |
| Static Configuration .....                               | 84        |
| Metastability .....                                      | 84        |
| Data Path Transfer Between ref_clk and hclk .....        | 85        |
| Reset .....                                              | 85        |
| Verification and Test Application .....                  | 85        |
| Testbench structure .....                                | 85        |
| Testbench block description .....                        | 89        |
| Testbench syntax description .....                       | 91        |
| CPU Based Testing .....                                  | 94        |
| <b>Section 4 – Integration Guide .....</b>               | <b>95</b> |
| Installation .....                                       | 95        |
| Compilation .....                                        | 95        |
| Simulation .....                                         | 95        |
| Synthesis .....                                          | 95        |
| Configuration .....                                      | 95        |
| Asynchronous and unregistered inputs .....               | 95        |
| Combinatorial Outputs .....                              | 95        |
| Latches .....                                            | 95        |
| SDC Constrains .....                                     | 95        |
| <b>Contact Information .....</b>                         | <b>96</b> |
| Cadence Design Foundry .....                             | 96        |
| Support .....                                            | 96        |

# **Section 1 – Module Data**

## **Scope**

This user guide describes a controller-only solution (IP6514E) referred to in this document as the Cadence Flash QSPI Controller IP.

This document is provided to enable the Cadence Flash QSPI Controller IP (subsequently referred to also as QSPI\_FLASH\_CTRL) to be programmed and fully integrated into an SoC design. It covers the following topics:

- Module Data
- Programming the IP
- Integrating the IP

The QSPI\_FLASH\_CTRL module also comes complete with the following:

- Verilog HDL
- Cadence RTL Compiler synthesis scripts
- Verilog testbench

## Features

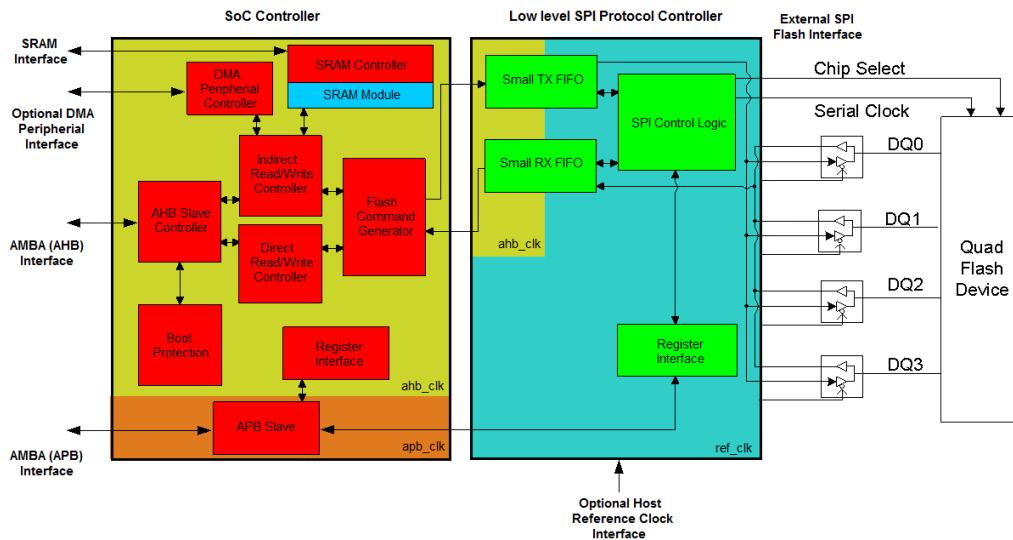
- Memory mapped 'direct' mode of operation for performing FLASH data transfers and executing code from FLASH memory
- Software triggered 'indirect' mode of operation for performing low latency and non processor intensive FLASH data transfers
- Optional DMA Peripheral interface to communicate indirect mode status with external DMA
- Local SRAM of configurable size to reduce AHB overhead and buffer FLASH data during indirect transfers
- Set of software APB accessible FLASH control registers to perform any FLASH command, including data transfers up to 8-bytes at a time
- Supports any device clock frequency, including current market device frequencies of 133MHz SDR or 80 MHz DDR
- Support for XIP (Execute in Place), sometimes referred to as continuous mode
- Support for DDR Mode and DTR Protocol
- Support for single, dual or quad I/O instructions
- Programmable device sizes
- Programmable write protected regions to block system writes from taking effect
- Programmable delays between transactions
- Legacy mode allowing software direct access to low level transmit and receive FIFOs, bypassing the higher layer processes
- Option to use an independent reference clock to decouple AHB clock from SPI clock – allows slow system clocks
- Serial clock with programmable polarity
- Programmable baud rate generator to generate device clocks
- Features included to improve high speed read data capture mechanism
- Option to use adapted clocks to further improve read data capturing
- Programmable interrupt generation
- Up to four external device selects
- Programmable AHB decoder, enables continuous addressing mode for each of connected devices and auto-detection of boundaries between devices
- Support BOOT mode

## Description

The QSPI Flash Controller (QSPI\_FLASH\_CTRL) can be used to provide access to Serial Flash devices. Standard Serial Peripheral Interface (SPI) is supported along with high performance Dual and Quad SPI variants.

The QSPI\_FLASH\_CTRL connects to an SoC environment through its AMBA AHB bus and APB bus interfaces. The AHB interface is used to transfer data, either in a memory mapped direct fashion (for example a processor wishing to execute code directly from external FLASH memory), or in an indirect fashion where the controller is setup via configuration registers to silently perform some requested operation, signalling its completion via interrupts or status registers. For indirect operations, data is transferred between system memory and external FLASH memory via an internal SRAM which is loaded for writes and unloaded for reads by an AHB master within the SoC environment at low latency AHB system speeds. Interrupts or status registers are used to identify the specific times at which this SRAM should be accessed using user programmable configuration registers. The size of the SRAM is configurable. An optional DMA peripheral bus is also available to optimize the data transfers between an external master and QSPI\_FLASH\_CTRL during indirect transfers.

## Block Diagram



## Signal Interfaces

The QSPI\_FLASH\_CTRL includes the following signal interfaces:

- AMBA Advanced Peripheral Bus (APB) slave interface for accessing the programmable registers
- AMBA Advanced High Speed Bus (AHB) slave interface for data transfer
- An optional DMA peripheral interface

The AMBA interfaces fully conform to the ARM AMBA Revision 2.0 specification.

### AMBA (APB) Interface

| Signal Name  | I/O | clk  | Description                                                                                                                                                                                                                                                      |
|--------------|-----|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| n_preset     | I   | pclk | Active low AMBA reset (nPRESET). This signal must be asserted low asynchronously, and deasserted high synchronously with pclk. Resets all APB registers                                                                                                          |
| pclk         | I   | -    | Peripheral bus clock (PCLK).                                                                                                                                                                                                                                     |
| psel         | I   | pclk | Peripheral select (PSEL). Active high select signal to indicate that a valid access is being made to one of the programmable registers.                                                                                                                          |
| penable      | I   | pclk | Peripheral enable (PENABLE). This indicates the second clock cycle of an access and indicates that the write data may be strobed into a register on the next rising edge of pclk, or that the read data is expected to be valid at the next rising edge of pclk. |
| pwrite       | I   | pclk | Peripheral write strobe (PWRITE). This indicates that a write access is taking place (if psel is active).                                                                                                                                                        |
| paddr[7:0]   | I   | pclk | Address bus from selected master (PADDR). Indicates which register is being accessed. This address is word-aligned and therefore bits [1:0] should always be '00'.                                                                                               |
| pwdata[31:0] | I   | pclk | Write data. Data to be written into the addressed register.                                                                                                                                                                                                      |
| prdata[31:0] | O   | pclk | Read data. Data read from the addressed register. For APB Rev 2.0, it is driven to logic 0 when not addressed.                                                                                                                                                   |
| pslverr      | O   | pclk | Indicates an APB access to an invalid address. Should be sampled when penable is high. This signal is optional and not a standard AMBA signal.                                                                                                                   |
| pready       | O   | pclk | Indicates the current APB access is to be delayed with wait states. If pready is tied high – i.e. no wait states.                                                                                                                                                |

|           |   |      |                                        |
|-----------|---|------|----------------------------------------|
| interrupt | O | pclk | Level sensitive active high interrupt. |
|-----------|---|------|----------------------------------------|

**AMBA (AHB) Interface**

| Signal Name  | I/O | clk  | Description                                                                                                                                                |
|--------------|-----|------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| n_hreset     | I   | hclk | Active low asynchronous reset. This signal must be asserted low asynchronously and deasserted high synchronously with hclk.                                |
| hclk         | I   | -    | AHB bus clock.                                                                                                                                             |
| hsel         | I   | hclk | AHB slave select.                                                                                                                                          |
| hready_in    | I   | hclk | Combined hready from all slaves in AHB system.                                                                                                             |
| htrans[1:0]  | I   | hclk | AHB transfer type as follows:<br>00 - Idle<br>01 - Busy<br>10 - Non-sequential<br>11 - Sequential.                                                         |
| haddr[31:0]  | I   | hclk | AHB address.                                                                                                                                               |
| hwrite       | I   | hclk | High for AHB write, low for AHB read.                                                                                                                      |
| hsize[2:0]   | I   | hclk | Transfer size, configured according to AMBA AHB data bus width as follows:<br>010 - 32 bit transfers<br>011 - 64 bit transfers<br>100 - 128 bit transfers. |
| hburst[2:0]  | I   | hclk | Burst type, 000 for single transfer, 001 for incrementing burst and 011 for four beat incrementing burst, etc                                              |
| hwdata[31:0] | I   | hclk | Write data from external AHB Master, 32 bit only                                                                                                           |
| hrdata[31:0] | O   | hclk | Read data to external AHB Master, 32 bit only<br>Unused bits will be tied to logic 0.                                                                      |
| hresp        | O   | hclk | Slave response, 0 for OK, any other response is not OK and triggers an interrupt. AMBA split and retry operations are not supported.                       |
| hready_out   | O   | hclk | hready generated by the Flash Controller slave.<br>When low, the flash controller is inserting wait states.                                                |

**Optional Host Reference Clock Interface**

| Signal Name | I/O | Description                                                                                                                                                                    |
|-------------|-----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| n_ref_rst   | I   | Active low ref_clk reset. This signal must be asserted low asynchronously, and deasserted high synchronously with ref_clk. Resets all registers in the reference clock domain. |
| ref_clk     | I   | Reference clock (ref_clk).                                                                                                                                                     |

**SRAM Interface**

The SRAM ports (used for indirect modes of data transfer only) are passed to the toplevel

| Signal Name   | I/O | clk  | Description                                   |
|---------------|-----|------|-----------------------------------------------|
| sram_cs       | O   | hclk | Active high chip select.                      |
| sram_we       | O   | hclk | Active high write enable.                     |
| sram_re       | O   | hclk | Active high read enable.                      |
| sram_add[N:0] | O   | hclk | SRAM address bus. Address width configurable. |
| sram_do[31:0] | O   | hclk | SRAM write data bus.                          |
| sram_di[31:0] | I   | hclk | SRAM read data bus.                           |

**External SPI Flash interface**

| Signal Name | I/O | clk     | Description                                                                                                                                                                                                                                                                                                                   |
|-------------|-----|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| mi0         | I   | async   | Quad-SPI master mode input, bit 0<br>The master input may be up to 4-bits wide in master mode, with 1-bit for standard serial SPI operation, 2-bits for Dual-SPI, and 4-bits for Quad-SPI<br>Internally, the Quad-SPI IP must switch between mi[1] for 1-bit wide input and mi[0] for the LSB of dual-mode or quad-mode input |
| mo0         | O   | ref_clk | MOSI signal, master mode output; Quad-SPI master mode output, bit 0                                                                                                                                                                                                                                                           |
| n_mo_en[0]  | O   | ref_clk | Master mode output enable for bit 0.<br>Active low. This pin retains its original function for serial SPI mode, but additionally will be driven dynamically for Dual- and Quad-SPI modes, based on the enabled mode and the current stage of the transfer.                                                                    |
| mi1         | I   | async   | MISO pin, master mode input; Quad-SPI master mode input, bit 1<br>This may be up to 4-bits wide in master mode, with 1-bit for standard serial SPI operation, 2-bits for Dual-SPI, and 4-bits for Quad-SPI                                                                                                                    |

|               |   |         |                                                                                                                                                                                                                                    |
|---------------|---|---------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|               |   |         | Internally, the Quad-SPI IP must switch between mi[1] for 1-bit wide input and m[0] for the LSB of dual-mode or quad-mode input                                                                                                    |
| mo1           | O | ref_clk | Quad-SPI master mode output, bit 1                                                                                                                                                                                                 |
| n_mo_en[1]    | O | ref_clk | Master mode output enable for bit 1<br>Active low.                                                                                                                                                                                 |
| mi2           | I | async   | Quad-SPI master mode input, bit 2<br>This may be up to 4-bits wide in master mode, with 1-bit for standard serial SPI operation, 2-bits for Dual-SPI, and 4-bits for Quad-SPI                                                      |
| mo2_wpn       | O | ref_clk | Quad-SPI master mode output, bit 2, for quad-mode writes to external SPI slave and used for write protect output to flash devices, active low, when not in quad mode.                                                              |
| n_mo_en[2]    | O | ref_clk | Master mode output enable for bit 2.<br>Active low. Only utilized if quad-mode commands are supported.                                                                                                                             |
| mi3           | I | async   | Quad-SPI master mode input, bit 3<br>This may be up to 4-bits wide in master mode, with 1-bit for standard serial SPI operation, 2-bits for Dual-SPI, and 4-bits for Quad-SPI                                                      |
| mo3_hold      | O | ref_clk | Quad-SPI master mode output, bit 3, for quad-mode writes to external SPI slave, and used for hold output to flash devices, active low, when not in quad mode.<br>NOTE: The use of the HOLD pin is not supported by the controller. |
| n_mo_en[3]    | O | ref_clk | Master mode output enable for bit 3.<br>Active low. Only utilized if quad-mode commands are supported.                                                                                                                             |
| n_ss_out[3:0] | O | ref_clk | Quad-SPI peripheral select outputs<br>The IP supplies up to 4 CS                                                                                                                                                                   |
| sclk_out      | O | ref_clk | Quad-SPI master clock output                                                                                                                                                                                                       |

### Optional DMA Peripheral Interface

| Signal Name   | I/O | clk  | Description                                                                                                           |
|---------------|-----|------|-----------------------------------------------------------------------------------------------------------------------|
| indwr_drtype  | O   | hclk | Driven by controller to indicate the request type the indirect write logic wants the dma to perform (single or burst) |
| indwr_drlast  | O   | hclk | Indirect write dma request last                                                                                       |
| indwr_drvalid | O   | hclk | Indirect write controller request to dma to perform a transfer                                                        |
| indwr_drready | I   | hclk | Indication that dma has accepted the indirect write dma request                                                       |
| indwr_datype  | I   | hclk | Driven by dma to indicate to the controller the type of transfer that is being acknowledged.                          |

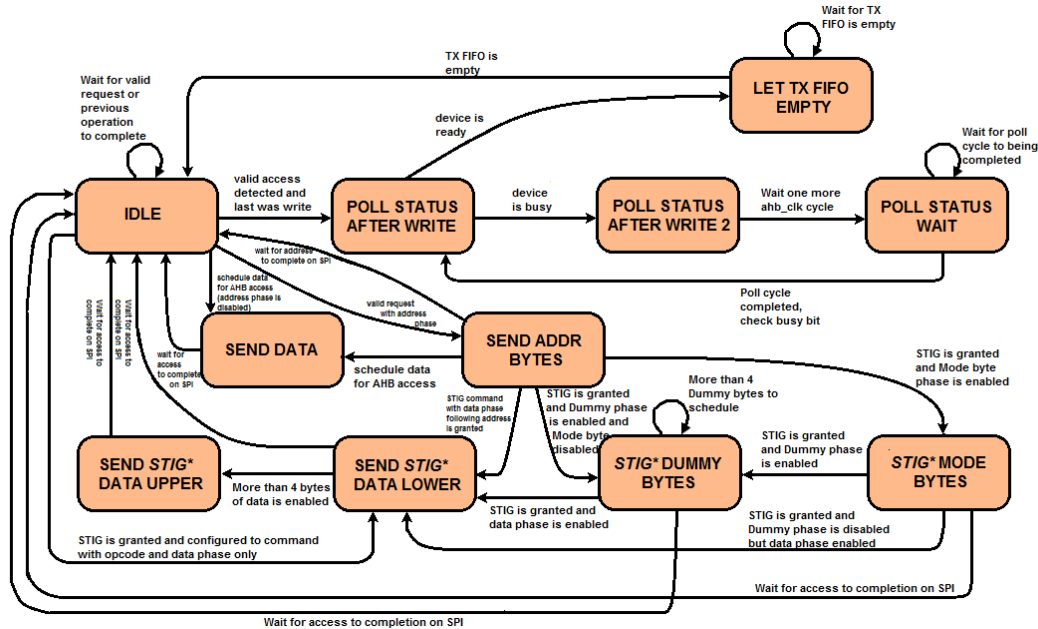
|               |   |      |                                                                                                                                                                        |
|---------------|---|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|               |   |      | The QSPI only uses this signal to identify an incoming DMA flush request.                                                                                              |
| indwr_davalid | I | hclk | Driven by dma to indicate that the dma request is complete                                                                                                             |
| indwr_daready | O | hclk | Driven by controller to indicate that it saw the indwr_davalid                                                                                                         |
| indrd_drtype  | O | hclk | Driven by controller to indicate the request type the indirect read logic wants the dma to perform (single or burst)                                                   |
| indrd_drlast  | O | hclk | Indirect read dma request last                                                                                                                                         |
| indrd_drvalid | O | hclk | Indirect read controller request to dma to perform a transfer                                                                                                          |
| indrd_drready | I | hclk | Indication that dma has accepted the indirect read dma request                                                                                                         |
| indrd_datype  | I | hclk | Driven by dma to indicate to the controller the type of transfer that is being acknowledged. The QSPI only uses this signal to identify an incoming DMA flush request. |
| indrd_davalid | I | hclk | Driven by dma to indicate that the indirect read dma request is complete                                                                                               |
| indrd_daready | O | hclk | Driven by controller to indicate that it saw the indrd_davalid                                                                                                         |

### Adapted Loopback Device Clock

| Signal Name | I/O | Description                                                                                                                                                                                                                                                                                                |
|-------------|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| sclk_dlyd_i | I   | This is an optional clock, used by the read delay capture circuit to sample the read data as it returns from the memory device. It is a delayed version of the 'sclk_out' output from the QSPI IP. The delay itself is implemented externally. Refer to section "Read Data Capturing" for further details. |

## FSMs description

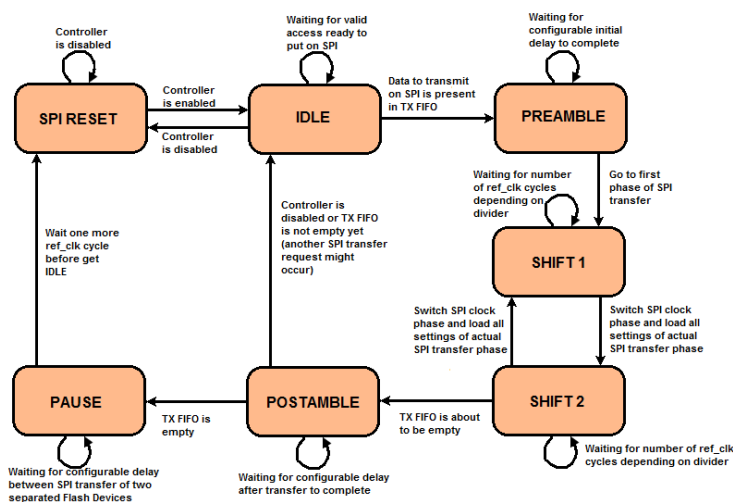
### Flash command generator FSM



This FSM works on `ahb_clk`. It is implemented within Flash Command Generator block (inside `cdnsahb_ctrl_cmdgen.v` file). It generates appropriate transfer queue depending on triggered access from outside the core. Data is then forwarded into low level SPI Module.

“STIG” refers to Mode of operation triggered by software. Detailed description of this Mode is presented in section called “Software Triggered Instruction Generator (STIG)”.

### Low-Level SPI Control FSM



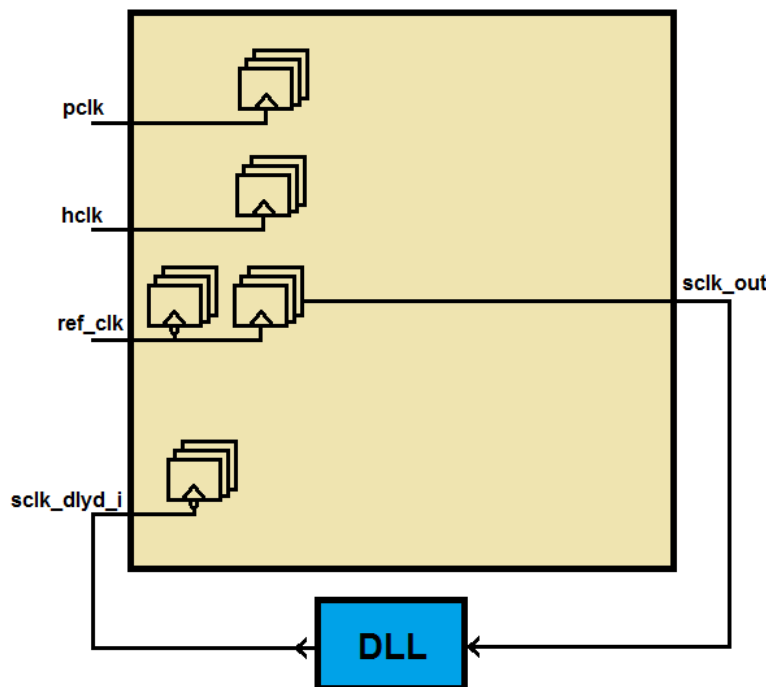
This FSM (placed inside `cdnsspi_control.v` file) works on `ref_clk` or `ahb_clk` (if optional reference clock is disabled). It identifies actual phase of SPI transfer.

Each transfer is launched based on data formulated by Flash Command Generator FSM.

## Clock Diagram

AHB, APB and reference clocks are intended to being generated continuously. There is also `sclk_out` which is the generated clock passing further outside the chip into clock input of the Flash Device.

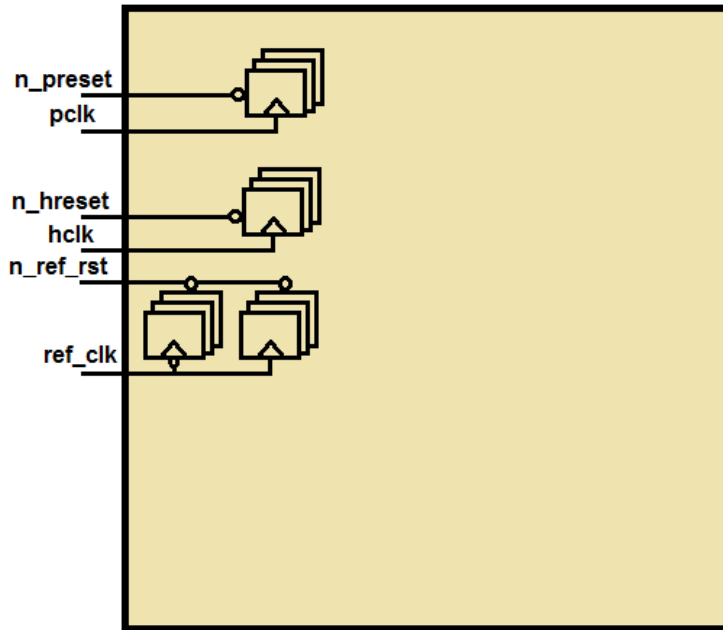
`Sclk_dlyd_i` takes the `sclk_out` shifted by external DLL designed by the user. Usage of this clock has some limitations described down the document.



## Reset Diagram

There are three asynchronous resets present in the design, one for each of the main clock domains (`ref_clk`, `hclk` and `pclk`). These resets are used directly to reset the synchronous parts of the design. The reset signals may be set low asynchronously with respect to the clock, but must be set high synchronously.

Since `sclk_dlyd_i` clock is used for sampling data only – there is no reset for this domain.

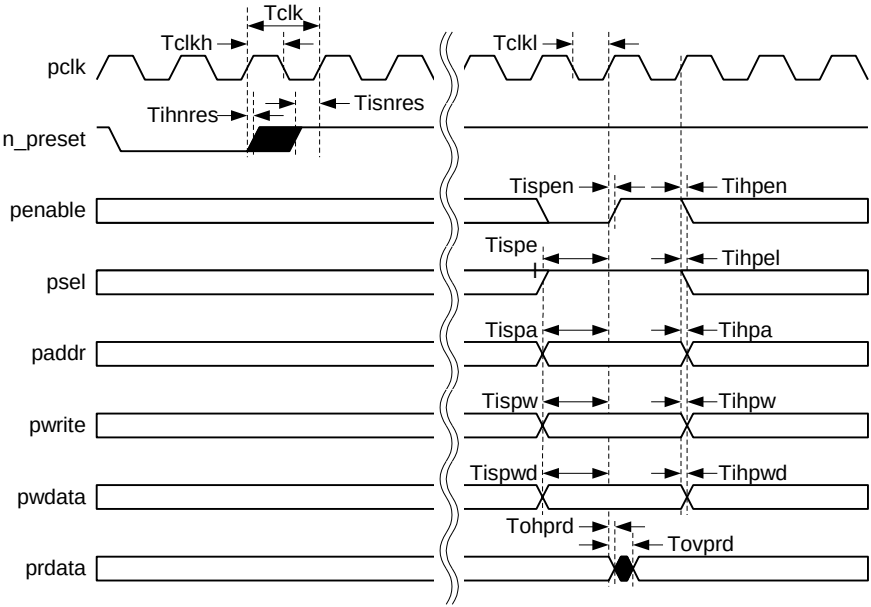


## Timing Requirements

### APB Timings

| Parameter | Description                                          | Min  | Max  | Unit |
|-----------|------------------------------------------------------|------|------|------|
| Tclk      | Clock period                                         | 10   | DC   | ns   |
| Tclkh     | Clock high                                           | N.A. | N.A. |      |
| Tclkl     | Clock low                                            | N.A. | N.A. |      |
| Tihnres   | <code>n_preset</code> hold after <code>pclk</code>   | N.A. | N.A. |      |
| Tisnres   | <code>n_preset</code> setup before <code>pclk</code> | N.A. | N.A. |      |
| Tihpen    | <code>penable</code> hold after <code>pclk</code>    | 0.5  | -    | ns   |
| Tispen    | <code>penable</code> setup before <code>pclk</code>  | 4.0  | -    | ns   |
| Tihpel    | <code>psel</code> hold after <code>pclk</code>       | 0.5  | -    | ns   |
| Tispel    | <code>psel</code> setup before <code>pclk</code>     | 4.0  | -    | ns   |
| Tihpa     | <code>paddr</code> hold after <code>pclk</code>      | 0.5  | -    | ns   |
| Tispa     | <code>paddr</code> setup before <code>pclk</code>    | 4.0  | -    | ns   |
| Tihpw     | <code>pwrite</code> hold after <code>pclk</code>     | 0.5  | -    | ns   |
| Tispw     | <code>pwrite</code> setup before <code>pclk</code>   | 4.0  | -    | ns   |
| Tihpwd    | <code>pdata</code> hold after <code>pclk</code>      | 0.5  | -    | ns   |
| Tispwd    | <code>pdata</code> setup before <code>pclk</code>    | 4.0  | -    | ns   |

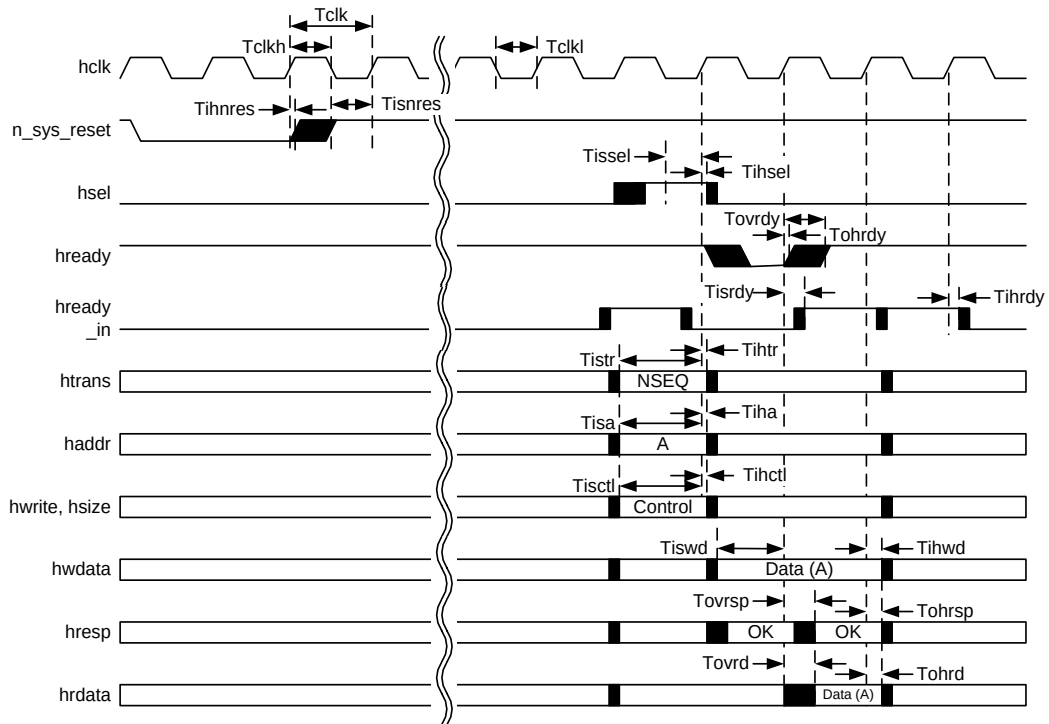
|        |                         |     |     |    |
|--------|-------------------------|-----|-----|----|
| Tohprd | prdata hold after pclk  | 0.5 | -   | ns |
| Tovprd | prdata valid after pclk | -   | 4.0 | ns |



**AHB Timing**

Maximum achieved AHB frequency is equal to 600 MHz so this requirement was inserted into the table. There are some limitations which could hamper correct work on such high frequency depending on individual requirements of the system. Reference value of this clock is 100 MHz.

| Parameter | Description                     | Min | Typ      | Max | Unit |
|-----------|---------------------------------|-----|----------|-----|------|
| Tclk      | Clock period, T0                | 1.6 | 10       | DC  | ns   |
| Tclkh     | Clock high                      | 40% | 40%      | —   | T0   |
| Tclkl     | Clock low                       | 40% | 40%      | —   | T0   |
| Tihnres   | n_sys_reset hold after hclk     | N/A | N/A      |     | T0   |
| Tisnres   | n_sys_reset setup before hclk   | N/A | N/A      |     | T0   |
| Tissel    | hsel setup before hclk          | 60% | 60%      | —   | T0   |
| Tihsel    | hsel hold after hclk            | 0.1 | 0.1      | —   | ns   |
| Tohrdy    | hready_out hold after hclk      | 0.1 | 0.1      | —   | ns   |
| Tovrdy    | hready_out valid after hclk     | —   | —        | 50% | T0   |
| Tihrdy    | hready_in hold after hclk       | 0.1 | 0.1      | —   | ns   |
| Tisrdy    | hready_in setup before hclk     | 60% | 60%      | —   | T0   |
| Tihtr     | htrans hold after hclk          | 0.1 | 0.1      | —   | ns   |
| Tistr     | htrans setup before hclk        | 60% | 60%      | —   | T0   |
| Tiha      | haddr hold after hclk           | 0.1 | 0.1      | —   | ns   |
| Tisa      | haddr setup before hclk         | 60% | 60%      | —   | T0   |
| Tihctl    | hwrite, hsize hold after hclk   | 0.1 | 0.1      | —   | ns   |
| Tisctl    | hwrite, hsize setup before hclk | 60% | 60%      | —   | T0   |
| Tihwd     | hwd data hold after hclk        |     | See Note |     |      |
| Tiswd     | hwd data setup before hclk      | 60% | 60%      | —   | T0   |
| Tohrsp    | hresp hold after hclk           | 0.1 | 0.1      | —   | ns   |
| Tovrsp    | hresp valid after hclk          | —   | —        | 50% | T0   |
| Tohrd     | hrdata hold after hclk          | 0.1 | 0.1      | —   | ns   |
| Tovrd     | hrdata valid after hclk         | —   | —        | 50% | T0   |

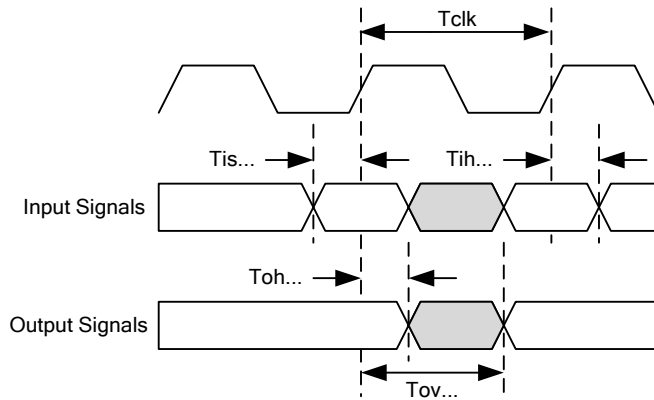


**Note:** All timings are specified relative to the target clock period,  $pclk/hclk$ . These timings are coded into the provided synthesis script, and have been achieved with a typical technology, but are for guidance only.

## QSPI Timing

Most outputs are registered on the ref\_clk. There is no combinatorial logic on these outputs. Master Output Interface which is directly forwarded to external Flash Device has multiplexer on its output for the optional delay inserting feature which is used to adjust hold timing. Given that this interface toggles based on programmable divider synchronous for ref\_clk – multi cycle path is ensured, therefore additional combinatorial delay is safe in this case. For the inputs, these are all data inputs, driven on one edge of sclk\_in and sampled by the QSPI flash controller on the ref\_clk.

| Parameter | Description                  | Min  | Max | Unit  |
|-----------|------------------------------|------|-----|-------|
| INPUTS    |                              |      |     |       |
| Tclk      | ref_clk period, T0           | 1.25 | DC  | ns    |
| Tihmi0    | mi0 hold after ref_clk       | 5%   | -   | %Tclk |
| Tismi0    | mi0 setup before ref_clk     | -    | 35% | %Tclk |
| Tihmi1    | mi1 hold after ref_clk       | 5%   | -   | %Tclk |
| Tismi1    | mi1 setup before ref_clk     | -    | 35% | %Tclk |
| Tihmi2    | mi2 hold after ref_clk       | 5%   | -   | %Tclk |
| Tismi2    | mi2 setup before ref_clk     | -    | 35% | %Tclk |
| Tihmi3    | mi3 hold after ref_clk       | 5%   | -   | %Tclk |
| Tismi3    | mi3 setup before ref_clk     | -    | 35% | %Tclk |
| OUTPUTS   |                              |      |     |       |
| Tohmo0    | mo0 hold after ref_clk       | 5%   | -   | %Tclk |
| Tovmo0    | mo0 valid after ref_clk      | -    | 50% | %Tclk |
| Tohmo1    | mo1 hold after ref_clk       | 5%   | -   | %Tclk |
| Tovmo1    | mo1 valid after ref_clk      | -    | 50% | %Tclk |
| Tohmo2    | mo2_wpn hold after ref_clk   | 5%   | -   | %Tclk |
| Tovmo2    | mo2_wpn valid after ref_clk  | -    | 50% | %Tclk |
| Tohmo3    | mo3_hold hold after ref_clk  | 5%   | -   | %Tclk |
| Tovmo3    | mo3_hold valid after ref_clk | -    | 50% | %Tclk |
| Tohmoen   | n_mo_en hold after ref_clk   | 5%   | -   | %Tclk |
| Tovmoen   | n_mo_en valid after ref_clk  | -    | 50% | %Tclk |
| Tohssso   | n_ss_out hold after ref_clk  | 5%   | -   | %Tclk |
| Tovssso   | n_ss_out valid after ref_clk | -    | 50% | %Tclk |



**Note 1:** Max timings are specified relative to the target clock period,  $T_{clk}$ . These are coded into the synthesis script provided. These timings have been achieved with a typical technology, but are for guidance only.

**Note 2:** Hold times specified are guidelines only. Min timings have no effect during synthesis.

## Clock Domains

There are three main asynchronous clock sources for the QSPI Controller (although this can be reduced optionally to 2) and one generated clock for clocking the external FLASH device. The AHB clock is the main system clock used to transfer data over the AHB bus between an external AHB master and the QSPI controller. The APB clock is used to access the register map of the QSPI controller and perform basic configuration and interrupt handling. The reference clock is optional via a configuration define. The reference clock (when enabled) or the AHB clock (when the reference lock is not enabled) is used to serialize the data and drive the external SPI interface. Using the reference clock allows the core to decouple the frequency of the SPI FLASH device from the SoC system clocks, thereby providing a more flexible clocking solution.

## Programming Interface

The following registers may be programmed for the QSPI\_FLASH\_CTRL module.

### Controller Register Map

| Offset | Function                                      | R/W |
|--------|-----------------------------------------------|-----|
| 0x0    | QSPI Configuration Register                   | R/W |
| 0x4    | Device Read Instruction Register              | R/W |
| 0x8    | Device Write Instruction Register             | R/W |
| 0xC    | QSPI Device Delay Register                    | R/W |
| 0x10   | Read Data Capture Register                    | R/W |
| 0x14   | Device Size Register                          | R/W |
| 0x18   | SRAM Partition Register                       | R/W |
| 0x1C   | Indirect AHB Address Trigger Register         | R/W |
| 0x20   | DMA Peripheral Register                       | R/W |
| 0x24   | Remap Address Register                        | R/W |
| 0x28   | Mode Bit Register                             | R/W |
| 0x2C   | SRAM Fill Level Register                      | RO  |
| 0x30   | TX Threshold Register                         | R/W |
| 0x34   | RX Threshold Register                         | R/W |
| 0x38   | Write Completion Control Register             | R/W |
| 0x3C   | Polling Expiration Register                   | R/W |
| 0x40   | Interrupt Status Register                     | R/W |
| 0x44   | Interrupt Mask                                | R/W |
| 0x50   | Lower Write Protection Register               | R/W |
| 0x54   | Upper Write Protection Register               | R/W |
| 0x58   | Write Protection Register                     | R/W |
| 0x60   | Indirect Read Transfer Register               | R/W |
| 0x64   | Indirect Read Transfer Watermark Register     | R/W |
| 0x68   | Indirect Read Transfer Start Address Register | R/W |
| 0x6C   | Indirect Read Transfer Number Bytes Register  | R/W |
| 0x70   | Indirect Write Transfer Register              | R/W |

|      |                                                |     |
|------|------------------------------------------------|-----|
| 0x74 | Indirect Write Transfer Watermark Register     | R/W |
| 0x78 | Indirect Write Transfer Start Address Register | R/W |
| 0x7C | Indirect Write Transfer Count Register         | R/W |
| 0x80 | Indirect AHB Trigger Address Range Register    | R/W |
| 0x90 | Flash Command Register                         | R/W |
| 0x94 | Flash Command Address Registers                | R/W |
| 0xA0 | Flash Command Read Data Register (Lower)       | RO  |
| 0xA4 | Flash Command Read Data Register (Upper)       | RO  |
| 0xA8 | Flash Command Write Data Register (Lower)      | R/W |
| 0xAC | Flash Command Write Data Register (Upper)      | R/W |
| 0xB0 | Polling Flash Status Register                  | RO  |
| 0xFC | Module ID Register                             | RO  |

## Physical Estimates

The physical estimates listed below are applicable to the Cadence Flash QSPI Controller IP without integrated PHY IP.

Approximate two input NAND equivalent gate counts: 58000

Flip Flop count in HCLK domain (rising edge/falling edge): 2019/0

Flip Flop count in PCLK domain (rising edge/falling edge): 894/0

Flip Flop count in REF\_CLK domain (rising edge/falling edge): 723/4

Flip Flop count in SCLK\_DLYD\_I domain (rising edge/falling edge): 0/4

## Section 2 - Programming the QSPI\_FLASH\_CTRL

### Introduction

This section enables the QSPI\_FLASH\_CTRL module to be programmed as part of a SoC design. It includes:

- Functional description and operation
- Programmer's Guide
- Programming interface

### Functional Description and Operation

#### AHB Control Interface

The AHB slave controller validates incoming AHB accesses, responds to invalid requests, performs any required byte and half-word reordering, blocks writes that violate the programmed write protection rules (only for direct access) and forwards the transfer request to either the direct access controller or the indirect access controller.

#### AHB Interface

The AHB interface conforms to the AMBA 3 AHB-Lite protocol specification from ARM. Locked transfers (HMASTLOCK) and transfers using the protection control signals (HPROT) are not supported.

The AHB data-width is 32 bits wide. Therefore only byte, half-word and word accesses will be permitted. For writes, only incrementing bursts are supported, wrapping write bursts received will generate an error i.e. the access completes with HRESP set to error. INCR16, INCR8, INCR4, INCR and SINGLE type bursts are permitted. For reads all burst types, including WRAPs, are supported. The slave will operate correctly if a burst is terminated early due to interconnect burst termination or if the slave errors an access within the burst.

#### Remapping the AHB address

The QSPI controller does not implement any specific address decoding to error incoming addresses that may lie outside the connected FLASH memory space but the AHB address decoder is implemented. If it is enabled, the QSPI Flash Controller is able to detect address range of individual device to give an option to auto-switching of active devices based on address from AHB. The incoming AHB address, by default, maps directly to the address sent serially to the FLASH device. If the FLASH device has a 24-bit address, then the 24 LSB's of the AHB address will be forwarded.

A remap feature is available to remap all incoming AHB addresses to ADDRESS+ N, where N is the value store in the remap address register (0x24). It is enabled via the QSPI Configuration Register. This feature could be used when software needs to move boot code to another FLASH region.

## Write Protection

In order to protect the FLASH device, a write protection feature is implemented in hardware and controlled from software. Any AHB write detected (by using DAC controller), pointing to an area of FLASH that is protected, will generate an error via the HRESP error output.

A programmable region of the FLASH device, defined as a number of FLASH 'blocks' starting from a particular block number can be protected. Three programmable registers (0x50 to 0x58) are provided. The first register defines the FLASH block that is located at the bottom of the region to be protected. The second register defines the FLASH block that is located at the top of the region to be protected. The third register is a control register consisting of 2 bits. Bit 0 allows software to invert the region that is being protected, causing the programmed region to become the only areas of FLASH memory that is not protected from writes. The bit 1 is the write protection enable bit. When low, the FLASH device is unprotected.

For implementation, the AHB controller must map the incoming AHB address into its associated FLASH block. A block can be between 1 and 65K bytes, programmed via the Device Size Configuration register.

## Access Forwarding

For legal accesses, the AHB controller will forward all AHB accesses to one of two access controllers: the direct access and indirect access controllers. Assuming the direct access controller has been enabled via the QSPI Configuration register (0x00), then by default all AHB accesses will be forwarded to the direct access controller. Before any accesses can be forwarded to the indirect access controller, it must first be configured by software. This process is fully explained in the indirect access controller section of this document. If the direct access controller is disabled, any incoming AHB access that cannot be forwarded to the indirect access controller will be completed immediately with an HRESP error. If the direct access controller is enabled, the same access will be forwarded and serviced by the direct access controller.

## Sequential Access Detection and Burst Length

In order to maximize performance, the AHB interface signal "htrans" is not used by the controller to help identify sequential and non-sequential accesses. The problem with "htrans" is that it identifies all accesses at the start of a new AHB burst as non-sequential, as well as all accesses after a 1K boundary even when the access may be sequential to the last. Instead non-sequential accesses are detected by comparing the address of the current access with the address of the previous access. A decision is then made as to whether the access is sequential or not. If the direction of the access (write/read) has changed, then the access is deemed non-sequential. If the size of the read access (byte/half-word/word) has changed, then the access is also non-sequential. If AHB address decoder is enabled and transition between devices has been detected, then the new SPI transfer has to be generated because currently selected device needs to be initialized. This is the only case when accesses are considered as non-sequential despite addresses are consecutive. Otherwise the access is sequential if the address of the current transfer is sequential when compared the previous transfer.

In addition, the "hburst" signal from AHB is not used by the controller to identify burst lengths. Instead a burst continues until a non-sequential access is received.

## AHB Address Decoder

It is possible to connect up to four devices to the controller. Emerging memories reached size of 2Gb and they are still increasing over this parameter. To ensure high performance in sequential transfer of data greater than a few Gb, AHB Address Decoder can be enabled. Address range from AHB standpoint is merged to continuous form. It allows the software driver to manage of accesses without additional steps to detect address boundaries between devices. Switching is performed automatically by the controller based on actual address from

AHB interface. Boundaries of addresses are calculated by hardware, based on programmable configuration. Each connected device can have another size ranged from 512Mb to 4Gb.

AHB Address Decoder should be used only for direct read transfers. Data to read should be stable to avoid necessity of polling each device after switching between them if write access was issued before. To make sure about stability of the data, is recommended to do first reads separately for each device without using of AHB decoder.

### Direct Access Controller (DAC)

Direct access refers to the operation where AHB accesses directly trigger a read or write to FLASH memory. It is memory mapped and can be used to both access and directly execute code from external FLASH memory. Any incoming AHB access that is not recognized as being within the programmable indirect trigger region is assumed to be a direct access and will be serviced by the direct access controller. Note that accesses that use the direct access controller do not use the embedded SRAM. The AHB will be throttled as the read or write burst is carried out, and the amount of wait states applied will be dependent on the latency through the controller. The latency has been carefully designed to be as small as possible. Latency is kept to a minimum when the use of XIP read instructions are enabled.

When servicing AHB reads, the DAC will always send one additional access downstream than was originally issued by the AHB in any single AHB burst. This is invisible from the system interface. It is a predicted read and ensures the underlying SPI core can operate at maximum bandwidth. The AHB burst defined here is not an AHB burst as defined by AMBA. It is defined as follows:

1. The first access of the AHB burst is defined by
  - a. An AHB access that is NON-SEQUENTIAL (Non-sequential based on address comparison, not based on htrans).
  - b. An AHB access that is NON-SEQUENTIAL or SEQUENTIAL when the downstream modules are IDLE.
2. The last access of the AHB burst is defined by an AHB access that is SEQUENTIAL and precedes a new burst as defined in 1) above
3. The size of an AHB burst is the number of AHB accesses, starting from the first access and ending with the last access
4. The number of DAC requests per AHB burst is equal to the size of the AHB burst + 1;

For AHB writes, the direct access controller will trigger a sequence of write commands that mimic the way reads are processed, although the number of DAC requests for writes is equal to the number of AHB write requests received. For writes, the AHB controller will ensure FLASH bursts will not break a FLASH page boundary. When a page boundary is detected, only the number of byte accesses up to that boundary will be forwarded. A sequential direct write request that crosses a page boundary must be detected as non-sequential to downstream modules, causing the downstream controller to force the Flash device to enter a self timed page program cycle.

Core is supporting splitting writes, which are crossing page boundary, only for word aligned addresses.

The external AHB master supplying the data for writes should attempt to guarantee write data for a particular page can be provided to the controller in a timely fashion to avoid downstream starvation. If there is too much delay in the system issuing a sequential write, then the FLASH write cycle may be initiated prematurely, reducing the effective life of the device. Note that if this cannot be guaranteed and is of concern, then the indirect write operation can instead be used to avoid this issue.

FLASH erase operations, which may be required before a page write, are triggered by software using the documented programming interface (detailed later).

Once a page program cycle has been started, the QSPI Flash Controller will automatically poll for the write cycle to complete before allowing any further AHB accesses to complete. This is achieved by holding any subsequent AHB direct accesses in wait state.

## Indirect Controller (INDAC)

### Indirect Read Controller

The aim of the indirect mode of operation is to read significant numbers of bytes from FLASH memory without requiring an AHB access to trigger it. Instead indirect operations are controlled and triggered by software via specific APB control/configuration registers (0x60 → 0x6c). This block will communicate with an embedded low level SPI protocol state machine module to perform an efficient and optimized FLASH read burst, placing the read data into the local SRAM module ready for fast and low latency delivery to any external AHB master.

By default, the indirect read controller is disabled. Before enabling it, software must configure how much data is required and starting from what address. The start address and total number of bytes to be fetched is defined in registers at addresses 0x68 and 0x6c respectively. Up to two indirect operations can be programmed at any one time. The second can be triggered while the first is in progress. Supporting two indirect operations allows a short turnaround time between the completion of one indirect operation and the start of the second. Refer to the indirect transfer queuing section 1 for further details.

The total number of bytes to read in an indirect operation is not limited by the size of the SRAM. The size of SRAM will only limit the size of requests made to the DMA (if the DMA peripheral interface module is enabled). In the case of an SRAM overrun, the controller will back pressure FLASH reads until space becomes available in the SRAM. Back pressuring the reads on the SPI interface is handled by completing any current read burst, waiting until space in the SRAM becomes available and then issuing a new read burst at the address where the previous terminated burst ended.

An external master will be able to fetch the data the controller has read from external FLASH memory by issuing AHB reads to the QSPI controller. The AHB address of the incoming read access must be in the range AHB Indirect trigger address (programmed via the Indirect AHB trigger address register) to AHB Indirect trigger address +  $2^{**}(\text{Indirect AHB trigger address range}) - 1$ . Default value of the range is equal to 16 locations. This allows a 16-beat burst to be applied starting from the AHB Indirect trigger address. The smaller bursts are possible to handle effectively as well with this approach. Furthermore it is not strict requirement to push consecutive address sequence. Actual AHB address just has to be in the Indirect Range to grant SRAM as source. Each valid AHB Indirect read will cause the internal SRAM to be popped, thereby decoupling the AHB address from the FLASH address – i.e. not direct mapped. Therefore AHB Indirect trigger address does not have any relationship with FLASH address. It is just to indicate that data should take SRAM as source instead of FLASH Memory array after triggering of any valid Indirect Read. FLASH address for Indirect Read is taken from the Indirect Transfer Start Address Register (0x68). Assuming the requested data is present in the SRAM at the point the AHB access is received by the QSPI controller, then the data will be fetched from SRAM and the response to the read burst will be achieved with minimum latency. Once the data has been read from SRAM, the QSPI controller will free up the associated resource in the SRAM.

If an AHB read access is received whose address is not within the range described above then that access will not be completed using the indirect controller. It will instead be serviced by the direct access controller.

If an AHB read access is received whose address is within the range described above but the requested data is not immediately present in the SRAM then AHB wait states will be applied until the data has been read from FLASH and pushed to the SRAM.

If an AHB read burst is received whose access elements traverse the AHB Indirect trigger range, then the accesses within the Indirect trigger range will be processed by the indirect controller and the rest will be taken by the direct access controller. Note this is likely to be a software configuration error.

The external master is only permitted to issue 32-bit AHB reads until the last word of an indirect transfer. This helps keep the SRAM control logic less complex. On the final read, the external master may issue a 16-bit (HalfWord) or byte access to complete the transfer. It is

also permitted for the external master to always issue a 32-bit Word read on the last indirect access. The controller will pad the upper bits of the response with zero.

The current expectation is that the SRAM will be kept fairly full while the read operation is carried out. The fill level of the SRAM is directly readable by software (0x2c). If the DMA peripheral interface controller is enabled, this will automatically request an external DMA fetch the data from the SRAM via the AHB in efficient chunks of data, each chunk being a part of the overall indirect transfer.

An indirect operation may be cancelled at any time by setting Indirect Transfer Control Register bit [1].

Any bus master should be allowed to initiate an indirect access. The DMA bus peripheral interface has been included in the design to offload some of the necessary software overhead to efficiently manage data transfers when an external DMA supporting this interface is added. The alternative to this is for software to access the SRAM fill-level directly via APB registers and then decide for itself when the data should be fetched from the local SRAM. When the DMA peripheral interface is disabled, the fill level watermark register accessible via the Indirect Transfer Watermark Register (0x64) is provided. When the SRAM fill level passes this watermark, an interrupt is generated. If the watermark value is >0, the watermark interrupt is also generated when the final byte of data has been read by the QSPI controller and placed in the SRAM, even if the actual SRAM fill level has not risen above the watermark. This last feature is useful to avoid software tracking how much data has been read and resetting the watermark value for the last few bytes of an indirect read transfer. Note that this watermark register is a dual-use register. When the DMA peripheral interface is enabled, it is used by hardware to control the rate at which the DMA requests are issued. When the DMA peripheral interface is disabled, the behavior is as described above.

Two further interrupt sources are provided to help understand the status of an indirect operation. Firstly, an interrupt is generated when an indirect operation has completed. Secondly, an interrupt is generated if an indirect read operation was requested but could not be accepted due to the fact 2 indirect operations have already been buffered by the QSPI controller.

Setting Indirect Transfer Control Register (0x60) bit[0] starts an indirect read operation, and a status bit (0x60 bit[2]) will be available to check the status.

### Indirect read transfer process

When the DMA peripheral controller is enabled, the following sequence can be followed

1. Setup QSPI Configuration register (0x00)
2. Setup the SRAM watermark in the Indirect Transfer Watermark Register (0x64)
3. Setup the indirect transfer's FLASH start address in the Indirect Transfer Start Address Register (0x68)
4. Setup the number of bytes to be transferred in the Indirect Transfer Number Bytes Register (0x6c)
5. Setup the indirect transfer's AHB trigger address in the Indirect AHB Trigger Address Register (0x1c)
6. Setup the indirect transfer's AHB trigger address range in the Indirect AHB Trigger Address Range Register (0x80)
7. Setup number of bytes for 'single' and 'burst' DMA peripheral burst type transfers in the DMA Peripheral Configuration Register (0x20)
8. Trigger Indirect Read access by setting Indirect Transfer Control Register[0] = '1'.
9. The completion status of the indirect read operation can be polled via the Indirect Transfer Control Register (0x60). Note this bit is write-to-clear. Alternatively, an Indirect Complete interrupt will be generated when the Indirect read operation has completed.
10. The number of indirect read operations completed will be readable in the same register.

When the DMA peripheral controller is disabled, the following sequence can be followed

1. Setup QSPI Configuration register (0x00)

2. Setup the indirect transfer's FLASH start address in the Indirect Transfer Start Address Register (0x68)
3. Setup the number of bytes to be transferred in the Indirect Transfer Number Bytes Register (0x6c)
4. Setup the indirect transfer's AHB trigger address in the Indirect AHB Trigger Address Register (0x1C)
5. Setup the indirect transfer's AHB trigger address range in the Indirect AHB Trigger Address Range Register (0x80)
6. If the watermark interrupt feature is to be used, set the SRAM fill level watermark register (0x64) which will cause an interrupt to be generated when the fill level increases beyond the watermark level. Setting the watermark can be useful indication to software when to read the next part of the indirect read transfer. Note that if the watermark is set to a value other than zero, the watermark interrupt will always trigger once the final byte of indirect transfer has been fetched and placed in the embedded SRAM, even if the watermark value is higher than the actual completed fill level.
7. Trigger Indirect Read access by setting Indirect Transfer Control Register[0] = '1'.
8. If the watermark interrupt feature is to be used, wait for watermark interrupt. Else poll the SRAM fill level to decide when sufficient data is in the SRAM to trigger AHB data fetches
9. Read the expected amount of data from SRAM. If there is still more data to fetch in order to complete the indirect read transfer, then loop back to 8. Otherwise continue to 10.
10. The completion status of the indirect read operation can be polled via the Indirect Transfer Control Register (0x60)
11. An Indirect Complete interrupt will be generated when the Indirect read operation has completed

### Indirect Write Controller

The aim of the indirect mode of operation is to perform bulk transfer of data from the processor or DMA into FLASH memory in the most efficient manner. The fewest possible write cycles inside the FLASH device will be carried out for the indirect transfer, thus maximizing the life of the device.

Indirect write operation can be thought of from a software perspective as the inverse of the indirect read. It is controlled and triggered by software via specific APB control/configuration registers (0x70 → 0x7c). This block will await delivery of the write data via the external AHB master, placing it in the local SRAM before communicating with the existing legacy SPI IP core to perform an efficient and optimized FLASH write burst.

By default, the indirect write controller is disabled. Before enabling it, software must configure how much data is required and starting from what address. The start address and total number of bytes to be written is defined in registers at addresses 0x78 and 0x7c respectively. Up to two indirect operations can be programmed at any one time. The second can be triggered while the first is in progress. Supporting two indirect operations allows a short turnaround time between the completion of one indirect operation and the start of the second. Indirect write queuing is very similar to indirect read queuing. Refer to the indirect transfer queuing section 1 for further details.

The total number of bytes to write in an indirect operation is not limited by the size of the SRAM. The size of SRAM will only limit the amount of data that can be accepted from the external AHB master. When the external master is the DMA, the amount of data requested by the controller via the DMA peripheral interface will never exceed the current fill level of the SRAM. However, this does not guarantee in itself that the DMA, or other external master, if the DMA is not used, will not attempt to push more data to the SRAM than the SRAM can accept. In the case of an SRAM overrun, the controller will back pressure the AHB with wait states. Note the fill level of the SRAM is readable via programmable registers (0x2c) and this can be used to avoid this situation.

An external master will provide the write data and will transfer this to the QSPI controller by issuing AHB writes. The AHB address of the incoming write access must be in the range AHB Indirect trigger address (programmed via the Indirect AHB trigger address register) to AHB Indirect trigger address +  $2^{**}(\text{Indirect AHB trigger address range}) - 1$ . Default value of the range is equal to 16 locations. This allows a 16-beat burst to be applied starting from the AHB Indirect trigger address. The smaller bursts are possible to handle effectively as well with this approach. Furthermore it is not strict requirement to push consecutive address sequence. Actual AHB address just has to be in the Indirect Range to grant SRAM as source. Each valid AHB Indirect write will cause the internal SRAM to be pushed, thereby decoupling the AHB address from the FLASH address – i.e. not direct mapped. Therefore AHB Indirect trigger address does not have any relationship with FLASH address. It is just to indicate that data should take SRAM as source instead of FLASH Memory array after triggering of any valid Indirect Write. FLASH address for Indirect Write is taken from the Indirect Transfer Start Address Register (0x78). Assuming the SRAM is not full at the point the AHB access is received by the QSPI controller, then the data will be pushed to the SRAM with minimum latency.

If an AHB write access is received whose address is not within the range described above then that access will not be completed using the indirect controller. It will instead be serviced by the direct access controller.

If an AHB write access is received whose address is within the range described above but the SRAM is full, then AHB wait states will be applied until some or all of the data has been pushed from SRAM onto the FLASH.

If an AHB write burst is received whose access elements traverse the AHB Indirect trigger range, then the accesses within the Indirect trigger range will be processed by the indirect controller, and the rest will be taken by the direct access controller. Note this is likely to be a software configuration error.

The external master is only permitted to issue 32-bit AHB writes until the last word of an indirect transfer. This helps keep the SRAM control logic less complex. On the final write, the external master may issue a 32-bit word, 16-bit (half-word) or a byte access to complete the transfer. If the number of bytes to write is less than 4 on the last transfer, the master is still permitted to issue a 32-bit transfer. In these cases, the extra bytes are discarded by the controller.

When the SRAM holds a number of bytes equal to or greater than the size of a FLASH page (which itself is programmed into the controller, with a default of 256 bytes) or when the SRAM holds all remaining bytes of the currently executing indirect transfer, the controller will initiate a write burst to the command generator.

An indirect operation may be cancelled at any time by setting Indirect Transfer Control Register bit [1].

Any bus master should be allowed to initiate an indirect access. The DMA bus peripheral interface has been included in the design to offload some of the necessary software overhead to efficiently manage data transfers when an external DMA supporting this interface is added. The alternative to this is for software to access the SRAM fill-level directly via APB registers and then decide for itself when the data should be written to the local SRAM. When the DMA peripheral interface is disabled, the fill level watermark register accessible via the Indirect Transfer Watermark Register (0x74) is provided. When the SRAM fill level falls below this watermark, an interrupt is generated. Note that this watermark register is a dual-use register. When the DMA peripheral interface is enabled, it is used by hardware to control the rate at which the DMA requests are issued. In this mode, since the QSPI will not initiate a write operation unless there is at least one FLASH page worth of data in the local SRAM, or the remaining bytes of the indirect transfer are in the SRAM, then it is important the watermark in DMA mode is programmed to a value  $\geq 1$  flash page.

Two further interrupt sources are provided to help understand the status of an indirect operation. Firstly, an interrupt is generated when an indirect operation has completed. Secondly, an interrupt is generated if an indirect write operation was requested but could not be accepted due to the fact 2 indirect operations have already been buffered by the QSPI controller.

Setting Indirect Transfer Control Register (0x70) bit[0] starts an indirect write operation, and a status bit (0x70 bit[2]) will be available to check the status.

**Indirect write transfer process**

When the DMA peripheral controller is enabled, the following sequence can be followed

1. Setup the indirect transfer's FLASH start address in the Indirect Transfer Start Address Register (0x78)
2. Setup the number of bytes to be transferred in the Indirect Transfer Number Bytes Register (0x7c)
3. Setup the indirect transfer's AHB trigger address in the Indirect AHB Trigger Address Register (0x1C)
4. Setup the indirect transfer's AHB trigger address range in the Indirect AHB Trigger Address Range Register (0x80)
5. Setup number of bytes for 'single' and 'burst' DMA peripheral burst type transfers in the DMA Peripheral Configuration Register (0x20)
6. Optional: Set the SRAM fill level watermark register (0x74) to control the rate at which DMA requests will be issued.
7. Trigger Indirect Write access by setting Indirect Transfer Control Register[0] = '1'.
8. The QSPI controller will request the DMA to transfer data using the DMA request interface.
9. The completion status of the indirect write operation can be polled via the Indirect Transfer Control Register (0x70). This will be write-to-clear. The number of indirect write operations completed will be readable in the same register.
10. An Indirect Complete interrupt will be generated when the Indirect write operation has completed

When the DMA peripheral controller is disabled, the following sequence can be followed

1. Setup the indirect transfer's FLASH start address in the Indirect Transfer Start Address Register (0x78)
2. Setup the number of bytes to be transferred in the Indirect Transfer Number Bytes Register (0x7c)
3. Setup the indirect transfer's AHB trigger address in the Indirect AHB Trigger Address Register (0x1C)
4. Setup the indirect transfer's AHB trigger address range in the Indirect AHB Trigger Address Range Register (0x80)
5. It is functionally valid for software to simply write all the data to the SRAM in one block transfer. However, if the total number of bytes to write is greater than the size of the partitioned SRAM, then it is quite likely the SRAM will become full causing the QSPI to back-pressure the system AHB bus for a considerable time. This time is based on the FLASH data-rate and the page-write time of the device. To avoid sending all the write data in one block transfer, software can make use of the watermark interrupt to identify a convenient time to send data a page at a time to the SRAM module. Alternatively, software can poll the SRAM fill level register directly to identify how empty the SRAM is at any one time in order to make a judgment as to when the most practical time to send the next part of the transfer.
6. If the watermark interrupt feature is to be used, set the SRAM fill level watermark register (0x74) which will cause an interrupt to be generated when the fill level falls below the watermark. The watermark should be set to a number between zero and a page size. Eg if the page size is 256 bytes, then setting the watermark to a value between 10 and 250 is reasonable and will cause the interrupt to trigger when the fill level drops below the programmed number. Setting the watermark can be useful to provide an indication to software when to write the next page of data to the SRAM.
7. Trigger Indirect Write access by setting Indirect Transfer Control Register[0] = '1'.
8. If the remaining number of bytes still to be transferred into the QSPI SRAM for the current indirect transfer is greater than a FLASH page, then write 1 FLASH page worth of data to the SRAM. Otherwise send the remaining data from the indirect transfer to SRAM.

9. If all the data in the indirect transfer has now been sent to the SRAM, then goto 11 and await indirect complete status. Otherwise if there is more data still to be transferred then either
  - a. If the watermark interrupt feature is being used, then wait for watermark interrupt.
  - b. Alternatively the SRAM fill level can be interrogated to identify a convenient time to send more data.
10. Loop back to 8)
11. Optional : The completion status of the Indirect write operation can be polled via the Indirect Transfer Control Register (0x70)
12. An Indirect Complete interrupt will be generated when the Indirect write operation has completed

### Indirect Access Queuing

Software is permitted to queue up to two indirect transfers for both the indirect write controller and the indirect read controller. Supporting two indirect operations allows a short turnaround time between the completion of one indirect operation and the start of the second. Any attempt to queue more than two operations will cause an interrupt to be generated. To take advantage of this feature, software should attempt to keep both indirect programming slots full at all times.

From the software perspective, indirect access queuing is achieved by triggering bit 0 of the indirect transfer control register (0x60 or 0x70) twice in short succession. The indirect number of bytes register and the indirect FLASH start address register must be setup with the relevant transfer data before bit 0 can be triggered for each transfer. Since these registers will change regularly, the hardware must keep sampled versions of these registers for the duration of the indirect transfer.

The internal register block will only issue an indirect start trigger to the key underlying datapath blocks one at a time. There are 2 independent datapath blocks in the indirect access controller that will receive and independently sample this information. The first is the datapath block on the AHB side of the SRAM. For indirect reads, this is a read interface, for indirect writes, it is a write interface. The second is the datapath block on the FLASH side of the SRAM. For indirect reads, this is a write interface, for indirect writes, it is a read interface. Both blocks will process the indirect transfers at different times. For example, for an indirect read operation, the datapath block on the FLASH side of the SRAM will be able to start processing the second queued transfer as soon as the last byte of the first transfer has been written to SRAM. Before commencing the second transfer, this block must resample the number of bytes and the indirect FLASH start address register. Similarly, the datapath block on the AHB side will resample the same registers locally when it has forwarded all the FLASH data associated with the first indirect transfer from SRAM onto AHB.

### Consecutive Writes and Reads using Indirect Transfers

It is permitted for software to trigger an indirect read operation while an indirect write operation is in progress. Similarly it is permitted to trigger an indirect write while an indirect read operation is in progress. Indirect write operations will take overall precedence.

### Accessing the SRAM

Physically, the SRAM will be a single port macro. The depth of this SRAM is a configuration choice at compile time. It will be segmented into two halves. The lower half is reserved for indirect read use. The upper is for indirect write use only. The size of each half will be programmable via the SRAM Partition Register (0x18). This allows the programmer to allocate how many bits of the SRAM address bus are allocated to indirect read. By default, this is set so that exactly half of the SRAM is portioned for use by the indirect read controller. To ensure the AHB read data bus is not directly fed by the SRAM read data through combinatorial logic, an extra bank of holding registers is included in the indirect read data

path. These registers act as an extra location to be added to the allocated number of SRAM locations for indirect read.

To illustrate how the SRAM (and the extra bank of holding registers) can be allocated between indirect read and write, the following example is provided. The depth of the SRAM in this example is configured to be 8 bits, equaling 256 locations...

- If the SRAM Partition Register is set to 0x00, then 256 locns are allocated to indirect writes and 1 locn to indirect reads.
- If the SRAM Partition Register is set to 0x01, then 255 locns are allocated to indirect writes and 2 locns to indirect reads.
- If the SRAM Partition Register is set to 0x02, then 254 locns are allocated to indirect writes and 3 locns to indirect reads.
- And so on until ...
- If the SRAM Partition Register is set to 0xfd, then 3 locns are allocated to indirect writes and 254 locns to indirect reads.
- If the SRAM Partition Register is set to 0xfe, then 2 locns are allocated to indirect writes and 255 locns to indirect reads.
- If the SRAM Partition Register is set to 0xff, then 1 locn is allocated to indirect writes and 256 locns to indirect reads.

\*Note that a value of 0xFF or 0x00 in the SRAM Partition Register should be avoided by software, as only the bottom 8 bits of the SRAM fill level are accessible through software (i.e. up to 255 limit) via the SRAM Fill Level Register. I.e. if the fill level reaches 256 on either the indirect read or write side, it will appear when reading the Fill Level to be 0.

There are four SRAM sources that are arbitrated and muxed onto the single SRAM port. Up to three sources may be accessing this port at any one time. The sources are described as follows:

1. Indirect Write, Write source. This is located on the AHB side of the SRAM.
2. Indirect Write, Read source. This is located on the FLASH side of the SRAM.
3. Indirect Read, Write source. This is located on the FLASH side of the SRAM.
4. Indirect Read, Read source. This is located on the AHB side of the SRAM.

A fixed priority arbitration scheme is implemented. The priority allocated to these sources is set as follows:

### SRAM Access Priority

|                |                                       | SRAM Access Priority                   |
|----------------|---------------------------------------|----------------------------------------|
| Indirect Write | Write to SRAM (From System AHB)       | 3rd (exclusive with AHB read request)  |
|                | Read from SRAM (From QSPI Controller) | 2nd                                    |
| Indirect Read  | Write to SRAM (From QSPI Controller)  | 1st                                    |
|                | Read from SRAM (From System AHB)      | 3rd (exclusive with AHB write request) |

With the exception of the write port during an Indirect Read operation (on the FLASH side of the SRAM), the logic driving all four sources must not assume single cycle completion. Writes to the SRAM during an indirect read must be allowed to complete immediately to avoid data loss. Therefore this port is given maximum priority.

## DMA Peripheral Controller

The peripheral interface will be used to trigger an external DMA into performing low-latency AHB data burst accesses to the controller. The DMA peripheral interface is directly meant for is the CoreLink DMA Controller DMA-330 core from ARM. The DMA peripheral interface is only used for the Indirect modes of operation where data is buffered in local SRAM in order to quickly service incoming AHB requests and have the core perform the low level FLASH transfers over a longer period of time. There are two identical DMA interfaces, one for each indirect controller. For the indirect read controller, the QSPI controller will only issue DMA requests after the data has been retrieved from FLASH and written to local SRAM. For the indirect write controller, the QSPI controller will issue DMA requests immediately after the transfer is triggered and will continue to do so until the entire indirect write transfer has been transferred. The rate at which these requests are made can be altered using the watermark registers.

### Interface

The following is true for both DMA interfaces.

The peripheral controller is responsible for driving the DR (peripheral to DMA) request bus and for acknowledging the DA (DMA to peripheral) acknowledge bus. The two buses are simple AXI like interfaces, which consist of a two-way VALID and READY handshake mechanism.

For the DR request bus, the controller drives the 'drvalid' signal when valid information is available on 'drtype' and 'drlast' signals. This information will remain set until the external DMA drives the 'drready' signal, indicating it can accept the request.

For the DA acknowledge bus, the DMA drives the 'davalid' signal high when valid information is available on 'datatype' signal. This information will remain set until the DMA peripheral controller drives the 'daready' signal high, indicating it has accepted the acknowledgement made by the DMA.

The controller will issue a request via the DR bus to the DMA when it requires the DMA (or other AHB master) to transfer data from the controller via its AHB slave port. The controller will issue two types of requests to achieve this, single or burst. Each of these request types will correspond to a programmable but fixed number of bytes. By default, both of these types will reset to a request size of just 1 byte. The DR bus is also used to acknowledge a Flush request which is described in more detail below. The DR request uses the 'drtype' bus to indicate the request type.

The DMA uses the DA bus to indicate when the DMA has completed the data transfer. This is acknowledged by the controller by driving the 'daready' signal, but no other action is taken. The DMA also uses the DA bus to request that the controller perform a flush operation. When this occurs, the controller will re-evaluate the FIFO levels and replay all DMA requests that may have been sent but have yet to be acknowledged. Note if a request on the DR channel is valid at the point the flush is received, the controller must hold the 'drvalid' and 'drtype' signals steady until the DMA acknowledges the request via 'drready'. Once the flush operation has completed, the controller will acknowledge the flush by sending a unique request on the DR channel, setting drtype to 2'b10.

### drtype/datatype Encoding

| drtype/datatype | Action                       |
|-----------------|------------------------------|
| 2'b00           | Single Request / Acknowledge |
| 2'b01           | Burst Request / Acknowledge  |
| 2'b10           | Flush Acknowledge / Request  |

Flushing a write request has no effect on writes that may have been already sent to the FLASH device.

## Order of operation

When an indirect operation is triggered, the total amount of data to be transferred from/to FLASH memory (in bytes) will be visible by the DMA peripheral controller. The controller will split this into a number of DMA burst and single requests by dividing the total number of bytes by the number of bytes programmed in the burst request, and then dividing the remainder by the number of bytes in a single request. It is for software to ensure there is no remainder following these divisions. To ensure the logic is kept simple, the divisions will always be a factor of 2, and the programmable registers defining them must be defined such. These programmable registers are defined in the DMA Peripheral Configuration Register at address 0x20. As an example, if the total amount of data selected to be read from FLASH using the low-latency Indirect mode is 512 bytes, the size of the SRAM is a fixed 256 bytes and software configures the burst type transfer size to be 256 bytes, then the SPI controller will trigger a DMA burst request when the first 256 bytes have been locally buffered. It can only then trigger the 2<sup>nd</sup> burst request when a further 256 bytes have been buffered in the local SRAM. Since the SRAM size was only 256 bytes itself, this means the DMA will have had to fetch the entire contents of the SRAM before the next request can be issued. The SRAM fill level will be made visible to the DMA peripheral controller. For indirect reads, requests will be issued to the external DMA based on the following state machine:

- 1) Wait for START trigger
- 2) If the next request to send is a BURST request, wait until both of the following is TRUE.
  - a. SRAM fill level is greater than or equal to the value stored in the burst\_size register
  - b. SRAM fill level is greater than or equal to the value stored in the watermark register

ACTION when TRUE:

  - SEND BURST.
  - Calculate local expected version of fill level - (SRAM fill level - num\_bytes\_requested\_to\_DMA)
  - If the next request to send is a BURST request, move to 4)
  - If the next request to send is a SINGLE request, move to 5)
  - Else move to 1)
- 3) If the next request to send is a SINGLE request, wait until the following is TRUE
  - a. SRAM fill level is greater than or equal to the value stored in the single\_size register

ACTION when TRUE:

  - SEND SINGLE.
  - Calculate local expected version of fill level - (SRAM fill level - num\_bytes\_requested\_to\_DMA)
  - If the next request to send is a SINGLE request, move to 5)
  - Else move to 1)
- 4) The next request to send is a BURST request. If the (SRAM fill level - num\_bytes\_requested\_to\_DMA) is less than the value stored in the burst\_size register then goto 2), else
 

ACTION:

  - SEND BURST.
  - Calculate local expected version of fill level - (SRAM fill level - num\_bytes\_requested\_to\_DMA)
  - If the next request to send is a BURST request, move to 4)
  - If the next request to send is a SINGLE request, move to 5)
  - Else move to 1)
- 5) The next request to send is a SINGLE request. If the (SRAM fill level - num\_bytes\_requested\_to\_DMA) is less than the value stored in the single\_size register then goto 3), else:
 

ACTION:

  - SEND SINGLE.
  - Calculate local expected version of fill level - (SRAM fill level - num\_bytes\_requested\_to\_DMA)
  - If the next request to send is a SINGLE request, move to 5)
  - Else move to 1)

For indirect writes, requests will be issued to the DMA based on the following state machine:

- 1) Wait for START trigger
- 2) If the next request to send is a BURST request, wait until both of the following is TRUE.
  - a. Remaining space in SRAM is greater than or equal to the value stored in the burst\_size register
  - b. SRAM fill level is less than or equal to the value stored in the watermark register

ACTION when TRUE:

- SEND BURST.
- Calculate local expected version of fill level - (SRAM fill level + num\_bytes\_requested\_to\_DMA)
- If the next request to send is a BURST request, move to 4)
- If the next request to send is a SINGLE request, move to 5)
- Else move to 1)

- 3) If the next request to send is a SINGLE request, wait until the following is TRUE
  - a. Remaining space in SRAM is greater than or equal to the value stored in the single\_size register

ACTION when TRUE:

- SEND SINGLE.
- Calculate local expected version of fill level - (SRAM fill level + num\_bytes\_requested\_to\_DMA)
- If the next request to send is a SINGLE request, move to 5)
- Else move to 1)

- 4) The next request to send is a BURST request. If the (Remaining space in SRAM - num\_bytes\_requested\_to\_DMA) is less than the value stored in the burst\_size register then goto 2), else

ACTION:

- SEND BURST.
- Calculate local expected version of fill level - (SRAM fill level + num\_bytes\_requested\_to\_DMA)
- If the next request to send is a BURST request, move to 4)
- If the next request to send is a SINGLE request, move to 5)
- Else move to 1)

- 5) The next request to send is a SINGLE request. If the (Remaining space in SRAM - num\_bytes\_requested\_to\_DMA) is less than the value stored in the single\_size register then goto 3), else

ACTION:

- SEND SINGLE.
- Calculate local expected version of fill level - (SRAM fill level + num\_bytes\_requested\_to\_DMA)
- If the next request to send is a SINGLE request, move to 5)
- Else move to 1)

For the indirect read operation, a programmable SRAM watermark register (0x64) will define the minimum fill level that the controller can issue the first DMA request – the higher this figure is, the more data that must be buffered in SRAM before the DMA will collect it. For the indirect write operation, a programmable SRAM watermark register (0x74) will define the maximum fill level that the controller can issue the first DMA burst/single request. The watermark registers allow the system to optionally concentrate AHB transfers over a short period of time. By default, the watermark registers are reset to zero, meaning the peripheral controller may issue DMA request as soon as possible. For the indirect read case, this means that there is sufficient data in the SRAM to perform a burst type request, or a single type request if there less data left to transfer than the number of bytes programmed in the burst type request. Note the DMA peripheral controller must never send an indirect read DMA request if the SRAM is empty, or an indirect write DMA request if the SRAM is full.

The DMA peripheral controller will send a trigger event to the external DMA by setting *drtype*[1:0] to the required type of access (single, burst or flush) along with *drvalid* for a single HCLK cycle to tell the DMA that there is READ data available. *drlast* is asserted at the same time as *drtype* to indicate the last request of the current transfer sequence. *drready* is driven by the DMA to indicate that it can accept requests from the Peripheral Controller.

The DMA will indicate completion of a previously requested data transfer which takes place on the AHB bus by asserting ACK on *datatype* along with *davalid*. *daready* is used to indicate to the DMA that the Peripheral Controller is available to accept status information on *datatype*. A series of requests will be sent on the DA bus until all data read/written from/to the FLASH device that is stored in the SRAM has been acknowledged as being transferred to/from the DMA.

The peripheral interface only acts on the DA acknowledge channel if the DMA issues an abort request. All other requests are simply accepted using *daready* as required. This means the controller is free to pipeline the DR requests as much as it likes, and will be limited only by the pipelining requirements of the DMA.

The DMA peripheral interface may be disabled via software. When the interface is disabled, no DR channel request will ever be issued. An incoming flush request on the DA channel will be internally ignored, although the *daready* must still be set to acknowledge the flush request. Note if an AHB master other than the DMA will perform the data transfer for indirect operations, the DMA peripheral interface must be disabled.

## Software Triggered Instruction Generator (STIG)

The direct and indirect access controllers are used to transfer data. In order to access the volatile and non-volatile configuration registers, the legacy SPI Status register, other status/protection registers as well as to perform ERASE functions, a separate software controller is required. The software triggered instruction generator, or STIG, is controlled using the Flash Command Control register (0x90) by setting up the command to issue to the FLASH device. This is a generic controller and can be used to perform any instruction that the FLASH device supports from the extended SPI protocol. Configuring of instructions which are not compliant with the specification of FLASH Devices could cause unpredicted behavior of the controller. Bit 0 is used to trigger the command, bit 1 used by software to poll the status of the command execution. For reads, when the command has been serviced (bit 1 toggles from '1' to '0'), up to 8 bytes of read data will be placed in the Flash Command Read Data Register (0xA0 and 0xA4). For writes, the write data should be placed in the Flash Command Write Data Register (0xA8 and 0xAC).

The code to implement this will be located inside the APB register block of the QSPI controller.

### Servicing a STIG request

A STIG request will cause the QSPI Flash Controller to interrogate the Flash Command Control Register (0x90) to determine what and how many bytes it should send to the FLASH device. Bits [31:24] of this register indicate the instruction to be sent and is always pushed first. If there is an address to send, then the address (the size of which is also programmed in the same register) is sent next. The address itself is stored in the Flash Command Address Register (0x94). If there are any dummy bytes to send (the size of which is also programmed in 0x90) then those are sent next. If there is data to write or read (the size of which is also programmed in 0x90) then for the case of writes, up to 8 bytes can be sent (as stored in the Flash Command Write Data registers, 0xA8 and 0xAC) next. In the read case, when the read data has been collected from the FLASH device, the QSPI Flash Controller stores that in the Flash Command Read Data Registers (0xA0 and 0xA4).

When the QSPI Flash Controller starts to service a STIG request, it sets bit 1 of the Flash Command Control register to indicate a command execution is in progress.

When the QSPI Flash Controller is in the auto-polling state, servicing a STIG request is slightly different. Most of devices are largely inaccessible after a program operation until the device has completed that write. Some group of them has a possibility to suspend programming page. It can be controlled by 8th bit of Polling Flash Status Register, which indicate active auto-polling phase. After requesting a STIG, the QSPI Flash Controller immediately issues appropriate OPCODE to Memory. During servicing a STIG (in auto-polling phase) the status bit of command execution remains steady and other parts of

transfer such as ADDRESS or DUMMY BITS etc. are disabled (to issued Program Suspend Command is needed OPCODE only).

There is a programmable option to add delay between every repetitive poll operation (delay is defined by MSB of Write Completion Control Register). This feature is implemented to free up SPI bandwidth if needed.

### Arbitration between Direct / Indirect Access Controller and STIG

When multiple controllers are active simultaneously, a simple fixed-priority arbitration scheme is used to arbitrate between each interface and access the external FLASH. The fixed priority is defined as follows, highest priority first.

1. The Indirect Access Write
2. The Direct Access Write
3. The STIG
4. The Direct Access Read
5. The Indirect Access Read

Each controller is back pressured while waiting to be serviced.

### SPI Command Translation

Requests issued by the direct access controller, the indirect access controller or the STIG will be translated into a sequence of byte transfers to send downstream (before serialization to the FLASH device). These sequences depend on the requested transfer but an example of a typical 1-byte non sequential READ is shown below:

- INSTRUCTION OPCODE -> ADDRESS -> Mode Byte -> Dummy Bytes -> 1 byte of don't care

For sequential accesses, an extra byte of data per read is pushed to the FLASH device on the back of the above sequence assuming it can be done so with no gap between each transferred byte.

The actual sequence sent to the FLASH device depends on the requested transfers, whether the transfer is non-sequential or sequential, whether the device has been configured in XIP mode and the state of the main Device Instruction Type programmable registers (at offsets 0x4 and 0x8)

For writes, the write enable latch (or WEL) within the FLASH device itself must be high before a write sequence can be issued. The QSPI Flash Controller will automatically issue the write enable latch command before triggering a write command via the direct or indirect access controllers (DAC/INDAC) – i.e the user does not need to perform this operation. For increasing flexibility and performance user can turn off this feature by setup bit 8th into the Device Write Instruction Configuration Register. The op-code for WREN is typically 0x06 and is common between devices.

When write requests from the direct or indirect access controllers are no longer being received and all outstanding requests have been sent, the FLASH device will automatically start the page program write cycle. Any incoming request at this time will be held in wait states until the cycle has completed. The QSPI Flash Controller will automatically poll the FLASH device legacy SPI status register to identify when the write cycle has completed. This is achieved by sending the RDSR opcode to the FLASH device and waiting until the device itself has indicated the write cycle has completed (until the Write in Progress bit [0] has cleared to zero and the write enable latch bit has also cleared to zero). The WREN and the RDSR device instructions are the only ones that are sent by the controller under the hood. For any other specific instruction that the user determines should be sent to the device (for example if the device needs to be unprotected before a write command is issued), these should be handled separately by issuing FLASH commands via the STIG.

## Selecting the Flash Instruction Type

In order to send the correct READ and WRITE opcodes, software should initialize the Device READ Instruction Type Register (Offset 0x4) and the Device WRITE Instruction Type Register (Offset 0x8). These registers include fields to setup the required instruction opcodes that is intended to be used to access the FLASH (default is basic READ and basic page program) as well as the instruction type, edge mode (DDR or SDR) and whether the instruction uses single, dual or quad pins for address and data transfer. Providing this level of control to the user provides a future proofed generic solution. To ensure the controller can operate from a reset state, the registers will be reset to an op-code compatible with SIO devices what can be modified using BOOT feature.

Despite being applicable for both READs and WRITES, the Instruction Type field in the Device READ Instruction Register only appears once – it is not included in the Device WRITE Instruction Register. If software sets this to anything other than '0', then the address xfer type and the data xfer type bits of both Device READ Instruction Type Register and Device Write Instruction Type Registers become don't care. It is made available to allow software to support the less common FLASH instructions where the opcode, address and data are sent on 2 or 4 lanes (the opcode from most instructions are sent serially to the FLASH device, even for dual/quad instructions). Also note that for devices that support instructions that can send the OPCODE over 2/4 lanes, the name for these instructions are not consistently named in the FLASH datasheets. One of the devices that support these instructions is the Numonyx (Micron) N25Q128. The extra READ instructions are called DCFR and QCFR. The WRITE instructions are DCPW and QCPW. For reference, below is a table illustrating how software should configure the QSPI for each specific READ and WRITE instruction supported by the N25Q128 device.

### READS

| OPCODE                        | OPCODE sent over how many lanes? | ADDRESS/DUMMY/MODE sent over how many lanes? | DATA bytes sent over how many lanes? | QSPI Instruction TYPE (Device Read Instruction Config Register) | QSPI Address Xfer type (Device Read Instruction Config Register) | QSPI Data Xfer type (Device Read Instruction Config Register) |
|-------------------------------|----------------------------------|----------------------------------------------|--------------------------------------|-----------------------------------------------------------------|------------------------------------------------------------------|---------------------------------------------------------------|
| READ                          | 1                                | 1                                            | 1                                    | 0                                                               | 0                                                                | 0                                                             |
| FAST_READ                     | 1                                | 1                                            | 1                                    | 0                                                               | 0                                                                | 0                                                             |
| DOFR (Dual O/p Fast Read)     | 1                                | 1                                            | 2                                    | 0                                                               | 0                                                                | 1                                                             |
| DIOFR (Dual I/O Fast Read)    | 1                                | 2                                            | 2                                    | 0                                                               | 1                                                                | 1                                                             |
| QOFR (Quad O/p Fast Read)     | 1                                | 1                                            | 4                                    | 0                                                               | 0                                                                | 2                                                             |
| QIOFR (Quad I/O Fast Read)    | 1                                | 4                                            | 4                                    | 0                                                               | 2                                                                | 2                                                             |
| DCFR (Dual Command Fast Read) | 2                                | 2                                            | 2                                    | 1                                                               | DON'T CARE                                                       | DON'T CARE                                                    |
| QCFR (Quad Command Fast Read) | 4                                | 4                                            | 4                                    | 2                                                               | DON'T CARE                                                       | DON'T CARE                                                    |

**WRITES**

| OPCODE                                   | OPCODE sent over how many lanes? | ADDRESS/DUMMY/MODE sent over how many lanes? | DATA sent over how many lanes? | QSPI Instruction TYPE (Device Read Instruction Config Register) | QSPI Address Xfer type (Device Write Instruction Config Register) | QSPI Data Xfer type (Device Write Instruction Config Register) |
|------------------------------------------|----------------------------------|----------------------------------------------|--------------------------------|-----------------------------------------------------------------|-------------------------------------------------------------------|----------------------------------------------------------------|
| PP                                       | 1                                | 1                                            | 1                              | 0                                                               | 0                                                                 | 0                                                              |
| DIFP (Dual Input Fast Program)           | 1                                | 1                                            | 2                              | 0                                                               | 0                                                                 | 1                                                              |
| DIEFP (Dual Input Extended Fast Program) | 1                                | 2                                            | 2                              | 0                                                               | 1                                                                 | 1                                                              |
| QIFP (Quad Input Fast Program)           | 1                                | 1                                            | 4                              | 0                                                               | 0                                                                 | 2                                                              |
| QIEFP (Quad Input Extended Fast Program) | 1                                | 4                                            | 4                              | 0                                                               | 2                                                                 | 2                                                              |
| DCPP (Dual Command Fast Program)         | 2                                | 2                                            | 2                              | 1                                                               | DON'T CARE                                                        | DON'T CARE                                                     |
| QCPP (Quad Command Fast Program)         | 4                                | 4                                            | 4                              | 2                                                               | DON'T CARE                                                        | DON'T CARE                                                     |

There are group of devices (e.g. Numonyx (Micron) N25Q512A) capable to handling Read Operations in Dual Data Rate Mode (DDR) (it is also called Dual Transfer Rate Mode (DTR)). That means they can issue the data on both rising and falling edges during working of dedicated command type. This enables the controller to maintain throughput at twice lower frequency of spi\_clk. The Device READ Instruction Type Register has DDR enable bit which inform QSPI Flash Controller that opcode written into Read Opcode field is capable with DDR command type. The field defined in Read Data Capture Register enables to shift the transmitted data in DDR mode. By default, data are shifted by 1 clock cycle to ensure hold timing greater than 0 during DDR transactions. It can be not sufficient for high ref\_clk frequency in accordance with the high dividers. The Configuration of DDR commands is compliant with SDR ones listed in READ table.

There is also MT25 family of Micron devices where DTR protocol was added. It enables device to handle all commands in DTR Mode. DTR Read commands detect DTR Mode based on dedicated OPCODE. Therefore OPCODE has to be send as STR. When DTR protocol is enabled, device does not need OPCODE to detect edge Mode because can recognize it based on volatile or non-volatile bit from configuration register of itself.

## APB Interface and Register Module

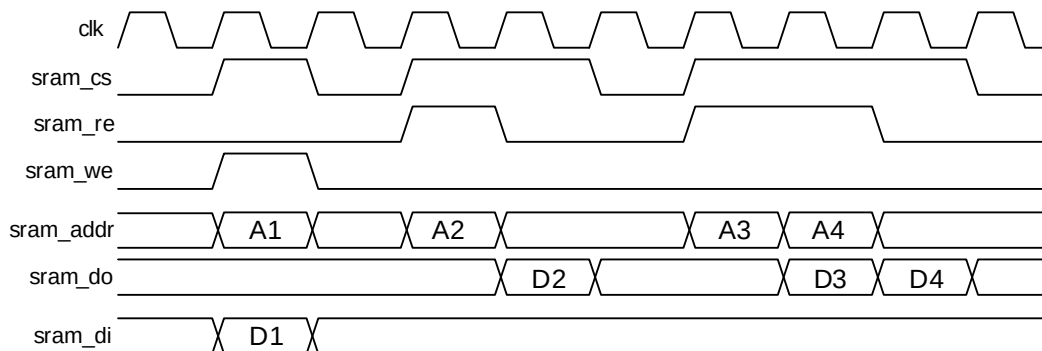
The APB interface conforms to AMBA v3.0. It is used to configure the core and perform software controlled FLASH accesses using the Flash Command Control register (refer to section “Software Triggered Instruction Generator (STIG)” and section “Using the Flash Command Control Register (STIG Operation)” of this document. The APB interface feeds a single register block containing the programmable register set. The register block is timed to the APB clock. All control or enable bits that are triggered by APB writes and cause an event to trigger, or an operation somewhere in the QSPI controller to be enabled are synchronized to the destination clock. Static configuration bits that have no effect while a separate enable bit is low do not require synchronization.

## SRAM Macro

The memory macro is required for the indirect mode of operation. The pins for this memory is routed to the top level allowing the integrator to select whether to use SRAM memory, register arrays or simple banks of flops. The memory macro required will be a single port only. The depth of the memory macro will be a configurable option and is application specific. The SRAM is HCLK timed.

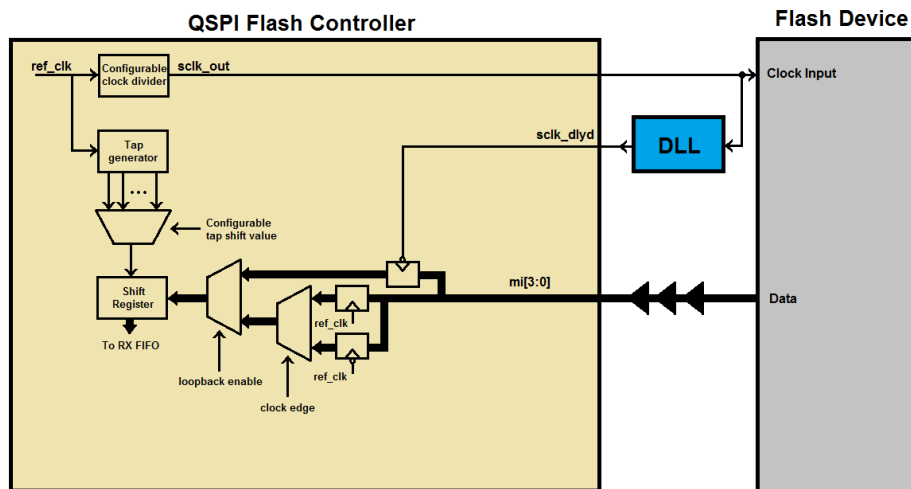
Write access to the SRAM memory takes one clock cycle.

Read access from the SRAM memory takes two clock cycles. Signal `sram_re` is used for latching the address. During the first cycle (of the single read access) `sram_re` is active (high) and memory is latching the address placed on `sram_addr` bus in internal address register. In the second cycle of the read access, SRAM memory is driving the outputs from memory cell selected by the latched address and then `sram_re` is inactive. For sequential read accesses the `sram_re` output is set active in order to read data from subsequent memory cells.



## Read Data Capturing

Sampling data diagram

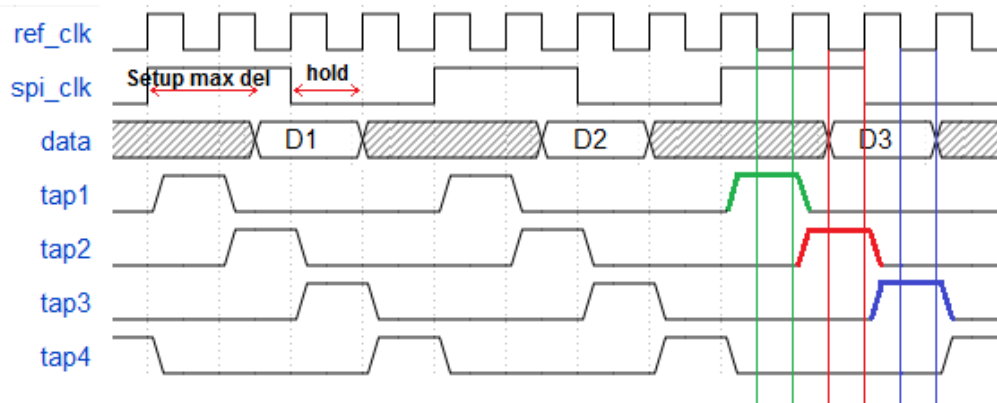


### Mechanism using taps

There are three programmable features allowing software to tune the logic that captures the read data from the device. After POR, the adapted loopback clock circuit and the **ref\_clk** delay register line both wake in a disabled state. This should be valid for most applications, and can be used for device enumeration, where the device clock will be configured slowly. The Read Data Capture register (0x10) provides the control for these features.

Bit 5'th selects the edge of the **ref\_clk**, on which data outputs from flash memory are sampled. There is the compilation option, to turn off capturing the data on negedge of the **ref\_clk** clock. Some systems are very limited over implementation of both negative and positive edges of flip-flops located in single domain. Furthermore necessity of increasing of sampling resolution two times is required for very high performance reads only. When the system does not have such restrictive requirements, it is recommended to remove flip-flops (working on negedge of the **ref\_clk** clock) from the project by change of the dedicated compilation parameter (**BOOT\_EDGES\_SAMPLING**) placed into **cdnsspi\_ctrl\_core\_defs.v** file.

Bits 4:1 controls the additional number of read data capture cycles (this is the fast **ref\_clk**, running at least x4 of the device clock) that should be applied to the internal read data capture circuit. The large clock-to-out delay of the flash memory together with trace delays as well as other device delays may impose a maximum flash clock frequency which is less than the flash memory device itself can operate at. To compensate, software should set this register to a value that guarantees robust data captures.

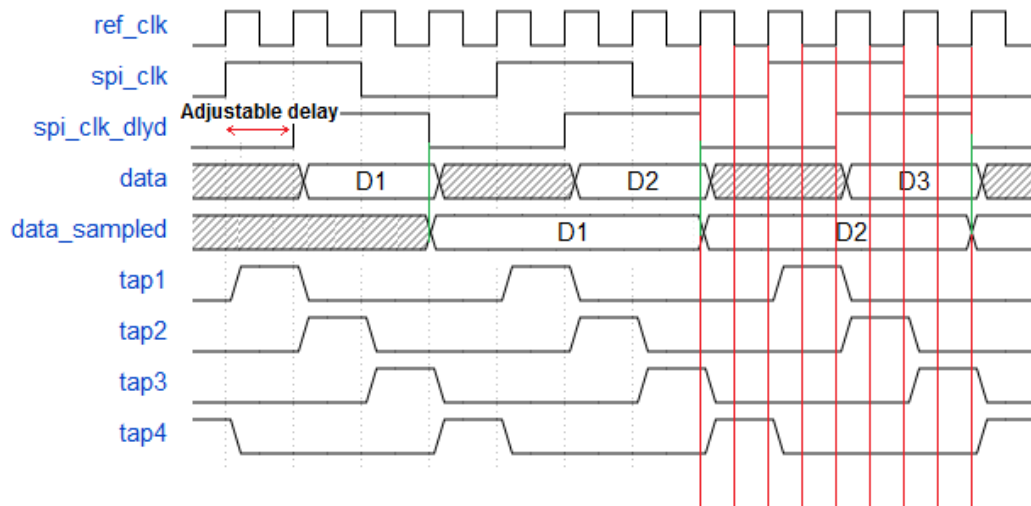


- At least two samples are valid
- Increasing of `ref_clk` allows to achieve better sampling resolution

### Mechanism using external DLL

To further improve the input timing of the design, the master output clock (`sclk_out`) can be looped back in the logic that is instantiating the controller IP and then passed through an adjustable delay (not included in the QSPI IP core). This feature may be required when operating at high speeds with devices that have a very small guaranteed read data sampling window. For example, a device operating at 100MHz (`ref_clk` at 400MHz) where the clock to output valid time can be as high as 8ns or even 9ns, while the output HOLD time can be very low, even as low as just 1ns. This would give a valid sampling window of only a few ns. While this would remain valid across all operating conditions, that window applies to the device only. Extra delays associated with trace and pad in routing the read data back to the QSPI IP core must also be considered, and these will vary depending on operating conditions also. With such a small read data window, the `ref_clk` delay feature (allowing samples to be taken at 1.25ns (both edges) intervals when the `ref_clk` is 400MHz) may not be flexible enough for some purposes. Using the adapted delay line feature can provide a mechanism to allow the sampling point used by the QSPI IP to move beyond the `ref_clk` interval limitation and also may help compensate for operating condition variance. The adapted clock will be an input to the QSPI IP and, when enabled, will be used to clock the first register in the data-capture logic. Note that if the read data window is large enough, the `ref_clk` delay register described earlier will be sufficient to capture all data and the adapted loopback circuit will not be required. Clearing bit 0 enables the adapted loopback clock circuit. Note that before the loopback clock is passed into the QSPI core, it must first be fed through a programmable delay line that is external to this IP. The user is responsible for implementing this delay line. The clock should be delayed such that the rising edge is in the center of the returned read data window.

Note that this mechanism will be working correctly only for SDR commands.



- Data is stable by full clock cycle of spi\_clk\_dlyd

## BOOT feature description

Default BOOT feature assumes following:

- Baud Rate Divisor = /32 (sclk = ref\_clk/32),
- Only CS[0] is active (Default BOOT feature allows to fetch data from single Flash Device only),
- Simple BOOT works in Extended SPI Protocol only (Single Mode),
- Simple BOOT feature uses simple Read Data Command (opcode: 0x03).

To further improve the BOOT feature the user has an option to change of default values of some programmable configuration fields to make BOOT feature more flexible. The controller supports BOOT operation in XIP Mode that allows to achieve the maximum performance. The mentioned parameters can be changed in `cdnsqspi_flash_ctrl_defs_default.v` file before compilation process. Correct setting of them requires checking of structure of individual commands for connected Flash Device. For reference, below is a table illustrating example of BOOT configuration for Fast Read instructions.

### BOOT configuration:

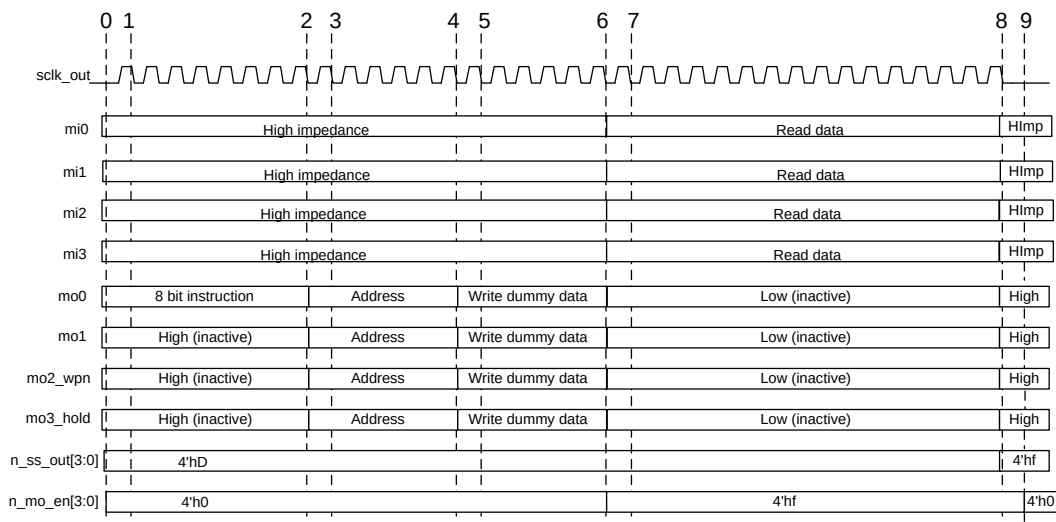
| BOOT Mode          | Parameters to change                                                                                                           |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------|
| 100 MHz QUAD SDR   | boot_baud_rate = 4'h1<br>boot_read_dummy = 5'h08<br>boot_data_type = 2'h2<br>boot_addr_type = 2'h2<br>boot_read_opcode = 8'heb |
| 100 MHz DUAL SDR   | boot_baud_rate = 4'h1<br>boot_read_dummy = 5'h08<br>boot_data_type = 2'h1<br>boot_addr_type = 2'h1<br>boot_read_opcode = 8'hbb |
| 100 MHz SINGLE SDR | boot_baud_rate = 4'h1<br>boot_read_dummy = 5'h08<br>boot_data_type = 2'h0<br>boot_addr_type = 2'h0<br>boot_read_opcode = 8'h0b |
| 50 MHz QUAD DDR    | boot_baud_rate = 4'h3                                                                                                          |

|                      |                                                                                                                                                   |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
|                      | boot_read_dummy = 5'h10<br>boot_data_type = 2'h2<br>boot_addr_type = 2'h2<br>boot_ddr_en = 1'h1<br>boot_read_opcode = 8'hed                       |
| 100 MHz XIP QUAD SDR | boot_baud_rate = 4'h1<br>boot_xip = 1'h1<br>boot_read_dummy = 5'h08<br>boot_data_type = 2'h2<br>boot_addr_type = 2'h2<br>boot_read_opcode = 8'h0b |

The boot\_baud\_rate values presented in the table were calculated based on assumption that frequency of the system clock is 400 MHz. Presented values can be slightly different for various devices. Furthermore the user should be aware that to executing BOOT reads in either DUAL or QUAD Mode the device has to be configured to handle these modes by non-volatile manner. Similarly, it has to be configured to XIP Mode to handle BOOT feature in XIP.

### Example of an 8 byte Read Transfer

To help understand the high level behaviour of the QSPI interface, the following diagram shows the primary stages in a read transfer. Note this is just one type of read instruction so there are many variations on this.



0. Start of transaction, chip select (n\_ss\_out) activates slave 1 (transitions from 4'hf to 4'hd).
1. Start of instruction phase. The first of 8 bits are output only on mo0. Other mo pins unused at this stage.
2. End of instruction phase. The last of the 8 instruction bits is output.
3. Start of address phase. The example uses a 3 byte address output across all 4 mo pins. One address nibble is output per clock cycle so 3 bytes takes 6 cycles.
4. End of address phase. The last address nibble is output using all 4 mo pins.
5. The example read transaction requires 3 bytes of dummy write data. This edge is where the first of 6 nibbles are output across all 4 mo pins.
6. Last nibble of dummy write data. This is the last write phase of the read transfer therefore the output enable (n\_mo\_en) switches from 4'h0 to 4'hf.
7. First cycle of read data on input mi pins. Example is reading 8 bytes which will take 16 cycles using all 4 mi pins. While read data is being received, the mo signals are inactive and low.
8. Last cycle of read data input. Mi input pins switch back to being high impedance. Mo output pins switch to high. Chip enable is de-asserted for slave1. Transaction complete.
9. Output enable switches back to 4'h0 read for next transaction.

## Programmer's Guide

Software is responsible for configuring the QSPI controller before it can communicate with the FLASH device. Refer to the device enumeration section of this document for details on how software may do this following POR.

We assume that the controller's static configuration bits are setup before the QSPI controller is "enabled" via bit [0] of the QSPI Configuration Register. This is necessary in order to avoid any clock boundary issues in paths where metastability protection has not been implemented. If the user wishes to change the controller's configuration, we recommend that the QSPI enable bit is first set to "disabled" before reconfiguring.

### Configuring the QSPI Controller for use after reset

The QSPI Controller has been designed to wake up in a state that is suitable for performing basic reads and writes using the direct access controller. The BASIC read (opcode 0x03) and BASIC write (opcode 0x02) instructions are operations supported by all target devices. The Controller also wakes up with a baud rate divider setting of divide-by-32. Assuming the reference clock is operating at 400MHz after reset, then this means the effective SPI clock is just 12.5 MHz. This should be slow enough to meet all timing requirements of all target devices without any further device programming.

If the target device does not use 3 address bytes, the device size configuration register must be modified to the appropriate size. All devices tested on this project use a page size of 256 bytes, in which case no register update is required.

If software plans to write to the device, and the number of bytes per device page is NOT equal to 256, then the device size configuration register must also be modified. All devices tested on this project use a page size of 256 bytes, in which case no register update is required.

While not a requirement, it is prudent for software to enable the write protect feature prior to enabling the QSPI controller. This will block any AHB writes from taking effect. To do so, the protection registers (0x50, 0x54 and 0x58) should be setup and the number of bytes per device block in the device size configuration register should also be setup.

After initialization, software can read from and write to the FLASH device (albeit slowly) after it has enabled the controller and enabled the DAC. Enabling the controller is achieved with just one write to the QSPI Configuration Register (offset 0x00). Take note to maintain the default values of the baud rate divisor and the default state of CPOL/CPHA which also are located in this register. A write data value of 0x00780081 is recommended.

### Configuring the QSPI Controller for optimal use

To access the flash optimally, software must configure the controller accurately.

1. Wait until any pending STIG or INDAC operation has completed or poll bit 31 of the QSPI Configuration Register (IDLE bit).
2. Disable the DAC – bit 7 of the QSPI Configuration Register. It is permitted, but not necessary to also disable the QSPI controller completely here via bit 0 of the same register.
3. Update the Device Instruction Configuration Register for the instruction type you wish to use for indirect and direct writes and reads.
4. Update mode bit configuration register, if mode bits have been enabled in the device instruction configuration register.
5. Update the device size configuration register if the contents are incorrect. Note parts or all of this register may have been updated after initialization. The number of address bytes is a key configuration setting required for performing reads and writes. The number of bytes per page is required for performing any write. The number of bytes per device block is only required if the write protect feature is used. If the default values are correct for the target device, or if some of the values (not including the number address bytes) were incorrect but device writes were not permitted.
6. Update QSPI device delay register. This register allows the user to tweak how the chip select is driven after each FLASH access. This is required as each device may have different timing requirements. As the serial clock frequency is increased, these timing requirements become more important. Note the numbers programmed in this register are based on the period of ref\_clk. Example: An ATMEL device needs 50ns minimum time before CS can be reasserted

after it has been deasserted. By default, the controller will only provide a minimum of 1 SCLK period. When the device is operating at 100MHz, the SCLK period is only 10ns, so 40ns extra is required. Since the register defines the number of ref\_clk cycles to add, and ref\_clk is running at 400MHz (2.5ns period), then the user should program a value of at least 16 to the d\_nss field of this register. This delay can be extended during auto-polling phase. There is possibility to define the polling repetition delay in the Write Completion Register.

7. Update Remap address register, if required. Affects DAC path only. Refer to section for further details.
8. Setup and enable write protection registers, if they are required and if they have not already been setup from post initialization.
9. Enable required interrupts via the interrupt mask register.
10. Setup the baud rate divisor in the QSPI Configuration register to define the required clock frequency of the target device.
11. Update read data capture register. This register will delay when the read data is captured and can help when the read data path from the device to the controller is long and the device clock frequency is high. An update to this register may not be necessary.
12. Enable the QSPI Controller and the DAC via the QSPI Configuration register.

### Using the Flash Command Control Register (STIG Operation)

The Flash Command Control register provides software a means to access the FLASH device in a flexible and programmable manner. This is known as a STIG operation (Software Triggered Instruction Generator). The instruction opcode, number of address bytes (if any), the address itself, number of dummy bytes (if any), number of write data bytes (if any), the write data itself and the number of read data bytes (if any) can be programmed. Once these have been programmed, software can trigger the command via bit 0 and wait for its acceptance by polling bit 1. When this bit turns de-asserted, another STIG can be triggered. This method of accessing the FLASH is the typical mechanism that software would use to access the FLASH device's registers, as well as for performing ERASE operations. It can also be used to access the FLASH array itself, although the maximum of 8 data bytes may be read or written at any one time, defined in the Flash Command Write and Read Data registers (0xA0, 0xA4, 0xA8 and 0xAC).

Commands issued using this interface have a higher priority than all other READ accesses coming from AHB, and will therefore interrupt any READ commands being requested by the indirect or direct controllers.

### Using SPI Legacy Mode

SPI legacy mode allows software to access the internal TX-FIFO and RX-FIFO directly, thus bypassing the direct, indirect and STIG controllers.

Legacy mode allows the user to issue any FLASH instruction to the device, but does place a heavy software overhead in order to manage the fill levels of the FIFO's effectively. This is because the legacy SPI core is bi-directional in nature, with data continuously being transferred in either direction while the chip select is enabled. Even if the driver only wishes to read data from the FLASH device, dummy data must be written out to ensure the chip select stays active, and vice versa for write transactions. This means that to perform a basic READ of 4 bytes to a device that has 3 address bytes, software would have to write a total of 8 bytes to the TX FIFO. The first byte would be the instruction opcode, the next 3 bytes are the address, and the final 4 bytes would be dummy data to ensure the chip select stays active while the read data is returned. Similarly, since 8 bytes were written to the TX-FIFO, software should expect 8 bytes to be returned in the RX-FIFO. The first 4 bytes of this would be discarded, leaving the final 4 bytes holding the required READ data.

Since the TX-FIFO and RX-FIFO are of limited depth, software has a responsibility to maintain the FIFO levels to ensure the TX-FIFO does not become exhausted during the instruction execution and the RX-FIFO doesn't overflow. This can place a lot of overhead on software. Interrupts are provided to indicate when the fill levels pass programmable watermarks, which are themselves programmable registers located at addresses 0x30 and 0x34.

Software accesses the TX-FIFO by writing any value to any address via the AHB interface to the QSPI controller while legacy mode is enabled. Software accesses the RX-FIFO by

reading any address via the AHB interface to the QSPI controller while legacy mode is enabled.

## Entering and Exiting XIP mode

### Entering XIP mode from POR

XIP is a mode that can be entered in a non-volatile way if the device has XIP enabled as a non-volatile configuration setting. Only a small subset of FLASH devices supports this feature. Software will not be able to discover the state of XIP from POR via FLASH status register reads as the only operation a FLASH device will recognize when XIP mode is enabled is an XIP read operation.

If it is already known that the device will enter XIP from POR, then the Mode Bit configuration register and bit 18 of the QSPI configuration register should be set in initial boot.

If it is not already known that the device will enter XIP from POR, and XIP from POR may be supported by the attached FLASH device, then software can attempt to exit XIP mode by issuing an XIP exit command using a STIG command (via the Flash Command Control Register). To do this, software must be aware of the mode bit requirements of that device, as XIP entry and exit changes per device. Devices that support XIP from POR from the list of tested devices are Micron N25Q and MT25 devices. For these devices, exiting XIP mode can be performed by configuring the opcode to 8'h00, the number of address bytes to 3, the number of dummy cycles to 16 and the number of read bytes to 1. The address that should be programmed should be {8 mode bits for exiting XIP mode, 16'h0000}. The mode bits for the Micron device must be 8'b10000000.

Note the XIP from POR mode can be enabled by issuing a NVCR write command via the Flash Command Control register. As this does not take effect until the next POR sequence, the effect of this will be picked up during the next enumeration sequence.

### Entering XIP mode otherwise

XIP mode is supported in most FLASH devices. However, FLASH manufacturers do not have a consistent standard approach for achieving this. Most use signature bits that are sent to the device immediately following the address bytes. Some, like the Micron devices, use signature bits and also require a FLASH device configuration register write to enable XIP. For the FLASH devices that must be compliant to this controller, the following steps can be taken by software to enter XIP mode.

#### ***Micron N25Q and MT25 Devices (No support for Basic-XIP)***

XIP mode must first be enabled by setting the VCR[3] within the FLASH device. The VCR can be written to using the Flash Command Control Register to issue a VCR write command. The steps to follow are described below:

Disable the Direct Access Controller and Indirect Access Controller to ensure no new AHB read accesses will be sent to the FLASH device

Configure the Flash Command Control Register to issue a VCR write to FLASH memory

Set the XIP mode bits in the Mode Bit configuration register (0x28) to 8'b00000000

Enable the local controllers XIP mode by setting bit 17 of the QSPI Configuration register

Re-enable the Direct Access Controller and, if required, the Indirect Access Controller

#### ***Other Micron Devices (Supporting Basic-XIP)***

Disable the Direct Access Controller and Indirect Access Controller to ensure no new AHB read accesses will be sent to the FLASH device

Set the XIP mode bits in the Mode Bit configuration register (0x28) to 8'b10000000

Enable the local controllers XIP mode by setting bit 17 of the QSPI Configuration register

Re-enable the Direct Access Controller and, if required, the Indirect Access Controller

**Winbond Devices**

Disable the Direct Access Controller and Indirect Access Controller to ensure no new AHB read accesses will be sent to the FLASH device

Set the XIP mode bits in the Mode Bit configuration register (0x28) to 8'b00100000

Enable the local controllers XIP mode by setting bit 17 of the QSPI Configuration register

Re-enable the Direct Access Controller and, if required, the Indirect Access Controller

**Spansion Devices**

Disable the Direct Access Controller and Indirect Access Controller to ensure no new AHB read accesses will be sent to the FLASH device

Set the XIP mode bits in the Mode Bit configuration register (0x28) to 8'b10100000

Enable the local controllers XIP mode by setting bit 17 of the QSPI Configuration register

Re-enable the Direct Access Controller and, if required, the Indirect Access Controller.

**Exiting XIP mode**

To exit XIP mode, software should first disable the Direct Access Controller and Indirect Access Controller to ensure no new AHB read accesses will be sent to the FLASH device. It should then set the mode bits to anything other than the mode bits specified in the specification of corresponding Flash Device. These are dependent on the FLASH device and manufacturer. Software should then reset bit 17 of the QSPI Configuration register.

Note the FLASH device must see a READ instruction before it can disable its internal XIP mode state, so this means XIP mode will internally stay active until the next READ instruction is serviced. Care must be taken to ensure that XIP mode is disabled before the end of any READ sequence.

**Using Indirect Data Transfer Mode**

The software sequence that should be followed when using the Indirect read controller, including how the DMA peripheral interface is setup is specified earlier in this document (Indirect Write Controller and Indirect Read Controller sections).

**Remapping AHB addresses**

The remap feature of the QSPI allows software to define how incoming AHB addresses translate to FLASH addresses when the access passes through the direct access controller. By default there is a 1:1 mapping. When bit 16 of the QSPI configuration register is enabled, the incoming AHB address is remapped to address+N, where N is the value stored in the Remap Address register (0x24). It is recommended that the QSPI is disabled before configuring the remap registers.

**Servicing Interrupts**

The QSPI provides a single active high level sensitive interrupt pin. Refer to the Interrupt Status register (0x40) for details of all the interrupt sources provided.

The interrupt source can be cleared when software reads the interrupt status register. The QSPI controller can also be configured so that interrupts are cleared on a write to the interrupt status register (providing a means to clear individual interrupt bits). This is setup in the `cdnsqspi_flash_ctrl_defs_default.v` verilog configuration file. Write-to-clear approach was used by default.

All interrupts can be masked using the Interrupt Mask registers. The mask register is set by writing a '0' to the Interrupt Mask register (0x44), cleared by writing a '1'.

**Using the AHB Protection Registers**

On POR, the AHB protection mechanism wakes in a disabled state. Software can use the protection registers to block AHB writes to certain registers. Refer to section "Write Protection" for details. It is recommended that the QSPI is disabled before configuring the protection registers.

## Programming Interface

### Registers

#### QSPI Configuration Register

| Offset | Bit   | R/W | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | Reset |
|--------|-------|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| 0x00   | 31    | RO  | <p>Serial Interface and QSPI pipeline is IDLE. This is a STATUS read-only bit. Note this is a retimed signal, so there will be some inherent delay on the generation of this status signal.</p> <p>NOTE:</p> <p>It is recommended to wait at least 4 cycles of ref_clk between getting IDLE bit asserted and switching any configuration field. This latency ensures that all low level synchronization is done, FIFOs are empty and the controller can be safely introduced into another mode of operation</p> | 0x1   |
| 0x00   | 30:26 | RO  | Unused bits. Read as zero.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | 0x0   |
| 0x00   | 25    | R/W | <p>Pipeline PHY Mode enable</p> <p>This bit is relevant only for configuration with PHY Module.</p>                                                                                                                                                                                                                                                                                                                                                                                                             | 0x0   |
| 0x00   | 24    | R/W | <p>Enable DTR Protocol</p> <p>This bit should be set if device is configured to work in DTR protocol.</p> <p>Note:</p> <p>DTR protocol provides all commands in DTR Mode. There are DTR Read commands which can be handled in STR Protocol (but in DTR Mode). This bit should be equal '0' to executing them. DTR commands in STR protocol are controlled by 10<sup>th</sup> bit of Device Read Configuration Register.</p>                                                                                     | 0x0   |
| 0x00   | 23    | R/W | <p>Enable AHB Decoder</p> <p>(Direct Access Mode Only)</p> <p>When set to '1', bits [13:10] are not relevant. Active slave is based on actual AHB address. The partition for each device is calculated with respect to bits [28:21] of Device Size Configuration Register.</p>                                                                                                                                                                                                                                  | 0x0   |
| 0x00   | 22:19 | R/W | <p>Master mode baud rate divisor (2 to 32),<br/>SPI baud rate = (master reference clock) /BD<br/>Where BD is:</p>                                                                                                                                                                                                                                                                                                                                                                                               | 0xf   |

|      |    |     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |     |
|------|----|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
|      |    |     | <p>0000 = /2<br/> 0001 = /4<br/> 0010 = /6<br/> 0011 = /8<br/> 0100 = /10<br/> 0101 = /12<br/> 0110 = /14<br/> 0111 = /16<br/> 1000 = /18<br/> ...<br/> 1111 = /32<br/> Set this register up before enabling the QSPI controller.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |     |
| 0x00 | 18 | R/W | <p>Enter XIP Mode immediately</p> <p>0 = If XIP is enabled, then setting to '0' will cause the controller to exit XIP mode after the next READ instruction has completed.</p> <p>1 = Operate the device in XIP mode immediately</p> <p>Use this register when the external device wakes up in XIP mode (as per the contents of its non-volatile configuration register). The controller will assume the next READ instruction will be passed to the device as an XIP instruction, and therefore will not require the READ opcode to be transferred.</p> <p>Note: To exit XIP mode, this bit should be set to 0 This will take effect in the attached device only AFTER the next READ instruction is executed. Software should therefore ensure that at least one READ instruction is requested after resetting this bit before it can be sure XIP mode in the device is exited.</p> <p>This bit is synchronized in hardware and can be set/unset while the controller is operational.</p> | 0x0 |
| 0x00 | 17 | R/W | <p>Enter XIP Mode on next READ</p> <p>0 = If XIP is enabled, then setting to '0' will cause the controller to exit XIP mode after the next READ instruction has completed.</p> <p>1 = If XIP is disabled, then setting to '1' will inform the controller that the device is ready to enter XIP on the next READ instruction. The controller will therefore send the appropriate command sequence, including mode bits to cause the device to enter XIP mode.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | 0x0 |

|         |                               |     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |         |               |      |      |      |      |      |      |      |      |      |                               |     |
|---------|-------------------------------|-----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|---------------|------|------|------|------|------|------|------|------|------|-------------------------------|-----|
|         |                               |     | <p>Use this register after the controller has ensured the attached FLASH device has been configured to be ready to enter XIP mode.</p> <p>Note: To exit XIP mode, this bit should be set to 0 This will take effect in the attached device only AFTER the next READ instruction is executed. Software should therefore ensure that at least one READ instruction is requested after resetting this bit before it can be sure XIP mode in the device is exited.</p> <p>This bit is synchronized in hardware and can be set/unset while the controller is operational.</p> |         |               |      |      |      |      |      |      |      |      |      |                               |     |
| 0x00    | 16                            | R/W | <p>Enable AHB Address Re-mapping</p> <p>(Direct Access Mode Only)</p> <p>When set to '1', the incoming AHB address will be adapted and sent to the FLASH device as (address + N), where N is the value stored in the remap address register.</p> <p>This bit is synchronized in hardware and can be set/unset while the controller is operational.</p>                                                                                                                                                                                                                   | 0x0     |               |      |      |      |      |      |      |      |      |      |                               |     |
| 0x00    | 15                            | R/W | <p>Enable DMA Peripheral Interface</p> <p>Set to '1' to enable the DMA handshaking logic. When enabled the QSPI will trigger DMA transfer requests via the DMA peripheral interface.</p> <p>Set to '0' to disable</p> <p>This bit is synchronized in hardware and can be set/unset while the controller is operational.</p>                                                                                                                                                                                                                                              | 0x0     |               |      |      |      |      |      |      |      |      |      |                               |     |
| 0x00    | 14                            | R/W | <p>Set to drive the Write Protect pin of the FLASH device. This is resynchronized to the generated memory clock as necessary. Note that the WP pin is only valid in SINGLE or DUAL transfer modes. During QUAD transfers, the WP pin is used for transferring data and therefore any setting of this register bit will be ignored.</p> <p>This bit is synchronized in hardware and can be set/unset while the controller is operational.</p>                                                                                                                             | 0x0     |               |      |      |      |      |      |      |      |      |      |                               |     |
| 0x00    | 13:10                         | R/W | <p>Peripheral chip select lines If bit 9 of this register = 0, ss[3:0] are output thus:</p> <table><tr><td>ss[3:0]</td><td>n_ss_out[3:0]</td></tr><tr><td>xxx0</td><td>1110</td></tr><tr><td>xx01</td><td>1101</td></tr><tr><td>x011</td><td>1011</td></tr><tr><td>0111</td><td>0111</td></tr><tr><td>1111</td><td>1111 (no peripheral selected)</td></tr></table> <p>else ss[3:0] directly drives n_ss_out[3:0]</p>                                                                                                                                                     | ss[3:0] | n_ss_out[3:0] | xxx0 | 1110 | xx01 | 1101 | x011 | 1011 | 0111 | 0111 | 1111 | 1111 (no peripheral selected) | 0x0 |
| ss[3:0] | n_ss_out[3:0]                 |     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |         |               |      |      |      |      |      |      |      |      |      |                               |     |
| xxx0    | 1110                          |     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |         |               |      |      |      |      |      |      |      |      |      |                               |     |
| xx01    | 1101                          |     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |         |               |      |      |      |      |      |      |      |      |      |                               |     |
| x011    | 1011                          |     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |         |               |      |      |      |      |      |      |      |      |      |                               |     |
| 0111    | 0111                          |     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |         |               |      |      |      |      |      |      |      |      |      |                               |     |
| 1111    | 1111 (no peripheral selected) |     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |         |               |      |      |      |      |      |      |      |      |      |                               |     |
| 0x00    | 9                             | R/W | <p>Peripheral select decode</p> <p>0 = only 1 of 4 selects n_ss_out[3:0] is active</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | 0x0     |               |      |      |      |      |      |      |      |      |      |                               |     |

|      |     |     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |     |
|------|-----|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
|      |     |     | 1 = allow external 4-to-16 decode<br>(n_ss_out = ss)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |     |
| 0x00 | 8   | R/W | Legacy IP Mode Enable<br>0 = Use Direct Access Controller/ Indirect Access Controller or STIG interface for data-transfers<br>1 = When set to '1', legacy mode is enabled. In this mode, any write to the controller via the AHB interface is serialized and sent to the FLASH device. Any valid AHB read will pop the internal RX-FIFO, retrieving data that was forwarded by the external FLASH device on the SPI lines, 4, 2 or 1 byte transfers are permitted and controlled via the HSIZE input. This bit is synchronized in hardware and can be set/unset while the controller is operational. | 0x0 |
| 0x00 | 7   | R/W | Enable Direct Access Controller<br>0 = disable the Direct Access Controller once current transfer of the data word (FF_W) is complete.<br>1 = enable the Direct Access Controller<br><br>When the Direct Access Controller and Indirect Access Controller are both disabled, all AHB requested are completed with an error response. This bit is synchronized in hardware and can be set/unset while the controller is operational.                                                                                                                                                                  | 0x1 |
| 0x00 | 6:4 | RO  | Unused bits. Read as zero.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | 0x0 |
| 0x00 | 3   | R/W | PHY Mode enable<br><br>This bit is relevant only for configuration with PHY Module.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | 0x0 |
| 0x00 | 2   | R/W | Clock phase – This maps to the standard SPI CPHA transfer format.<br><br>NOTE:<br>Keep this bit low when operating in DDR Mode or DDR Protocol to ensure the falling edge of SPI clock as the last one.                                                                                                                                                                                                                                                                                                                                                                                              | 0x0 |
| 0x00 | 1   | R/W | Clock polarity outside SPI word. This maps to the standard SPI CPOL transfer format.<br><br>NOTE:<br>Keep this bit low when operating in DDR Mode or DDR Protocol to ensure the falling edge of SPI clock as the last one.                                                                                                                                                                                                                                                                                                                                                                           | 0x0 |
| 0x00 | 0   | R/W | QSPI Enable<br>0 = disable the QSPI once current transfer of the data word (FF_W) is complete.<br>1 = enable the QSPI                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | 0x1 |

|  |  |  |                                                                                                                                              |  |
|--|--|--|----------------------------------------------------------------------------------------------------------------------------------------------|--|
|  |  |  | When <code>spi_enable = 0</code> , all output enables are inactive and all pins are set to input mode. This bit is synchronized in hardware. |  |
|--|--|--|----------------------------------------------------------------------------------------------------------------------------------------------|--|

**Device Read Instruction Register**

This register should be setup while the controller is idle.

| Offset | Bit   | R/W | Description                                                                                                                                                                                                                                                                                                                                                                                                                                    | Reset |
|--------|-------|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| 0x04   | 31:29 | RO  | Unused bit. Read as zero.                                                                                                                                                                                                                                                                                                                                                                                                                      | 0x0   |
| 0x04   | 28:24 | R/W | Number of Dummy Clock Cycles required by device for Read Instruction.                                                                                                                                                                                                                                                                                                                                                                          | 0x0   |
| 0x04   | 23:21 | RO  | Unused bit. Read as zero.                                                                                                                                                                                                                                                                                                                                                                                                                      | 0x0   |
| 0x04   | 20    | R/W | Mode Bit Enable<br>Set to '1' to ensure the mode bits as defined in the Mode Bit Configuration register are sent following the address bytes.                                                                                                                                                                                                                                                                                                  | 0x0   |
| 0x04   | 19:18 | RO  | Unused bit. Read as zero.                                                                                                                                                                                                                                                                                                                                                                                                                      | 0x0   |
| 0x04   | 17:16 | R/W | Data Transfer Type for Standard SPI modes (as defined by the instruction type in bits [9:8])<br><br>0 : SIO mode – data is shifted to the device on DQ0 only and from the device on DQ1 only<br>1: Used for Dual Input/Output instructions. For data transfers, DQ0 and DQ1 are used as both inputs and outputs.<br>2: Used for Quad Input/Output instructions. For data transfers, DQ0, DQ1, DQ2 and DQ3 are used as both inputs and outputs. | 0x0   |
| 0x04   | 15:14 | RO  | Unused bit. Read as zero.                                                                                                                                                                                                                                                                                                                                                                                                                      | 0x0   |
| 0x04   | 13:12 | R/W | Address Transfer Type for Standard SPI modes (as defined by the instruction type in bits [9:8])<br><br>0 : Addresses can be shifted to the device on DQ0 only<br>1 : Addresses can be shifted to the device on DQ0 and DQ1 only<br>2 : Addresses can be shifted to the device on DQ0, DQ1, DQ2 and DQ3                                                                                                                                         | 0x0   |
| 0x04   | 11    | RO  | Unused bit. Read as zero.                                                                                                                                                                                                                                                                                                                                                                                                                      | 0x0   |
| 0x04   | 10    | R/W | DDR Bit Enable<br>Set to '1' when opcode from bits 7 to 0 is compliant with DDR command.                                                                                                                                                                                                                                                                                                                                                       | 0x0   |
| 0x04   | 9:8   | R/W | Instruction Type<br><br>0 : Use Standard SPI mode (instruction always shifted into the device on DQ0 only)<br>1 : Use DIO-SPI mode (Instructions, Address and Data always sent on DQ0 and DQ1)<br>2 : Use QIO-SPI mode ((Instructions, Address and Data always sent on DQ0,DQ1,DQ2 and DQ3)                                                                                                                                                    | 0x0   |

|      |     |     |                                                                                                                                                                                      |      |
|------|-----|-----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|
|      |     |     | NOTE: These bits are relevant not just to READ transfers. They are global settings and will affect READ's, WRITES via the DAC and INDAC as well as any transfer sent using the STIG. |      |
| 0x04 | 7:0 | R/W | Read Opcode to use when not in XIP mode                                                                                                                                              | 0x03 |

**Device Write Instruction Configuration Register**

This register should be setup while the controller is idle.

| Offset | Bit   | R/W | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                | Reset |
|--------|-------|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| 0x08   | 31:29 | RO  | Unused bit. Read as zero.                                                                                                                                                                                                                                                                                                                                                                                                                                  | 0x0   |
| 0x08   | 28:24 | R/W | Number of Dummy Clock Cycles required by device for Write Instruction                                                                                                                                                                                                                                                                                                                                                                                      | 0x0   |
| 0x08   | 23:18 | RO  | Unused bit. Read as zero.                                                                                                                                                                                                                                                                                                                                                                                                                                  | 0x00  |
| 0x08   | 17:16 | R/W | Data Transfer Type for Standard SPI modes (as defined by the instruction type in bits [9:8] of reg 0x04)<br><br>0 : SIO mode – data is shifted to the device on DQ0 only and from the device on DQ1 only<br>1: Used for Dual Input/Output instructions. For data transfers, DQ0 and DQ1 are used as both inputs and outputs.<br>2: Used for Quad Input/Output instructions. For data transfers, DQ0, DQ1, DQ2 and DQ3 are used as both inputs and outputs. | 0x0   |
| 0x08   | 15:14 | RO  | Unused bit. Read as zero.                                                                                                                                                                                                                                                                                                                                                                                                                                  | 0x0   |
| 0x08   | 13:12 | R/W | Address Transfer Type for Standard SPI modes (as defined by the instruction type in bits [9:8] of reg 0x04)<br><br>0 : Addresses can be shifted to the device on DQ0 only<br>1 : Addresses can be shifted to the device on DQ0 and DQ1 only<br>2 : Addresses can be shifted to the device on DQ0, DQ1, DQ2 and DQ3                                                                                                                                         | 0x0   |
| 0x08   | 11:9  | RO  | Unused bit. Read as zero.                                                                                                                                                                                                                                                                                                                                                                                                                                  | 0x0   |
| 0x08   | 8     | R/W | WEL Disable<br>This bit allow to turn off automatic issuing of WEL Command before write operation for DAC or INDAC.                                                                                                                                                                                                                                                                                                                                        | 0x0   |
| 0x08   | 7:0   | R/W | Write Opcode                                                                                                                                                                                                                                                                                                                                                                                                                                               | 0x02  |

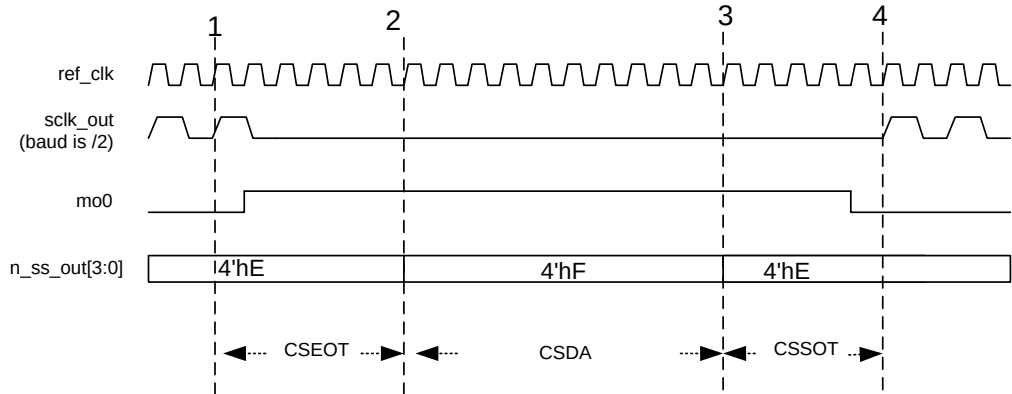
**QSPI Device Delay Register**

This register is used to introduce relative delays into the generation of the master output signals. This allows the timings to be adjusted to meet the requirements of a specific flash device. All timings are defined in cycles of the SPI master ref clock. This register should be setup while the controller is idle.

For an illustration of the register fields, refer to the two diagrams below this table.

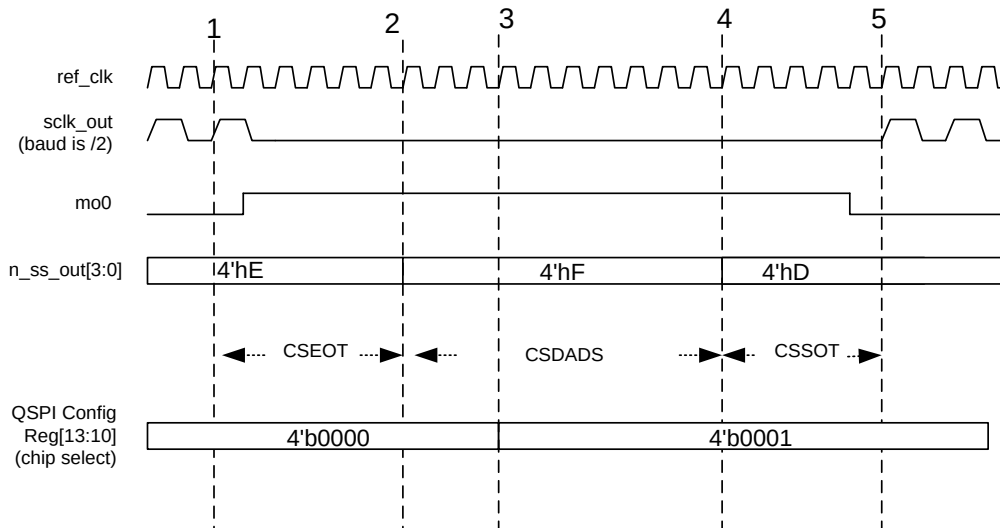
| Offset | Bit   | R/W | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | Reset    |
|--------|-------|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| 0x0C   | 31:24 | R/W | <p><b>CSDA – Chip Select De-Assert</b><br/>           Added delay in master reference clocks (ref_clk) for the length that the master mode chip select outputs are de-asserted between transactions.<br/>           The minimum delay for chip select to be de-asserted (CSDA=0) is:<br/> <math>1 \text{ sclk\_out} + 1 \text{ ref\_clk}</math><br/>           to ensure the chip select is never re-asserted within an sclk_out period.<br/>           If CSDA=X, then the chip select de-assert time will be:<br/> <math>1 \text{ sclk\_out} + 1 \text{ ref\_clk} + X \text{ ref\_clks}</math>.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | 0x000000 |
| 0x0C   | 23:16 | R/W | <p><b>CSDADS – Chip Select De-Assert Different Slaves</b><br/>           Delay in master reference clocks (ref_clk) between one chip select being de-activated and the activation of another. This is used to ensure a quiet period between the selection of two different slaves. CSDADS is only relevant when switching between 2 different external flash devices.</p> <p>An example scenario might be if you have 2 FLASH devices named A and B, each connected to the controller on a different chip select. You might have a requirement to have chip select inactive for a particular period of time between each successive non-sequential transfer of the same device. You would control this using bits CSDA. There may be a different requirement to hold chip select inactive for a longer period of time between the end of one transfer on device A and the start of the next transfer on device B. You would control this using CSDADS.</p> <p>The minimum delay (CSDADS=0) is:<br/> <math>1 \text{ sclk\_out} + 3 \text{ ref\_clks}</math>.<br/>           If CSDADS=X, then the delay will be:<br/> <math>1 \text{ sclk\_out} + 3 \text{ ref\_clks} + X \text{ ref\_clks}</math></p> | 0x000000 |
| 0x0C   | 15:8  | R/W | <p><b>CSEOT – Chip Select End Of Transfer</b><br/>           Delay in master reference clocks between last bit of current transaction and de-asserting the device chip select (n_ss_out).<br/>           By default (when CSEOT=0), the chip select will be de-asserted on the last falling edge of sclk_out at the completion of the current transaction.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | 0x000000 |

|      |     |     |                                                                                                                                                                                                                                                                                                                                                                                    |          |
|------|-----|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
|      |     |     | If CSEOT=X, then chip selected will de-assert X ref_clks after the last falling edge of sclk_out.                                                                                                                                                                                                                                                                                  |          |
| 0x0C | 7:0 | R/W | <b>CSSOT – Chip Select Start Of Transfer</b><br>Delay in master reference clocks between setting n_ss_out low and first bit transfer.<br>By default (CSSOT=0), chip select will be asserted half a SCLK period before the first rising edge of sclk_out.<br>If CSSOT=X, chip select will be asserted half a sclk_out period before the first rising edge of sclk_out + X ref_clks. | 0x000000 |



1. Last active sclk\_out cycle in SPI transaction. This does not mark the end of the overall SPI transaction.
2. Chip select de-asserted, SPI transaction is now complete.
3. Chip select asserted, first step of a new SPI transaction.
4. This is the first rising edge of sclk\_out for the new transaction.

**Diagram showing chip select behaviour between 2 consecutive transfers to the same slave**



1. Last active sclk\_out cycle in SPI transaction. This does not mark the end of the overall SPI transaction.
2. Chip select slave 0 de-asserted, SPI transaction is now complete.
3. While controller is idle, APB write to QSPI config register, selects slave 1 and de-selects slave 0.
4. Chip select for slave 1 asserted, first step of a new SPI transaction.
5. This is the first rising edge of sclk\_out for the new transaction.

**Diagram showing chip select behaviour between 2 consecutive transfers to different slaves**

### Read Data Capture Register

This register should be setup while the controller is idle.

| Offset | Bit   | R/W | Description                                                                                                                                                                                                                                                                                                                                                              | Reset |
|--------|-------|-----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| 0x10   | 31:20 | RO  | Unused bits. Read as zero.                                                                                                                                                                                                                                                                                                                                               | 0x0   |
| 0x10   | 19:16 | R/W | <p>Delay the transmitted data by the programmed number of ref_clk cycles (in order to improve the hold timing during transfers in DDR Mode).</p> <p>This field is only relevant when DDR Read Command is executed. Otherwise can be ignored.</p> <p><math>\text{delay\_value} = 1 \times \text{ref\_clk} + \text{&lt;field\_value&gt;} \times \text{ref\_clk}</math></p> | 0x0   |
| 0x10   | 15:6  | RO  | Unused bits. Read as zero.                                                                                                                                                                                                                                                                                                                                               | 0x0   |
| 0x10   | 5     | R/W | <p>Sample edge selection (of the flash memory data outputs).</p> <p>0 = data outputs from flash memory are sampled on falling edge of the ref_clk<br/> 1 = data outputs from flash memory are sampled on rising edge of the ref_clk</p>                                                                                                                                  | 0x0   |
| 0x10   | 4:1   | R/W | Delay the read data capturing logic by the programmed number of ref_clk cycles                                                                                                                                                                                                                                                                                           | 0x0   |

|      |   |     |                                                                                                                                                                                                                                           |     |
|------|---|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| 0x10 | 0 | R/W | <p>Bypass of the adapted loopback clock circuit<br/>0 = enable the adapted loopback clock circuit<br/>1 = disable the adapted loopback clock circuit</p> <p>For detailed information please refer to the Read Data Capturing section.</p> | 0x1 |
|------|---|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|

**Device Size Configuration Register**

This register should be setup while the controller is idle.

| Offset | Bit   | R/W | Description                                                                                                                                                                                                                                                   | Reset |
|--------|-------|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| 0x14   | 31:29 | RO  | Unused bits. Read as zero.                                                                                                                                                                                                                                    | 0x00  |
| 0x14   | 28:27 | R/W | Size of Flash Device connected to CS[3] pin.<br>This field is only valid when AHB Decoder is enabled.<br><br>00 = size of 512Mb<br>01 = size of 1Gb<br>10 = size of 2Gb<br>11 = size of 4Gb                                                                   | 0x00  |
| 0x14   | 26:25 | R/W | Size of Flash Device connected to CS[2] pin.<br>This field is only valid when AHB Decoder is enabled.<br><br>00 = size of 512Mb<br>01 = size of 1Gb<br>10 = size of 2Gb<br>11 = size of 4Gb                                                                   | 0x00  |
| 0x14   | 24:23 | R/W | Size of Flash Device connected to CS[1] pin.<br>This field is only valid when AHB Decoder is enabled.<br><br>00 = size of 512Mb<br>01 = size of 1Gb<br>10 = size of 2Gb<br>11 = size of 4Gb                                                                   | 0x00  |
| 0x14   | 22:21 | R/W | Size of Flash Device connected to CS[0] pin.<br>This field is only valid when AHB Decoder is enabled.<br><br>00 = size of 512Mb<br>01 = size of 1Gb<br>10 = size of 2Gb<br>11 = size of 4Gb                                                                   | 0x00  |
| 0x14   | 20:16 | R/W | Number of bytes per block.<br>This is required by the controller for performing the write protection logic. The number of bytes per block must be a power of 2 number.<br>i.e. 0 = 1byte<br>1 = 2 bytes<br>2 = 4 bytes<br>3 = 8 bytes<br>16 = 65535 bytes etc | 0x10  |
| 0x14   | 15:4  | R/W | Number of bytes per device page.<br>This is required by the controller for performing FLASH writes up to and across page boundaries.                                                                                                                          | 0x100 |
| 0x14   | 3:0   | R/W | Number of address bytes.<br>A value of 0 = 1 byte                                                                                                                                                                                                             | 0x2   |

**SRAM Partition Configuration Register**

X in this register is the SRAM depth, configured in the verilog defines configuration file. This register should be setup while the controller is idle.

| Offset | Bit   | R/W | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | Reset                                                                       |
|--------|-------|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| 0x18   | 31:X  | RO  | Unused bits. Read as zero.                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | 0x0                                                                         |
| 0x18   | X-1:0 | R/W | <p>Defines the size of the indirect read partition in the SRAM, in units of SRAM locations. By default, half of the SRAM is reserved for indirect read operation, and half for indirect write. The size of this register will scale with the depth of the SRAM. The number of locations allocated to indirect read = SRAM_PARTITION_REG+1. The number of locations allocated to indirect write = (2**X)-SRAM_PARTITION_REG.</p> <p>Avoid setting SRAM_PARTITION_REG to 0x0000 or (2**X)-1)</p> | <p>{1'b1,{X-1{1'b0}}};</p> <p>Top bit is set, all other bits are zeroed</p> |

**Indirect AHB Address Trigger register**

This register should be setup while the controller is idle.

| Offset | Bit  | R/W | Description                                                                                                                                                                                                                                                                                                                                              | Reset      |
|--------|------|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| 0x1C   | 31:0 | R/W | <p>Indirect Trigger Address</p> <p>This is the base address that will be used by the AHB controller. When the incoming AHB read access address matches a range of addresses from this trigger address to the trigger address + &lt;configured range&gt;, then the AHB request will be completed by fetching data from the Indirect Controllers SRAM.</p> | 0x00000000 |

**DMA Peripheral Configuration Register**

This register should be setup while the controller is idle.

| Offset | Bit   | R/W | Description                                                                                                                                                                                                                                                                                            | Reset  |
|--------|-------|-----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| 0x20   | 31:12 | RO  | Unused bits. Read as zero.                                                                                                                                                                                                                                                                             | 0x0000 |
| 0x20   | 11:8  | R/W | Number of bytes in a burst type request on the DMA peripheral request. A programmed value of 0 represents a single byte. This should be setup before starting the indirect read or write operation. The actual number of bytes used is 2**(value in this register) which will simplify implementation. | 0x00   |
| 0x20   | 7:4   | RO  | Unused bits. Read as zero.                                                                                                                                                                                                                                                                             | 0x0000 |
| 0x20   | 3:0   | R/W | Number of bytes in a single type request on the DMA peripheral request. A programmed value of 0 represents a single byte. This should be setup before starting the indirect read or write operation.                                                                                                   | 0x00   |

|  |  |  |                                                                                                    |  |
|--|--|--|----------------------------------------------------------------------------------------------------|--|
|  |  |  | The actual number of bytes used is 2**(value in this register) which will simplify implementation. |  |
|--|--|--|----------------------------------------------------------------------------------------------------|--|

### Remap Address Register

This register should be setup while the controller is idle.

| Offset | Bit  | R/W | Description                                                                                          | Reset      |
|--------|------|-----|------------------------------------------------------------------------------------------------------|------------|
| 0x24   | 31:0 | R/W | This register is used to remap incoming AHB address to a different address used by the FLASH device. | 0x00000000 |

### Mode Bit Configuration Register

This register should be setup while the controller is idle.

| Offset | Bit  | R/W | Description                                                                              | Reset |
|--------|------|-----|------------------------------------------------------------------------------------------|-------|
| 0x28   | 31:8 | RO  | Unused bits. Read as zero.                                                               | 0x0   |
| 0x28   | 7:0  | R/W | Mode bits. These are the 8 bits that are sent to the device following the address bytes. | 0x00  |

### SRAM Fill Level Register

| Offset | Bit   | R/W | Description                                                                  | Reset |
|--------|-------|-----|------------------------------------------------------------------------------|-------|
| 0x2C   | 31:16 | RO  | SRAM Fill Level (Indirect Write Partition) in units of SRAM Words (4 bytes). | 0x0   |
| 0x2C   | 15:0  | RO  | SRAM Fill Level (Indirect Read Partition) in units of SRAM Words (4 bytes).  | 0x0   |

### TX Threshold Register

X in this register is the TX FIFO address width, configured in the verilog defines configuration file. This register should be setup while the controller is idle.

| Offset | Bit   | R/W | Description                                                                                                                                                                      | Reset |
|--------|-------|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| 0x30   | 31:X  | RO  | Unused bits. Read as zero.                                                                                                                                                       | 0x0   |
| 0x30   | X-1:0 | RW  | Defines the level at which the small TX FIFO not full interrupt is generated. This is only relevant when accessing the FLASH device in legacy mode. It can be otherwise ignored. | 0x01  |

### RX Threshold Register

X in this register is the RX FIFO address width, configured in the verilog defines configuration file. This register should be setup while the controller is idle.

| Offset | Bit   | R/W | Description                                                                                          | Reset |
|--------|-------|-----|------------------------------------------------------------------------------------------------------|-------|
| 0x34   | 31:X  | RO  | Unused bits. Read as zero.                                                                           | 0x0   |
| 0x34   | X-1:0 | RW  | Defines the level at which the small RX FIFO not empty interrupt is generated. This is only relevant | 0x01  |

|  |  |  |                                                                                 |  |
|--|--|--|---------------------------------------------------------------------------------|--|
|  |  |  | when accessing the FLASH device in legacy mode.<br>It can be otherwise ignored. |  |
|--|--|--|---------------------------------------------------------------------------------|--|

## Write Completion Control Register

This register defines how the controller will poll the device following a write transfer. By default it is set to poll bit 0 of the device STATUS register (using opcode 0x05), which is common across all devices to indicate WRITE IN PROGRESS (or WIP). However, in some devices, notably the Micron devices that are  $\geq 512\text{MB}$  in size, it is required that the controller polls a different bit of a different device register. This means the controller must issue a different command during this polling phase. For example the Micron N25Q devices that are  $\geq 512\text{MB}$  in size, it is bit 7 of the FLAG STATUS register (opcode 0x70) instead of bit 0 of the normal STATUS register that must be polled. For this reason, the following register has been provided for software to determine the bit and the opcode to use to poll for write completion, and also the number of successive valid polls that should take place.

This register should be setup while the controller is idle.

| Offset | Bit   | R/W | Description                                                                                                                                                                                                                                                                           | Reset |
|--------|-------|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| 0x38   | 31:24 | RW  | Polling repetition delay<br>Defines additional delay for maintain Chip Select de-asserted during auto-polling phase.                                                                                                                                                                  | 0x0   |
| 0x38   | 23:16 | RW  | Poll_count<br>Defines the number of times the controller should expect to see a true result from the polling in successive reads of the device register. Adding additional cycles might be necessary when the controller is working on high throughput.                               | 0x01  |
| 0x38   | 15    | RO  | Unused bits. Read as zero.                                                                                                                                                                                                                                                            | 0x0   |
| 0x38   | 14    | RW  | Disable_polling<br>This switches off the automatic polling function                                                                                                                                                                                                                   | 0x0   |
| 0x38   | 13    | RW  | Polling_polarity<br>Defines the polling polarity.<br>If '1', then the write transfer to the device will be complete if the polled bit is equal to '1'.<br><br>If '0', then the write transfer to the device will be complete if the polled bit is equal to '0'.                       | 0x0   |
| 0x38   | 12:11 | RO  | Unused bits. Read as zero.                                                                                                                                                                                                                                                            | 0x0   |
| 0x38   | 10:8  | RW  | Polling_bit_index<br>Defines the bit index that should be polled.<br>A value of 010 means that bit 2 of the returned data will be polled for.<br>A value of 111 means that bit 7 of the returned data will be polled for.                                                             | 0x0   |
| 0x38   | 7:0   | RW  | Opcode<br>Defines the opcode that should be issued by the controller when it is automatically polling for device program completion. This command is issued followed all device write operations.<br>By default, this will poll the standard device STATUS register using opcode 0x05 | 0x05  |

## Polling Expiration Register

This register should be setup while the controller is idle.

| Offset | Bit  | R/W | Description                                                                                | Reset      |
|--------|------|-----|--------------------------------------------------------------------------------------------|------------|
| 0x3C   | 31:0 | RW  | Defines the numbers of polls cycles after which polling expiration interrupt is generated. | 0xffffffff |

## Interrupt Status Register

The status fields in this register are set when the described event occurs and the interrupt is enabled in the mask register. When any of these bit fields are set, the interrupt output is asserted high. The fields can be cleared using write-one-to-clear or read-clear approach. The choice is made statically in the verilog defines configuration file. It is assumed in the table below that a write-one-to-clear approach is taken. Note that bit fields 7 thru 11 are only valid when legacy SPI mode is active.

| Offset | Bit   | R/W | Description                                                                                                                                                                                                                                                                                                                                                     | Reset |
|--------|-------|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| 0x40   | 31:14 | RO  | Unused bits. Read as zero.                                                                                                                                                                                                                                                                                                                                      | 0x0   |
| 0x40   | 13    | RWC | The maximum number of programmed polls cycles is expired.                                                                                                                                                                                                                                                                                                       | 0x0   |
| 0x40   | 12    | RWC | Indirect Read partition of SRAM is full and unable to immediately complete indirect operation                                                                                                                                                                                                                                                                   | 0x0   |
| 0x40   | 11    | RWC | Small RX FIFO full (current FIFO status)<br>Can be ignored in non-SPI legacy mode<br><br>0 = FIFO is not full<br>1 = FIFO is full                                                                                                                                                                                                                               | 0x0   |
| 0x40   | 10    | RWC | Small RX FIFO not empty (current FIFO status)<br>Can be ignored in non-SPI legacy mode<br><br>0 = FIFO has < RX THRESHOLD entries<br>1 = FIFO has ≥ THRESHOLD entries                                                                                                                                                                                           | 0x0   |
| 0x40   | 9     | RWC | Small TX FIFO full (current FIFO status)<br>Can be ignored in non-SPI legacy mode<br><br>0 = FIFO is not full<br>1 = FIFO is full                                                                                                                                                                                                                               | 0x0   |
| 0x40   | 8     | RWC | Small TX FIFO not full (current FIFO status)<br>Can be ignored in non-SPI legacy mode<br><br>0 = FIFO has > THRESHOLD entries<br>1 = FIFO has ≤ THRESHOLD entries                                                                                                                                                                                               | 0x0   |
| 0x40   | 7     | RWC | Receive Overflow<br>This should only occur in Legacy SPI mode.<br><br>Set if an attempt is made to push the RX FIFO when it is full. This bit is reset only by a system reset and cleared only when this register is read. If a new push to the RX FIFO occurs coincident with a register read this flag will remain set.<br>0 = no overflow has been detected. | 0x0   |

|      |   |     |                                                                                                                                                                                                                                                                              |     |
|------|---|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
|      |   |     | 1 = an overflow has occurred.                                                                                                                                                                                                                                                |     |
| 0x40 | 6 | RWC | Indirect Transfer Watermark Level Breached                                                                                                                                                                                                                                   | 0x0 |
| 0x40 | 5 | RWC | Illegal AHB Access Detected. AHB write wrapping bursts and the use of SPLIT/RETRY accesses will cause this interrupt to trigger.<br>This interrupt is also triggered, if AHB access is performed when the DAC controller is disabled.                                        | 0x0 |
| 0x40 | 4 | RWC | Write to protected area was attempted and rejected.                                                                                                                                                                                                                          | 0x0 |
| 0x40 | 3 | RWC | Indirect operation was requested but could not be accepted. Two indirect operations already in storage.                                                                                                                                                                      | 0x0 |
| 0x40 | 2 | RWC | Controller has completed last triggered indirect operation                                                                                                                                                                                                                   | 0x0 |
| 0x40 | 1 | RWC | Underflow Detected<br>0 = no underflow has been detected<br>1 = underflow is detected and an attempt to transfer data is made when the small TX FIFO is empty. This may occur when AHB write data is being supplied too slowly to keep up with the requested write operation | 0x0 |
| 0x40 | 0 | RWC | Mode fail M<br>Legacy interrupt which is not being used any more.                                                                                                                                                                                                            | 0x0 |

### Interrupt Mask Register

This register should be setup while the controller is idle.

| Offset | Bit   | R/W | Description                                                                                                                                                             | Reset |
|--------|-------|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| 0x44   | 31:14 | RO  | Unused bits. Read as zero.                                                                                                                                              | 0x0   |
| 0x44   | 13:0  | R/W | 0 = the interrupt for the corresponding interrupt status register bit is disabled.<br>1 = the interrupt for the corresponding interrupt status register bit is enabled. | 0x000 |

**Lower Write Protection Register**

| Offset | Bit  | R/W | Description                                                                                                                                                                                                                                                                                                                             | Reset      |
|--------|------|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| 0x50   | 31:0 | R/W | The block number that defines the lower block in the range of blocks that is to be locked from writing.<br>The definition of a block in terms of number of bytes is programmable via the Device Size Configuration Register.<br>Modify this bit only when write protection feature (via bit 1 of the write protection register) is low. | 0x00000000 |

**Upper Write Protection Register**

| Offset | Bit  | R/W | Description                                                                                                                                                                                                                                                                                                                             | Reset      |
|--------|------|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| 0x54   | 31:0 | R/W | The block number that defines the upper block in the range of blocks that is to be locked from writing.<br>The definition of a block in terms of number of bytes is programmable via the Device Size Configuration Register.<br>Modify this bit only when write protection feature (via bit 1 of the write protection register) is low. | 0x00000000 |

**Write Protection Register**

| Offset | Bit  | R/W | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | Reset |
|--------|------|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| 0x58   | 31:2 | RO  | Unused bits. Read as zero.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | 0x0   |
| 0x58   | 1    | R/W | <p>Write Protection Enable Bit</p> <p>When set to '1', any AHB write access with an address within the protection region defined in the lower and upper write protection registers is rejected. An AHB error response is generated and an interrupt source triggered.</p> <p>When set to '0', the protection region is disabled.</p> <p>This bit is internally synchronized in hardware and does not have specific software requirements on when this bit is allowed to change state.</p>                        | 0x0   |
| 0x58   | 0    | R/W | <p>Write Protection Inversion Bit.</p> <p>When set to '1', the protection region defined in the lower and upper write protection registers is inverted meaning it is the region that the system is permitted to write to.</p> <p>When set to '0', the protection region defined in the lower and upper write protection registers is the region that the system is not permitted to write to.</p> <p>Modify this bit only when write protection feature (via bit 1 of the write protection register) is low.</p> | 0x0   |

**Indirect Read Transfer Control Register**

| Offset | Bit  | R/W | Description                                                                                                                                                                                                                                                                                          | Reset |
|--------|------|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| 0x60   | 31:8 | RO  | Unused bits. Read as zero.                                                                                                                                                                                                                                                                           | 0x0   |
| 0x60   | 7:6  | RO  | Number of indirect operations completed. This is used in conjunction with bit 5. incremented by H/W when an indirect operation has completed, decremented when bit 5 is written with a '1'                                                                                                           | 0x0   |
| 0x60   | 5    | R/W | Indirect Completion Status (status). Set by H/W when an indirect operation has completed, cleared by writing a '1'                                                                                                                                                                                   | 0x0   |
| 0x60   | 4    | RO  | Two indirect read operations have been queued (status)<br>This bit is set and cleared by H/W                                                                                                                                                                                                         | 0x0   |
| 0x60   | 3    | R/W | SRAM full and unable to immediately complete indirect operation (status)<br>Set by H/W, cleared by writing a '1'                                                                                                                                                                                     | 0x0   |
| 0x60   | 2    | RO  | Indirect read operation in progress (status). This bit can only be cleared by H/W                                                                                                                                                                                                                    | 0x0   |
| 0x60   | 1    | WO  | Cancel indirect read (control)<br>Writing a '1' to this bit will cancel an ongoing Indirect READ operation. This will cancel all indirect operations currently executing. This bit is internally synchronized in hardware.                                                                           | 0x0   |
| 0x60   | 0    | WO  | Start indirect read (control)<br>Writing a '1' to this bit will trigger an indirect READ operation. The assumption is that the indirect start address and Indirect number of bytes register is setup before triggering the indirect READ operation. This bit is internally synchronized in hardware. | 0x0   |

**Indirect Read Transfer Watermark Register**

This register should be setup before an indirect read transfer is triggered.

| Offset | Bit  | R/W | Description                                                                                                                                                                                                                                                                           | Reset      |
|--------|------|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| 0x64   | 31:0 | R/W | Watermark value<br>This represents the minimum fill level of the SRAM before a DMA peripheral access is permitted. When the SRAM fill level passes the watermark, an interrupt source is also generated. This can be disabled by writing a value of all zeroes.<br>In units of Bytes. | 0x00000000 |

**Indirect Read Transfer Start Address Register**

This register should be setup before an indirect read transfer is triggered.

| Offset | Bit  | R/W | Description                                                                                                                  | Reset      |
|--------|------|-----|------------------------------------------------------------------------------------------------------------------------------|------------|
| 0x68   | 31:0 | R/W | Start of Indirect Access<br>This is the FLASH start address from which the indirect access will commence its READ operation. | 0x00000000 |

**Indirect Read Transfer Number Bytes Register**

This register should be setup before an indirect read transfer is triggered.

| Offset | Bit  | R/W | Description                                                                                                                                         | Reset      |
|--------|------|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| 0x6C   | 31:0 | R/W | Indirect Number of Bytes<br>This is the number of bytes that the indirect access will consume. This can be bigger than the configured size of SRAM. | 0x00000000 |

**Indirect Write Transfer Control Register**

| Offset | Bit  | R/W | Description                                                                                                                                                                                                                                                                                                | Reset |
|--------|------|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| 0x70   | 31:8 | RO  | Unused bits. Read as zero.                                                                                                                                                                                                                                                                                 | 0x0   |
| 0x70   | 7:6  | RO  | Number of indirect operations completed.<br>This is used in conjunction with bit 5.<br>incremented by H/W when an indirect operation has completed, decremented when bit 5 is written with a '1'                                                                                                           | 0x0   |
| 0x70   | 5    | R/W | Indirect Completion Status (status).<br>Set by H/W when an indirect operation has completed, cleared by writing a '1'                                                                                                                                                                                      | 0x0   |
| 0x70   | 4    | RO  | Two indirect write operations have been queued (status)<br>This bit is set and cleared by H/W                                                                                                                                                                                                              | 0x0   |
| 0x70   | 3    | RO  | Unused                                                                                                                                                                                                                                                                                                     | 0x0   |
| 0x70   | 2    | RO  | Indirect write operation in progress (status). This bit can only be cleared by H/W                                                                                                                                                                                                                         | 0x0   |
| 0x70   | 1    | WO  | Cancel indirect write (control)<br>Writing a '1' to this bit will cancel an ongoing Indirect Write operation. This will cancel all indirect operations currently in progress.<br>This bit is internally synchronized in hardware.                                                                          | 0x0   |
| 0x70   | 0    | WO  | Start indirect write (control)<br>Writing a '1' to this bit will trigger an indirect WRITE operation. The assumption is that the indirect start address and Indirect number of bytes register is setup before triggering the indirect WRITE operation.<br>This bit is internally synchronized in hardware. | 0x0   |

**Indirect Write Transfer Watermark Register**

This register should be setup before an indirect write transfer is triggered.

| Offset | Bit  | R/W | Description                                                                                                                                                                                                                                                                       | Reset      |
|--------|------|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| 0x74   | 31:0 | R/W | Watermark value<br>This represents the maximum fill level of the SRAM before a DMA peripheral access is permitted. When the SRAM fill level falls below the watermark, an interrupt is also generated. This can be disabled by writing a value of all ones.<br>In units of Bytes. | 0xffffffff |

**Indirect Write Transfer Start Address Register**

This register should be setup before an indirect write transfer is triggered.

| Offset | Bit  | R/W | Description                                                                                                                   | Reset      |
|--------|------|-----|-------------------------------------------------------------------------------------------------------------------------------|------------|
| 0x78   | 31:0 | R/W | Start of Indirect Access<br>This is the FLASH start address from which the indirect access will commence its write operation. | 0x00000000 |

**Indirect Write Transfer Number Bytes Register**

This register should be setup before an indirect write transfer is triggered.

| Offset | Bit  | R/W | Description                                                                                                                                         | Reset      |
|--------|------|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| 0x7C   | 31:0 | R/W | Indirect Number of Bytes<br>This is the number of bytes that the indirect access will consume. This can be bigger than the configured size of SRAM. | 0x00000000 |

**Indirect Trigger Address Range Register**

This register should be setup while the controller is idle.

| Offset | Bit  | R/W | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | Reset |
|--------|------|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| 0x80   | 31:4 | RO  | Unused bits. Read as zero.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | 0x0   |
| 0x80   | 3:0  | R/W | Indirect Range Width<br>This is the address offset of Indirect Trigger Address Register. When any valid Indirect Access is triggered and AHB address fits to the range, the request is forwarded into Indirect Write/Read Controller.<br><br>The value is given as power of 2. The default one is $2^4 = 16$ what allows 16-byte bursts to be performed. This field reflects width of the range so number of locations of valid addresses (single location has 32 bits) are shaping from Indirect Trigger Address to Indirect Trigger Address + <Indirect Range Width – 1> (Indirect Trigger | 0x4   |

|  |  |  |                                                         |  |
|--|--|--|---------------------------------------------------------|--|
|  |  |  | Address -> Indirect Trigger Address + 15 [by default]). |  |
|--|--|--|---------------------------------------------------------|--|

**Flash Command Control Register (Using STIG)**

| Offset | Bit   | R/W | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | Reset |
|--------|-------|-----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| 0x90   | 31:24 | R/W | <p>Command Opcode</p> <p>The command opcode field should be setup before triggering the command. For example, 0x20 maps to SubSector Erase. Writing to the execute field (bit 0) of this register launches the command. NOTE: Using this approach to issue commands to the device will make use of the instruction type of the device instruction configuration register. If this field is set to 2'b00, then the command opcode, command address, command dummy bytes and command data will all be transferred in a serial fashion. If this field is set to 2'b01, then the command opcode, command address, command dummy bytes and command data will all be transferred in parallel using DQ0 and DQ1 pins. If this field is set to 2'b10, then the command opcode, command address, command dummy bytes and command data will all be transferred in parallel using DQ0, DQ1, DQ2 and DQ3 pins.</p> | 0x00  |
| 0x90   | 23    | R/W | <p>Read Data Enable</p> <p>Set to '1' if the command specified in bits 31:24 requires read data bytes to be received from the device.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | 0x0   |
| 0x90   | 22:20 | R/W | <p>Number of Read Data Bytes</p> <p>Up to 8 Data bytes may be read using this command (Set to 0 for 1 byte, 7 for 8 bytes).</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | 0x0   |
| 0x90   | 19    | R/W | <p>Command Address Enable</p> <p>Set to '1' if the command specified in bits 31:24 requires an address.</p> <p>This should be setup before triggering the command via bit 0 of this register.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | 0x0   |
| 0x90   | 18    | R/W | <p>Mode Bit Enable</p> <p>Set to '1' to ensure the mode bits as defined in the Mode Bit Configuration register are sent following the address bytes.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | 0x0   |
| 0x90   | 17:16 | R/W | <p>Number of Address Bytes</p> <p>Set to the number of address bytes required (the address itself is programmed in the FLASH COMMAND ADDRESS REGISTERS).</p> <p>This should be setup before triggering the command via bit 0 of this register.</p> <p>00 : 1 address byte<br/>01 : 2 address bytes</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | 0x0   |

|      |       |     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |     |
|------|-------|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
|      |       |     | 10 : 3 address bytes<br>11 : 4 address bytes                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |     |
| 0x90 | 15    | R/W | Write Data Enable<br>Set to '1' if the command specified in bits 31:24 requires write data bytes to be sent to the device                                                                                                                                                                                                                                                                                                                                                                                              | 0x0 |
| 0x90 | 14:12 | R/W | Number of Write Data Bytes<br>Up to 8 Data bytes may be written using this command (Set to 0 for 1 byte, 7 for 8 bytes)                                                                                                                                                                                                                                                                                                                                                                                                | 0x0 |
| 0x90 | 11:7  | R/W | Number of Dummy Bytes<br>Set to the number of dummy bytes required<br>This should be setup before triggering the command via bit 0 of this register.<br><br>NOTE:<br>This field must not be greater than 5'h03. Setting more dummy bytes can cause TX FIFO full condition to be triggered or overlooking some data for read commands and the behaviour of the core is unpredictable.<br><br>Do not confuse dummy cycles with dummy bytes and check carefully how it is defined in specification of using Flash Device. | 0x0 |
| 0x90 | 6:2   | RO  | Unused bits. Read as zero.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | 0x0 |
| 0x90 | 1     | RO  | Command execution in progress.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | 0x0 |
| 0x90 | 0     | WO  | Execute the command.<br>This bit is internally synchronized.                                                                                                                                                                                                                                                                                                                                                                                                                                                           | N/a |

**Flash Command Address Register**

| Offset | Bit  | R/W | Description                                                                                                                                                                                                                       | Reset      |
|--------|------|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| 0x94   | 31:0 | R/W | Command Address<br>This should be setup before triggering the command in bit 0 of the FLASH COMMAND CONTROL register.<br><br>It is the address used by the command specified in bits 31:24 of the FLASH COMMAND CONTROL register. | 0x00000000 |

**Flash Command Read Data Register (Lower)**

| Offset | Bit  | R/W | Description                                                                                                                                                                                                                                                                                    | Reset      |
|--------|------|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| 0xA0   | 31:0 | RO  | <p>Command Read Data (Lower byte)</p> <p>This is the data that is returned by the FLASH device for any status or configuration read operation carried out by triggering the event in the control register. The register will be valid when the polling bit in the control register is low.</p> | 0x00000000 |

**Flash Command Read Data Register (Upper)**

| Offset | Bit  | R/W | Description                                                                                                                                                                                                                                                                                    | Reset      |
|--------|------|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| 0xA4   | 31:0 | RO  | <p>Command Read Data (Upper byte)</p> <p>This is the data that is returned by the FLASH device for any status or configuration read operation carried out by triggering the event in the control register. The register will be valid when the polling bit in the control register is low.</p> | 0x00000000 |

**Flash Command Write Data Register (Lower)**

| Offset | Bit  | R/W | Description                                                                                                                                                                                                                                                                                                                   | Reset      |
|--------|------|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| 0xA8   | 31:0 | R/W | <p>Command Write Data Lower Byte</p> <p>This should be setup before triggering the command in bit 0 of the FLASH COMMAND CONTROL register.</p> <p>This is the data that is to be written to the FLASH device for any status or configuration write operation carried out by triggering the event in the control register.</p> | 0x00000000 |

**Flash Command Write Data Register (Upper)**

| Offset | Bit  | R/W | Description                                                                                                                                                                                                                                                                                                                   | Reset      |
|--------|------|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| 0xAC   | 31:0 | R/W | <p>Command Write Data Upper byte</p> <p>This should be setup before triggering the command in bit 0 of the FLASH COMMAND CONTROL register.</p> <p>This is the data that is to be written to the FLASH device for any status or configuration write operation carried out by triggering the event in the control register.</p> | 0x00000000 |

## Polling Flash Status Register

This register is useful in the auto-polling phase only.

| Offset | Bit  | R/W | Description                                                                     | Reset    |
|--------|------|-----|---------------------------------------------------------------------------------|----------|
| 0xB0   | 31:9 | RO  | Unused bits. Read as zero.                                                      | 0x000000 |
| 0xB0   | 8    | RO  | Polling Status Valid<br>This bit is set when value in bits from 7 to 0 is valid | 0x0      |
| 0xB0   | 7:0  | RO  | Flash Status<br>Defines actual Status Register of Device                        | 0x00     |

## Module ID Register

| Offset | Bit   | R/W | Description                                                                 | Reset    |
|--------|-------|-----|-----------------------------------------------------------------------------|----------|
| 0xFC   | 31:24 | RO  | Fix number related to revision described by 3 LSB of the Module ID register | 0x00     |
| 0xFC   | 23:0  | RO  | Module/Revision ID number                                                   | 0x010400 |

## Performance Characteristics

For the direct read operations, currently QSPI Flash devices support a maximum clock speed of 133 MHz (65 Mbytes/sec). Burst access to QSPI Flash devices incurs approximately 10% overhead.

The basic performance test will be the time to read a 60KB region of FLASH memory. The specific location of this region is not considered, and will remain constant through all test runs.

When using direct access controller and issuing constant bursts of 16 to the controller, the following numbers were measured. No IDLE or BUSY accesses were issued to the controller during the 60KB fetch – it should be noted that in a real system the numbers of these IDLE or BUSY accesses applied to the controller during a large fetch will only affect the overall time for retrieving the data when the AHB data rate is reduced so significantly that it causes the controller to back-pressure – i.e. the average AHB data rate becomes close or less than the FLASH data rate programmed.

### Performance (Direct Access)

| Flash Clock Frequency (MHz) | Flash Instruction used | Controller Used | MAX theoretical FLASH Data-rate (not inclusive of overhead) Mbps | Best case num cycles to fetch and forward to AHB (Theoretical no overhead) | Best case num cycles to fetch and forward to AHB (Actual) |
|-----------------------------|------------------------|-----------------|------------------------------------------------------------------|----------------------------------------------------------------------------|-----------------------------------------------------------|
| 12.5                        | SLOW READ              | DIRECT          | 12.5                                                             | 3932165                                                                    | 3932423                                                   |
| 50                          | SLOW READ              | DIRECT          | 50                                                               | 983040                                                                     | 983112                                                    |
| 100                         | SLOW READ              | DIRECT          | 100                                                              | 491520                                                                     | 491560                                                    |
| 133                         | FAST READ              | DIRECT          | 133                                                              | 369564                                                                     | 369662                                                    |
| 12.5                        | QIO READ               | DIRECT          | 50                                                               | 983040                                                                     | 983208                                                    |
| 50                          | QIO READ               | DIRECT          | 200                                                              | 245760                                                                     | 245808                                                    |
| 100                         | QIO READ               | DIRECT          | 400                                                              | 122880                                                                     | 122908                                                    |
| 133                         | QIO READ               | DIRECT          | 520                                                              | 92391                                                                      | 92428                                                     |
| 66                          | QIO DDR READ           | DIRECT          | 520                                                              | 92391                                                                      | 92438                                                     |

The actual numbers are very close to the theoretical best. This is because the controller will automatically detect that each AHB read request issued is sequential to the last. This is despite the fact the requests themselves are issued in accordance with AHB rules and restrictions which will give indications of non-sequential accesses even when the access is sequential to the last (such as when an access crosses a 1K boundary or when a fixed burst such as INCR16 is completed).

The numbers reported in the table assume the AHB requests were all made in WORD (32-bit) transfers. If the master issuing the requests sent the full bulk transfer using different sized requests, the controller would have separated the full transfer into multiple smaller bursts, a burst split occurring whenever the AHB request size changed. For each split burst, the controller would require to send overhead bytes in the form of opcode+address+dummy bytes.

The equivalent test carried out with XIP mode enabled yields the following results:

#### Performance (XIP Mode)

| Flash Clock Frequency (MHz) | Flash Instruction used | Controller Used | MAX theoretical FLASH Data-rate (not inclusive of overhead) Mbps | Best case num cycles to fetch and forward to AHB (Theoretical no overhead) | Best case num cycles to fetch and forward to AHB (Actual) |
|-----------------------------|------------------------|-----------------|------------------------------------------------------------------|----------------------------------------------------------------------------|-----------------------------------------------------------|
| 12.5                        | SLOW READ              | DIRECT          | 12.5                                                             | 3932165                                                                    | -                                                         |
| 50                          | SLOW READ              | DIRECT          | 50                                                               | 983040                                                                     | -                                                         |
| 100                         | SLOW READ              | DIRECT          | 100                                                              | 491520                                                                     | -                                                         |
| 133                         | FAST READ              | DIRECT          | 133                                                              | 369564                                                                     | -                                                         |
| 12.5                        | QIO READ               | DIRECT          | 50                                                               | 983040                                                                     | 983145                                                    |
| 50                          | QIO READ               | DIRECT          | 200                                                              | 245760                                                                     | 245793                                                    |
| 100                         | QIO READ               | DIRECT          | 400                                                              | 122880                                                                     | 122901                                                    |
| 133                         | QIO READ               | DIRECT          | 520                                                              | 92391                                                                      | 92422                                                     |
| 66                          | QIO DDR READ           | DIRECT          | 520                                                              | 92391                                                                      | 92437                                                     |

The results are almost the same as the non-XIP data. This is because the instruction opcode itself is only sent once through the entire bulk transfer. For the XIP test, this means a reduction in time of only 8 cycles.

Note that Basic READ instruction cannot operate in XIP mode and so is omitted.

When using 'Indirect' mode, the results are also very close to the theoretical best. This is because in indirect mode the accesses sent to the FLASH device are handled internally by the controller and are always issued sequential throughout the block fetch, meaning there is no requirement from the AHB interface to maintain maximum performance at this stage. Even less much less overhead and the data is fetched from FLASH into SRAM quicker. There is extra overhead required to move the data from the local SRAM to the external AHB master requesting the data.

The time to read the data from FLASH memory using an indirect access and push this to SRAM is as follows:

**Performance (Indirect Access)**

| Flash Clock Frequency (MHz) | Flash Instruction used | Controller Used | MAX theoretical FLASH Data-rate (not inclusive of overhead) Mbps | Best case num cycles to fetch and forward to AHB (Theoretical no overhead) | Best case num cycles to fetch and forward to SRAM (Actual) |
|-----------------------------|------------------------|-----------------|------------------------------------------------------------------|----------------------------------------------------------------------------|------------------------------------------------------------|
| 12.5                        | SLOW READ              | INDIRECT        | 12.5                                                             | 3932165                                                                    | 3932425                                                    |
| 50                          | SLOW READ              | INDIRECT        | 50                                                               | 983040                                                                     | 983113                                                     |
| 100                         | SLOW READ              | INDIRECT        | 100                                                              | 491520                                                                     | 491561                                                     |
| 12.5                        | QIO READ               | INDIRECT        | 50                                                               | 983040                                                                     | 983203                                                     |
| 50                          | QIO READ               | INDIRECT        | 200                                                              | 245760                                                                     | 245803                                                     |
| 100                         | QIO READ               | INDIRECT        | 400                                                              | 122880                                                                     | 122903                                                     |

The time taken to fetch this data from SRAM and forward to AHB is system dependent and not included in this table. However, the QSPI controller will only insert wait states to the AHB when fetching this data from SRAM when the SRAM itself is busy being accessed. This is dependent on how many other ports are trying to access the SRAM. If the indirect write operation is idle, at worst, this is every 8 AHB clocks. Obviously this is very system dependent on the number of IDLE/BUSY accesses sent to the controller.

## Section 3 - Integrating the QSPI\_FLASH\_CTRL

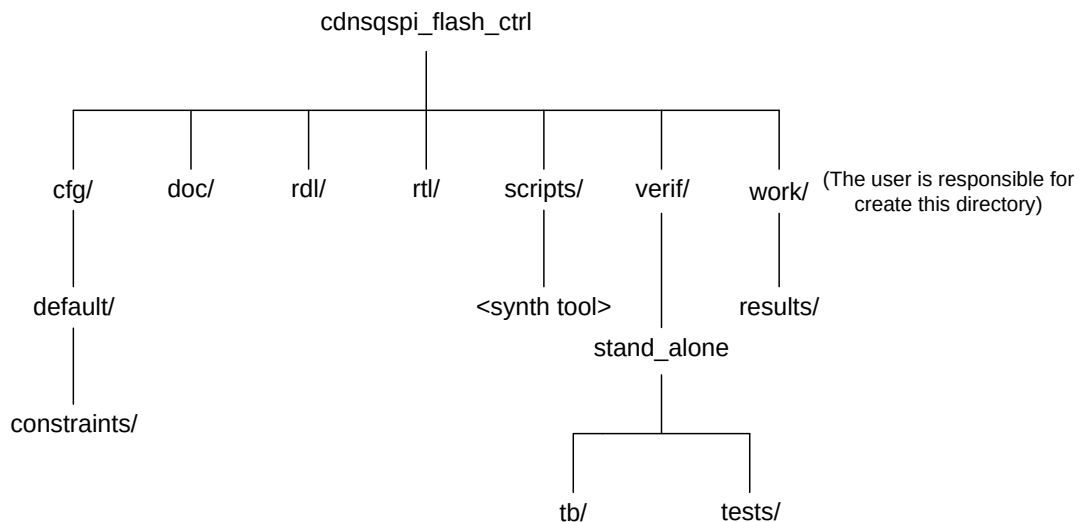
### Introduction

This section enables the QSPI\_FLASH\_CTRL module to be fully integrated into a SoC design. It covers the following topics:

- IP Content
- Implementation Application Notes
- Verification and Test Application

### IP Content

The following diagram illustrates the IP content directory structure.



### Reference Environment

#### Tools

- |                          |                                      |
|--------------------------|--------------------------------------|
| • RTL Simulator:         | Cadence Incisive Simulator 14.10.002 |
| • Code Coverage          | Cadence Incisive Simulator 14.10.002 |
| • Logic & Test Synthesis | Cadence RTL Compiler 14.10.000       |
| • Perl                   | v5.6.1 built for i686-linux          |

### Implementation Application Notes

#### Micron N25Q Family of Devices

Micron N25Q devices  $\geq 512\text{Mb}$  in size contain a new register, the Flag Status Register, which requires to be polled in order to determine whether the program or erase controller bit is Busy or Ready. The QSPI controller has an automatic method to poll the device for write completion and it does this by default polling bit 0 of the device STATUS register using opcode 0x05. This is the write in progress (or WIP as it is referred in many device datasheets).

For these larger Micron devices however, the FLAG STATUS register needs to be polled instead. This can be achieved by carefully programming the write completion control register (refer to programming section of this document, register at offset 0x38).

At the time of writing no other Micron parts or known parts from any other vendor require a change to the default value of the write completion control register (offset 0x38).

## Micron MT25Q Family of Devices

Micron MT25Q devices can achieve the clock frequency up to 133 MHz (in STR) and 66 MHz (in DTR) what results in throughput up to 65 MB/s. Transfer rate has been increased comparing with N25Q ones which can handle data rate of 54 MB/s.

Adding support for DTR protocol is another significant enhancement. It can be activated by volatile or non-volatile way. It allows device to work in DTR Mode for each possible command (not only DTR READ ones). Additional benefit over this feature is possibility to handle OPCODE phase in DTR Mode what improves overall performance.

## Clocking

There are four clock sources for the QSPI Controller and one generated clock. These are described individually below.

### AHB Clock (hclk)

The AHB clock is the main system clock used to transfer data over the AHB bus between an external master and the QSPI controller. When the reference clock is included (the use of `ref_clk` is optional and controlled via the verilog define file `cdnsqspi_flash_ctrl_defs.v`), this clock is only used at the front end of the QSPI controller and must be fast enough to guarantee the overall bandwidth can be met. I.e. there is a minimum frequency, but no maximum. The maximum allowed frequency is only limited by the speed of the technology used and the required clock relationships as defined in the next section. When the reference clock is not used, this clock will be used to both transfer data over the AHB bus and also to transfer data to and from the external flash device. In this mode, the frequency is linked directly (via the baud rate divider) to the required FLASH clock rate.

The QSPI controller only uses the rising edge of this clock and there are therefore no duty cycle requirements. The use of the AHB clock in the design has been implemented in a way that means it can be considered asynchronous to all the other input clocks.

### APB Clock (pclk)

The APB clock is used to access the register map of the QSPI controller, perform controller and device configuration, service interrupts and control certain run time modes. The allowed frequency is limited by the speed of the technology used and the required clock relationships. It is acceptable, but not a requirement, for this signal to be tied to `hclk`. The QSPI controller only uses the rising edge of this clock and has therefore no duty cycle requirements. The use of the APB clock in the design has been implemented in a way that means it can be considered asynchronous to all the other input clocks.

### Reference Clock (ref\_clk)

The reference clock is used to serialize the data and drive the external SPI interface. It decouples the system clocks (AHB and APB) from the SPI device. The allowed frequency is limited by the speed of the technology used and the required clock relationships. The QSPI controller generally uses the rising edge of this clock and there are therefore no duty cycle

requirements. When `BOOT_EDGE_SAMPLING` parameter is uncommented, the controller has also falling edge flops which only mediate in sampling data from device and do not any further impact on control logic of the core. The use of the reference clock in the design has been implemented in a way that means it can be considered asynchronous to all the other input clocks. This clock can range from 0-800 MHz, however, to handle upper half of its range additional timing constraints may be necessarily to add (depending on actual system requirements). The selected frequency is linked directly (via the baud rate divider) to the required FLASH clock rate.

### Loopback Clock (`spi_dlyd_i`)

This clock is used for prolonging the read data window to ensure more valid samples for tap data capturing mechanism. It comes from Delay Line located external for the controller. This clock is optional and the input may be set to high if loopback feature is not intended to using. Generation path for this clock was presented in the clock architecture diagram. More detailed description of mechanism using this clock is included in Read Data Capturing Section.

### Generated Clock (`sclk_out`)

The `sclk_out` pin is used as the clock source of external SPI slave FLASH devices. As far as the QSPI controller is concerned, the generated clock is not a clock, but a `ref_clk` timed output from the registers block. For synthesis, it will be treated as a data output of the core. It is synchronous to the reference clock, divided down via the internal baud rate dividers. The master mode baud rate divisor allows division by any even number up to 32 as follows:

| Divisor | Baud Rate |
|---------|-----------|
| 0000    | /2        |
| 0001    | /4        |
| 0010    | /6        |
| 0011    | /8        |
| 0100    | /10       |
| 0101    | /12       |
| 0110    | /14       |
| 0111    | /16       |
| ...     | ...       |
| 1111    | /32       |

The `baud_rate` along with the polarity and phase settings (CPOL and CPHA) for this clock are programmable bits in the QSPI Configuration register (0x00).

### Clock Relationships

The AHB, APB and reference clock can be considered asynchronous to one another, although there will be some constraints that must be adhered to with respect to minimum frequencies.

The `hclk` clock frequency must be chosen such that the bandwidth requirements of the required data rates can be met:

The reference clock frequency must be at least 4X the generated memory clock frequency (for SDR commands) and at least 8X (for DDR commands).

### Timing and Load Parameters

For all internal interfaces (including AHB and APB), all inputs must be setup a minimum of 40% of the clock period before the rising edge of the related clock and all outputs will be valid at a maximum of 40% of the clock period after the rising edge of the related clock. Given that there are combinational input-output paths over SRAM interface, in order to minimize latency, these requirements could be slightly tighten (especially for high `ahb_clk`

value). If Static Timing Analysis failed for these paths – we recommend to consider keep `sram_cs` signal continuously enabled and treat de-asserted `sram_we` as `sram_re`. In this case these paths could be specified as false ones.

## Designing Across Clock Boundaries

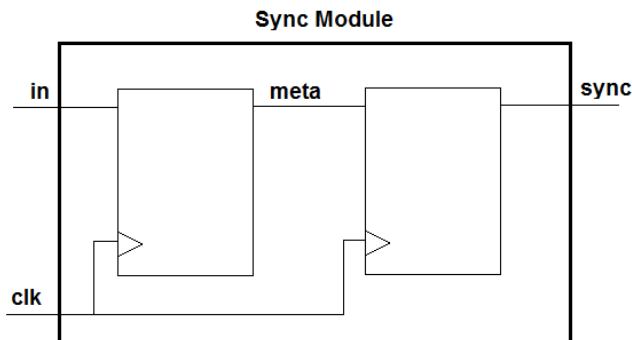
This section defines the rules governing designing across the 3 asynchronous clock boundaries (`ref_clk`, `hclk` and `pclk`).

### Static Configuration

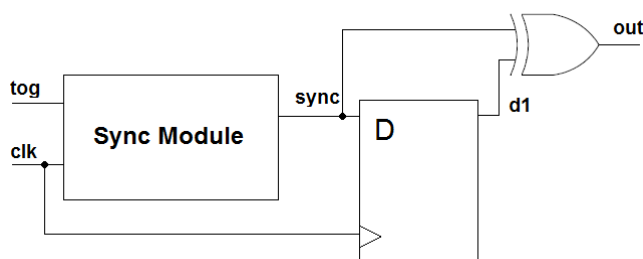
Global static configuration signals driven in the APB clock domain while the QSPI controller is disabled, or configuration signals specific to a certain mode of operation that are driven while that mode is disabled do not need to be synchronized.

### Metastability

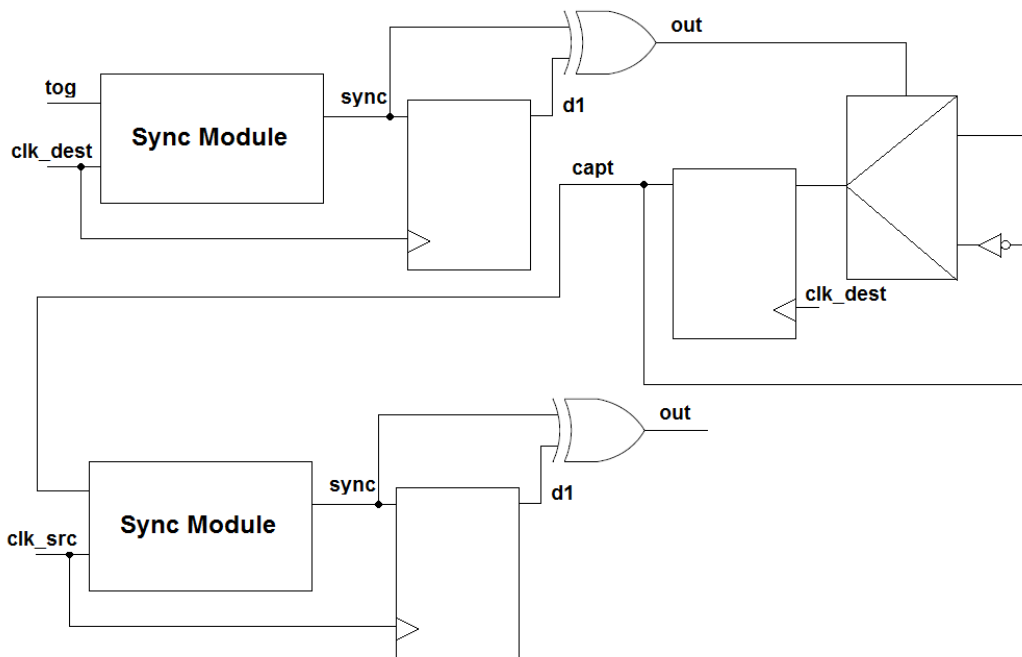
To address the problem of metastability, all critical signals not covered specifically in this document that pass between clock domains are synchronised by passing through at least two ganged flip-flops. Because `hclk`, `pclk` and `ref_clk` can all be asynchronous with respect to each other. Two stages synchronizer is randomized for verification purposes.



Some group of signals are synchronized with “enable pulse synchronization mechanism”. Input signal toggles between occurrences.



For signals which require acknowledge signal from destination domain before unblocking for toggling synchronization scheme is extended.



### Data Path Transfer Between `ref_clk` and `hclk`

The legacy SPI core handles the key clock boundary between the AHB domain and the reference clock domain. This is part of the existing silicon proven IP and is handled using carefully implemented transmit and receive FIFO's and handshaking logic using the standard metastability rules.

### Reset

There are three asynchronous resets present in the design, one for each of the main clock domains (`ref_clk`, `hclk` and `pclk`). These resets are used directly to reset the synchronous parts of the design. The reset signals may be set low asynchronously with respect to the clock, but must be set high synchronously.

### Verification and Test Application

#### Testbench structure

There is one supplied verilog based test-bench in the `verif/stand_alone` directory. It has been developed around the RTL description, and has been used with a synthesised net-list (not provided). Numerous tests are provided with the test bench.

This verilog based verification environment uses memory models downloaded from various memory manufacturers. These cannot be delivered to IP customers due to copyright restrictions and therefore must be downloaded separately from the manufacturer's website if the user wishes to rerun tests on the delivered testbench.

The following models were used in this environment. Note that these models may have been updated or no longer available since they were used initially to help verify the QSPI design IP.

1. Atmel AT25DQ161.v - Note Atmel memory have been bought by Adesto and so this model should be requested by Adesto.

2. Atmel AT25DQ321.v - Note Atmel memory have been bought by Adesto and so this model should be requested by Adesto.
3. Micron N25Q1024A13E.v
4. Micron N25Qxxx.v (MT25QU128)
5. Spansion S25FL129P.v
6. Spansion s25fs512s.v

Since QSPI Flash Devices generally keep their backward compatibility over the interface, integration of emerging devices not yet connected into testbench is not very time consuming task. The Models are being developed by third party vendors therefore Cadence cannot guarantee that existing tests get passed state even for updated versions of the same devices from the list pointed above. Delivered test cases should be treated as functional guideline and good template for particular user scenarios.

Simulation must be performed in the `work` directory, which must contain a `results` and a `files` subdirectory. These are created automatically by the delivered scripts.

Test scenarios are stored in the `verif/stand_alone/tests_<configuration>` directories; these can be used as a starting point for development of further test scenarios. These test scenarios are translated using a Perl program called `trans.pl`, stored in the `verif/stand_alone/tb` directory. The output is test stimulus and comparison data stored in the `work/files` directory.

Simulation scripts are supplied in the `scripts/sim` directory; these are configured to use the Cadence Incisive NC Verilog Simulator tool.

To run the simulation scripts an environment variable called `$KITSLIB` must be initialised. This is used during gate level simulations to point to the required libraries. If only performing RTL simulations `$KITSLIB` can be set to a dummy value.

To simulate the test scenarios, the RTL and testbench code must first be compiled, this is accomplished using the `compile.pl` script held in the `scripts/sim` directory and again assumes that Cadence Incisive NC Verilog Simulator has been properly installed.

For example from the `work` directory use:

```
../scripts/sim/compile.pl
```

Once the RTL and testbench code has been compiled, the `simulate.pl` script should be called. To run all the tests, call the `simulate.pl` script with the option `-all`, otherwise use `-run [test case]`.

For example from the `work` directory use:

```
../scripts/sim/simulate.pl -run basic_rw
```

For further command switch options, the `-help` command switch is available.

Note that all the `runscripts` are designed to be run from the `work` directory and execute with `../scripts/[script name]`. If the `work` directory is not the current directory, an error message will be displayed.

Tesbench setting up example:

- a) It is assumed that Cadence Incisive NC Verilog Simulator has been properly installed and design has been unpacked from delivered tar.
- b) type (from console window): `cd cdnsqspi_flash_ctrl/`
- c) create "work" directory: `mkdir work`
- d) type: `cd work`
- e) try to run compile script: `../scripts/run/compile.pl`

- f) You will get following errors (Memory Models are missing):
- ```
ncvlog: *E,COFILE: cannot open source file
'../verif/stand_alone/tb/../../QSPI_Memory_Models/AT25DQ321.v'
ncvlog: *E,COFILE: cannot open source file
'../verif/stand_alone/tb/../../QSPI_Memory_Models/S25FL129P.v'
ncvlog: *E,COFILE: cannot open source file
'../verif/stand_alone/tb/../../QSPI_Memory_Models/Micron/N25Q1024A13E.v'.
```
- g) Download Models from network (Micron ones)
- Go to: [www.micron.com](http://www.micron.com)
  - Register and log in there
  - Go to: <https://www.micron.com/products/nor-flash/serial-nor-flash> and select any device from the list
  - Click on device density and then name i.e. 2Gb/[MT25QL02GCBA8E12-0SIT](https://www.micron.com/products/nor-flash/serial-nor-flash/MT25QL02GCBA8E12-0SIT)
  - Find the Verilog model and download it
- h) Download Models from network (Spansion ones)
- Go to: <http://www.spansion.com/Products/memory/Serial-Flash/Pages/Spansion%20FL.aspx>
  - Click on “design model” card and download any model from the list
- i) Download Models from network (Adesto ones)
- Go to: <http://www.adeptotech.com/simulation-tools/> and select any verilog model from the list i.e. AT45DQ161
- j) It is not mandatory to instantiate more than one memory model. Example shows how to run the simulation with have just downloaded Spansion S25FL512S Model. Integration of another ones is equivalent.
- k) Create QSPI\_Memory\_Models directory within  
cdnsqspi\_flash\_ctrl/verif/stand\_alone
- l) Copy all files (Spansion S25FL512S has just one Verilog file) from downloaded package into  
cdnsqspi\_flash\_ctrl/verif/stand\_alone/QSPI\_Memory\_Models
- m) Open in editor: cdnsqspi\_flash\_ctrl/verif/stand\_alone/tb/cdnsqspi\_flash\_ctrl.v and exclude all Memory Models from the project:

```
// ATMEL AT25DQ321 FLASH MODEL - 32Mbit
/* AT25DQ321 i_atmel_flash_model (
    .CSB      (n_ss_out[0]),
    .SCK      (sclk_out),
    .SI       (flash_si[0]),
    .WPB      (flash_wpn[0]),
    .SO       (flash_so[0]),
    .HOLDB    (flash_holdn[0]),
    .VCC      (1'b1),
    .GND      (1'b0)
);*/

// SPANSION S25FL129P 128Mbit Flash Model
/* s25fl129p00 i_s25fl129p_spansion_flash_model (
    .SCK      (sclk_out),
    .SI       (flash_si[1]),
```

```

        .CSNeg      (n_ss_out[1]),
        .HOLDNeg    (flash_holdn[1]),
        .WPNeg      (flash_wpn[1]),
        .SO          (flash_so[1])
    );*/

/*
    // Micron N25Q128 128Mbit Flash Model
    N25Q128 i_N25Qxxx_micron_flash_model (
        .C          (sclk_out),
        .DQ0         (flash_si),
        .S           (n_ss_out[2]),
        .HOLD_DQ3    (flash_holdn),
        .Vpp_W_DQ2   (flash_wpn),
        .DQ1         (flash_so),
        .Vcc          (32'hffffffff)
    );
    */

    // Micron N25Q1024 1Gbit Flash Model
    /* N25QxxxTop i_N25Qxxx_micron_flash_model (
        .C          (sclk_out),
        .DQ0         (flash_si[2]),
        .S           (n_ss_out[2]),
        .HOLD_DQ3    (flash_holdn[2]),
        .Vpp_W_DQ2   (flash_wpn[2]),
        .DQ1         (flash_so[2]),
        .Vcc          (32'hffffffff)
    );*/

```

- n) Instantiate downloaded model instead of removed ones.

```

s25fs512s i_s25fs512s_spansion_flash_model (
    .SCK      (sclk_out),
    .SI       (flash_si[1]),
    .CSNeg    (n_ss_out[1]),
    .RESETNeg (flash_holdn[1]),
    .WPNeg    (flash_wpn[1]),
    .SO       (flash_so[1])
);

```

- o) Add new needed files and remove non using models from  
cdnsqspi\_flash\_ctrl/verif/stand\_alone/tb/  
cdnsqspi\_flash\_ctrl.f

```

../QSPI_Memory_Models/AT25DQ321.v
../QSPI_Memory_Models/S25FL129P.v
../QSPI_Memory_Models/Micron/N25Q1024A13E.v
../QSPI_Memory_Models/s25fs512s.v

```

- p) try to run compile script from work directory: ../scripts/run/compile.pl  
q) Compilation should finish without errors

- r) run one of the test cases in GUI: `../scripts/run/simulate.pl -run <file_name> -gui`
- s) Press F2 to run simulation after ncsim GUI environment is loaded

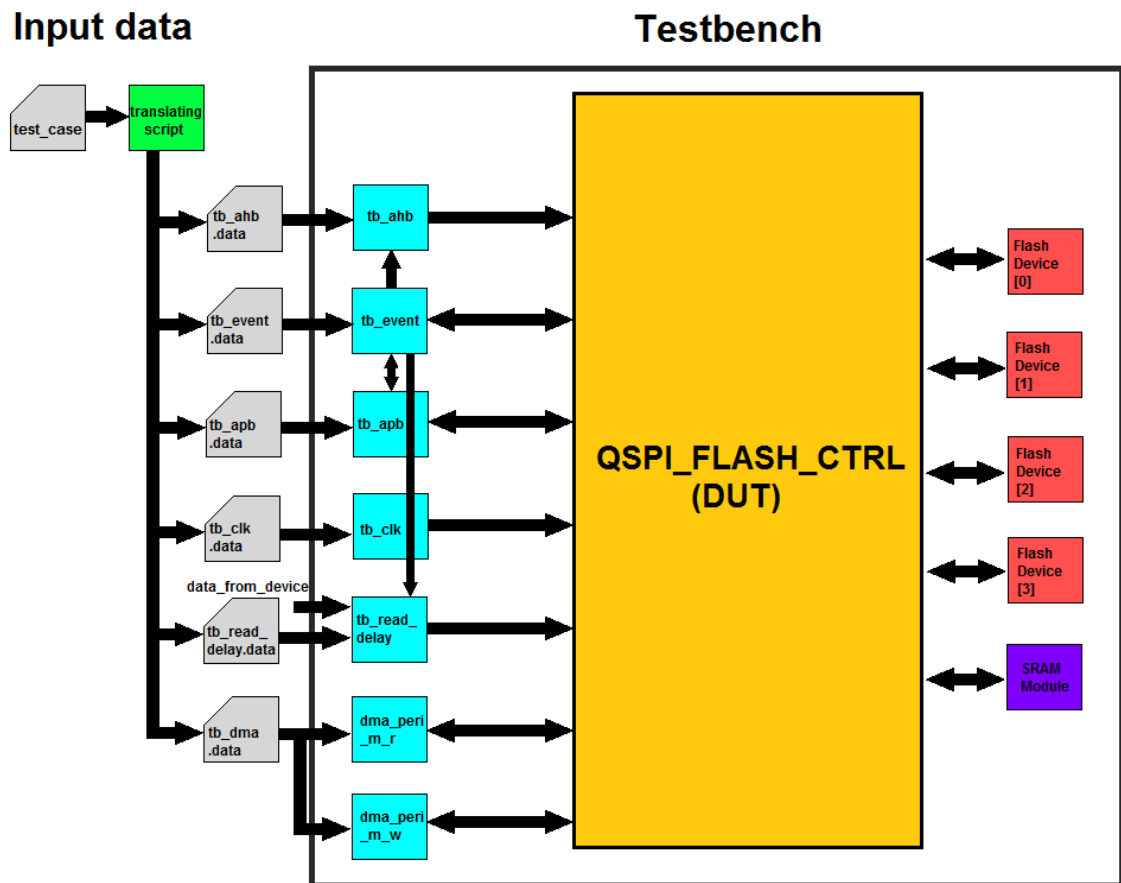
The delivered scripts have `-man` option which allows to enroll all another switches within particular script i.e. `../scripts/run/simulate.pl -man`

After a test scenario has been simulated a brief pass or fail status is written to the `results` directory. This status is written to a file with the same name as the test scenario with an extension of `.res`. A summary of previous simulation results can be returned by executing the Unix command `cat results/*` from the `work` directory.

NOTE: If user is not using the provided compilation script, then he should change name of the `cdnsqspi_flash_ctrl_defs_default.v` file to `cdnsqspi_flash_ctrl_defs.v` before run compilation.

## Testbench block description

### Testbench Block Diagram



#### “test\_case” block

This block represents file with test case selected to being run.

#### “translating\_script” block

This block represents the perl script file which translates tb commands into data chain files.

**“\*.data” blocks**

These files contain separated data chains for particular command groups described later.

**“tb\_ahb” block**

This block translates data chain into actual AHB Interface.

**“tb\_apb” block**

This block translates the data chain into actual APB Interface. It communicates with the controller in both directions. This is caused by the fact that interrupt logic (generated by the controller) was implemented in APB domain.

**“tb\_event” block**

This block controls simulation time of all interface stimuli scheduled to execute within the test case.

**“tb\_read\_delay” block**

This block emulates additional read data delay between the Flash Device data outputs and the controller data inputs.

**“dma\_peri\_m\_r” block**

This block emulates external DMA Module Interface for Indirect Read Mode.

**“dma\_peri\_m\_w” block**

This block emulates external DMA Module Interface for Indirect Write Mode.

**“QSPI\_FLASH\_CTRL (DUT)” block**

Device under test.

**“Flash Device [3:0]”**

Memory Models from third party vendors.

**“SRAM Module”**

Verification Model of SRAM.

## Testbench syntax description

### APB Interface commands

#### APB write

Syntax:

```
apbwr(<simulation time>) <address> <data>
```

Description:

Address parameter is size of 1 byte and data parameter is size of 4 bytes. Both need to be given in hex base. Simulation time is calculated based on configured resolution of it. When simulation time parameter is not specified, valid access is queued to handle just after the last one from actual clock domain.

Example:

```
apbwr(1000) 00 00080481 --Write data into configuration register
```

#### APB read

Syntax:

```
apbrd(<simulation time>) <address> <expected_data>
```

Description:

Address parameter is size of 1 byte and data parameter is size of 4 bytes. Both need to be given in hex base. Simulation time is calculated based on configured resolution of it. When simulation time parameter is not specified, valid access is queued to handle just after the last one from actual clock domain. If data was not predicted, it would be filled with "xxxxxxx".

Example:

```
apbrd(1000) 00 00080481 --Read configuration register and check its value
```

### AHB Interface commands

#### AHB write

Syntax:

```
ahbwr(<simulation time>) <hburst> <htrans> <hsize> <address> <data>
```

Description:

First three parameters (hburst, htrans and hsize) need to be given in dec base. Both Address parameter and data parameter are size of 4 bytes. They need to be given in hex base. Simulation time is calculated based on configured resolution of it. When simulation time

parameter is not specified, valid access is queued to handle just after the last one from actual clock domain.

Example:

```
ahbwr(1000) 3 3 2 00000000 12345678 --Perform sequential write 4-byte burst access
```

### AHB read

Syntax:

```
ahbrd(<simulation time>) <hburst> <htrans> <hsize> <address> <data>
```

Description:

First three parameters (hburst, htrans and hsize) need to be given in dec base. Both Address parameter and data parameter are size of 4 bytes. They need to be given in hex base. Simulation time is calculated based on configured resolution of it. When simulation time parameter is not specified, valid access is queued to handle just after the last one from actual clock domain. If data was not predicted, it would be filled with "xxxxxxx".

Example:

```
ahbrd(1000) 3 3 2 00000000 12345678 --Perform sequential read 4-byte burst access and check correctness of expected read data
```

### AHB bulk transfer commands

#### AHB bulk write

Syntax:

```
do_bulk_write(<simulation time>) <no_of_AHB_words> <address> <data>  
1/<no_of_wait_states>
```

Description:

This command handles large data transfers and calculates their performance. Interface values are automatically set as following:

hburst = 1, htrans = 2, hsize = 2

Address parameter refers to initial address. Data parameter can be set as following:

incrementing, decrementing, <size\_of\_4\_bytes\_hex\_value>

Simulation time is calculated based on configured resolution of it. When simulation time parameter is not specified, valid access is queued to handle just after the last one from actual clock domain.

Examples:

```
do_bulk_write(1000) 1024 00000000 ffffffff 1/1  
do_bulk_write(2000) 1024 00000000 incrementing 1/1
```

**AHB bulk read**

Syntax:

```
do_bulk_read(<simulation time>) <no_of_AHB_words> <address> <data>
1/<no_of_wait_states>
```

Description:

This command handles large data transfers and calculates their performance. Interface values are automatically set as following:

hburst = 1, htrans = 2, hsize = 2

Address parameter refers to initial address. Data parameter can be set as following:

incrementing, decrementing, <size\_of\_4\_bytes\_hex\_value>

Simulation time is calculated based on configured resolution of it. When simulation time parameter is not specified, valid access is queued to handle just after the last one from actual clock domain.

Examples:

```
do_bulk_read(1000) 1024 00000000 ffffffff 1/1
do_bulk_read(2000) 1024 00000000 decrementing 1/1
```

**Other commands****Data path delay**

Syntax:

```
set_read_data_path_delay(<simulation time>) <delay_value>ns
```

Description:

This command models additional delay of data from device.

Example:

```
set_read_data_path_delay(100) 3ns --calculated data path delay for
individual purpose is different than one returned by Simulation Model of
device. Therefore adjustment is needed.
```

**DMA interface settings**

Syntax:

```
set_tb_dma <time_in_ns> <drready> <davalid> <datatype>
```

Description:

This command emulates interface signals from external DMA Module. All parameters need to be given in dec base.

Example:

```
set_tb_dma 25200000 0 1 2
```

### **Clock values settings**

Syntax:

```
clk_values <apb_clk> <ahb_clk> <ref_clk>
```

Description:

This command must be the first one in each test and cannot be used more than once within single testcase. It configures clock values. Each parameter should be given in MHz.

Example:

```
clk_values 100 100 400
```

### **Special characters**

Each test should be ended with end(<simulation time>) keyword. Any comments should be preceded by --.

Example:

```
end(10000) -- End of the test
```

### **CPU Based Testing**

No CPU based tests are supplied with this release of the IP.

## Section 4 – Integration Guide

### Installation

All the information needed for a quick start with the QSPI IP are included in Verification and Test Application Section.

### Compilation

Detailed description of usage and run guidelines was included in Verification and Test Application Section.

### Simulation

Detailed description of usage and run guidelines was included in Verification and Test Application Section.

### Synthesis

Target technology for synthesis is TSMC\_28\_HPM.

Before run synthesis please comment the following line ``define MDV` and uncomment the following one ``define synth into cdnsqspi_flash_ctrl_defs_default.v` file.

### Configuration

The configuration contains the QSPI Flash Controller only solution without the PHY Module.

### Asynchronous and unregistered inputs

Because of the asynchronous nature of Serial Flash Devices, `mi_4bits` vector is an unregistered input. It determines Master Input Data from external memory.

### Combinatorial Outputs

There are combinational outputs of QSPI Flash Controller for SRAM Interface to ensure maximum performance of Indirect transfers. It is recommended to locate the SRAM hard macro close enough to the controller for ensure the timing closure.

### Latches

There are no LATCH registers in this IP.

### SDC Constrains

SDC Constrains file is located in the 'constrains' directory of the project.

## **Contact Information**

### **Cadence Design Foundry**

Cadence Design Foundry, a division of Cadence Design Systems, Inc. is a leading provider of electronic design and supply chain management services. Leading and emerging technology companies around the globe leverage Cadence Design Foundry's engineering services and intellectual property for the design of complex integrated circuits and systems-on-chip (SoC). Cadence is head quartered in San Jose, California.

### **Support**

Support is available for the Qspi Flash Controller (QSPI\_FLASH\_CTRL) under terms agreed in your Design Technology Licence Agreement (or other contract). To get support please contact your local AE, or proceed as otherwise specified in your agreement.